Gestione assenze

1 Introduzione ... 6
1.1 Informazioni sul progetto ... 6
1.2 Scopo ... 6
2 Analisi ... 7
2.1 Analisi del dominio ... 7
2.2 Analisi e specifica dei requisiti ... 8
2.2.1 Spiegazione elementi tabella dei requisiti: ... 16
2.3 Use case ... 17
2.3.1 Use case richieste di congedo ... 17
2.3.2 Use case segnalazione assenze ... 18
2.3.3 Use case segnalazione ritardo ... 19
2.4 Pianificazione ... 20
2.4.1 Sprint 1 ... 21
2.4.2 Sprint 2 ... 21
2.4.3 Sprint 3 ... 21
2.5 Analisi dei mezzi ... 22
2.5.1 Software ... 22
2.5.2 Hardware ... 22
3 Progettazione ... 23
3.1 Design dei dati e database ... 23
3.2 Design delle interfacce ... 26
3.2.1 Interfaccia dashboard allievo ... 26
3.2.2 Interfaccia dashboard docente di classe ... 27
3.3 Design procedurale ... 28
3.3.1 Activity diagram richieste di congedo ... 28
3.3.2 Activity diagram segnalazione assenza ... 29
3.3.3 Activity diagram segnalazione ritardi ... 30
4 Implementazione ... 31
4.1 Struttura generale dell'applicazione ... 31
4.1.1 Cartelle principali del progetto ... 31
4.1.2 Inizializzazione del frontend con Inertia ... 31
4.2 Gestione delle rotte ... 32
4.2.1 Organizzazione delle rotte ... 32
4.2.2 Rotte dell'allievo ... 32
4.2.3 Rotte del docente ... 33
4.2.4 Rotte del capo laboratorio ... 33
4.2.5 Rotte amministrative ... 34
4.2.6 Rotte pubbliche con token per il tutore ... 34
4.2.7 Riepilogo delle rotte principali ... 35
4.3 Gestione ruoli e accesso alle dashboard ... 35
4.3.1 Ruoli previsti dal sistema ... 35
4.3.2 Controllo del ruolo utente ... 36
4.3.3 Reindirizzamento alla dashboard corretta ... 36
4.3.4 Dashboard disponibili ... 36
4.3.5 Protezione delle funzionalità per ruolo ... 38
4.3.6 Accesso del tutore tramite token ... 38
4.4 Layout base e dati condivisi con Inertia ... 39
4.4.1 Scopo del layout principale ... 39
4.4.2 Dati condivisi da Laravel a React ... 40
4.4.3 Serializzazione dell'utente autenticato ... 40
4.4.4 Utilizzo dei dati nel layout React ... 41
4.4.5 Menu laterale in base al ruolo ... 41
4.4.6 Notifiche nel layout ... 41
4.4.7 Ricerca globale ... 42
4.5 Segnalazione assenza da parte dell'allievo ... 43
4.5.1 Scopo della funzionalità ... 43
4.5.2 Apertura della maschera di segnalazione ... 43
4.5.3 Validazione dei dati inseriti ... 44

4.5.4 Controllo di assenze sovrapposte ... 44
4.5.5 Creazione della richiesta di assenza ... 45
4.5.6 Registrazione nello storico e invio firma tutore ... 45
4.5.7 Flusso riassuntivo ... 46
4.6 Upload e gestione certificato medico ... 46
4.6.1 Scopo della funzionalità ... 46
4.6.2 Validazione del file caricato ... 46
4.6.3 Controllo dell'assenza e dei permessi ... 47
4.6.4 Controllo della scadenza ... 47
4.6.5 Prevenzione di certificati duplicati ... 48
4.6.6 Salvataggio del file nell'archivio studente ... 48
4.6.7 Associazione del certificato all'assenza ... 48
4.6.8 Registrazione nello storico ... 49
4.6.9 Flusso riassuntivo ... 49
4.7 Firma tutore tramite token ... 50
4.7.1 Scopo della funzionalità ... 50
4.7.2 Generazione del token ... 50
4.7.3 Invio dell'email al tutore ... 51
4.7.4 Verifica dello stato del token ... 51
4.7.5 Apertura della pagina di firma ... 52
4.7.6 Salvataggio della firma grafica ... 53
4.7.7 Conferma della firma e utilizzo del token ... 53
4.7.8 Registrazione nello storico ... 54
4.7.9 Flusso riassuntivo ... 54
4.8 Validazione assenza da parte del docente ... 55
4.8.1 Scopo della funzionalità ... 55
4.8.2 Accesso alle assenze della propria classe ... 55
4.8.3 Approvazione dell'assenza ... 56
4.8.4 Approvazione senza firma del tutore ... 57
4.8.5 Rifiuto dell'assenza ... 57
4.8.6 Modifica dei dati dell'assenza ... 58
4.8.7 Validazione del certificato medico ... 58
4.8.8 Registrazione nello storico ... 59
4.8.9 Flusso riassuntivo ... 59
4.9 Gestione ritardi ... 60
4.9.1 Scopo della funzionalità ... 60
4.9.2 Validazione dei dati del ritardo ... 60
4.9.3 Apertura della maschera ritardo ... 60
4.9.4 Creazione della segnalazione ritardo ... 61
4.9.5 Registrazione nello storico ... 61
4.9.6 Validazione del ritardo da parte del docente ... 62
4.9.7 Registrazione del ritardo nel conteggio ... 62
4.9.8 Approvazione senza firma del tutore ... 62
4.9.9 Applicazione delle regole sui ritardi ... 63
4.9.10 Accesso solo ai ritardi della propria classe ... 63
4.9.11 Flusso riassuntivo ... 64
4.10 Gestione congedi ... 65
4.10.1 Scopo della funzionalità ... 65
4.10.2 Validazione della richiesta di congedo ... 65
4.10.3 Controllo di congedi sovrapposti ... 66
4.10.4 Creazione della richiesta ... 66
4.10.5 Upload della documentazione ... 67
4.10.6 Invio firma tutore ... 67
4.10.7 Gestione da parte del capo laboratorio ... 67
4.10.8 Approvazione del congedo ... 68
4.10.9 Registrazione del congedo come assenza ... 68
4.10.10 Rifiuto del congedo ... 68
4.10.11 Flusso riassuntivo ... 69

4.11 Invio email e notifiche interne ... 70
4.11.1 Scopo della funzionalità ... 70
4.11.2 Invio notifiche interne ... 70
4.11.3 Invio a più destinatari ... 70
4.11.4 Risoluzione dei destinatari ... 71
4.11.5 Scelta del canale di notifica ... 71
4.11.6 Contenuto della notifica interna ... 72
4.11.7 Invio email specifiche ... 72
4.11.8 Oggetto e contenuto dell'email ... 73
4.11.9 Tracciamento degli invii email ... 73
4.11.10 Flusso riassuntivo ... 74
4.12 Archiviazione documenti per studente ... 75
4.12.1 Scopo dell'archiviazione ... 75
4.12.2 Struttura dell'archivio ... 75
4.12.3 Categorie dei documenti ... 75
4.12.4 Costruzione del percorso di salvataggio ... 76
4.12.5 Salvataggio di un file caricato dall'utente ... 76
4.12.6 Generazione del nome del file ... 77
4.12.7 Prevenzione dei file duplicati ... 77
4.12.8 Archiviazione dei report mensili ... 78
4.12.9 Esempi di utilizzo dell'archivio ... 78
4.12.10 Flusso riassuntivo ... 78
4.13 Report mensili ... 79
4.13.1 Scopo del report mensile ... 79
4.13.2 Raccolta dei dati del report ... 79
4.13.3 Generazione dei report per gli studenti ... 80
4.13.4 Creazione del PDF ... 81
4.13.5 Dati inseriti nel PDF ... 81
4.13.6 Archiviazione del report ... 82
4.13.7 Invio email del report ... 82
4.13.8 Upload del report firmato ... 83
4.13.9 Approvazione del report ... 83
4.13.10 Generazione tramite comando ... 84
4.13.11 Tabella riassuntiva ... 84
4.14 Comandi automatici e scheduler ... 85
4.14.1 Scopo dei comandi automatici ... 85
4.14.2 Pianificazione nello scheduler ... 85
4.14.3 Generazione automatica dei report mensili ... 86
4.14.4 Uso dei job per i report mensili ... 87
4.14.5 Comando per assenze arbitrarie ... 87
4.14.6 Comando per ritardi scaduti ... 88
4.14.7 Pulizia dei log ... 88
4.14.8 Aggiornamento tutore al compimento dei 18 anni ... 89
4.14.9 Tabella riassuntiva ... 89
4.15 Script Python di supporto ... 90
4.15.1 Scopo dello script Python ... 90
4.15.2 Analisi del PDF ... 90
4.15.3 Riconoscimento dei colori ... 90
4.15.4 Conversione delle celle in date ... 91
4.15.5 Output JSON dello script ... 91
4.15.6 Collegamento con Laravel ... 92
4.15.7 Importazione nel database ... 92
4.16 Storiche operazioni e log ... 94
4.16.1 Tabella riassuntiva ... 94
4.16.2 Storico operazioni e log ... 95
4.16.3 Scopo dello storico ... 95
4.16.4 Struttura della tabella dei log ... 95
4.16.5 Registrazione di un'operazione ... 96

4.16.6 Esempio di log applicativo ... 96
4.16.7 Traduzione delle azioni ... 97
4.16.8 Visualizzazione dei log ... 97
4.16.9 Filtri di ricerca ... 98
4.16.10 Notifica automatica agli amministratori ... 98
4.16.11 Esportazione dei log ... 99
4.16.12 Retention e pulizia automatica ... 100
5 Test ... 101
5.1 Protocollo di test ... 101
5.2 Risultati test ... 116
5.3 Mancanze/limitazioni conosciute ... 116
6 Consuntivo ... 117
7 Conclusioni ... 118
7.1 Sviluppi futuri ... 118
7.2 Considerazioni personali ... 118
8 Glossario ... 119
9 Bibliografia ... 120
9.1 Sitografia ... 120
10 Allegati ... 120

1 Introduzione

1.1 Informazioni sul progetto

- Allievo: Alan Gregorio
- Docente: Ingrid Cereda
- Scuola e classe: SAMT Sezione informatica, I4AA
- Data inizio consegna: 26.01.2026 – 24.04.2026

1.2 Scopo

Dal punto di vista didattico, il progetto ha l'obiettivo di applicare in modo pratico le competenze acquisite durante la formazione, in particolare nell'ambito dello sviluppo di applicazioni web, della progettazione di database e dell'analisi dei requisiti. Il lavoro permette inoltre di affrontare un progetto strutturato, dalla fase di analisi fino all'implementazione e alla documentazione.

Dal punto di vista operativo, il progetto mira a realizzare un sistema web che supporti in modo digitale e centralizzato la gestione di assenze e ritardi in ambito scolastico. L'obiettivo è migliorare l'organizzazione, la tracciabilità e l'efficienza dei processi attualmente svolti in modo manuale e cartaceo, offrendo uno strumento facile da usare e accessibile per studenti, docenti e altri attori coinvolti come i tutori legali.

2 Analisi

2.1 Analisi del dominio

Il prodotto verrà utilizzato all'interno della scuola per la gestione di assenze, ritardi e giustificazioni degli studenti. Attualmente queste attività vengono svolte in modo manuale, tramite cartelline cartacee e moduli separati, rendendo complessa la consultazione dello storico e la tracciabilità delle operazioni.

Il sistema dovrà operare in un contesto scolastico strutturato, coinvolgendo studenti, docenti di classe, capi laboratorio e, per gli studenti minorenni, genitori o tutori. Gli studenti necessitano di uno strumento semplice per notificare le assenze e caricare eventuali certificati medici o visualizzare tramite dashboard le assenze, mentre i docenti e capi lab devono poter controllare, gestire e validare assenze e ritardi in modo centralizzato.

Gli utenti possiedono competenze informatiche di base; in particolare gli studenti sono giovani e abituati all'utilizzo di applicazioni web, rendendo adeguata una soluzione digitale accessibile tramite browser.

Nel dominio scolastico esistono regole e convenzioni già definite. Dopo tre giorni consecutivi di assenza è richiesto un certificato medico; inoltre, al superamento delle 40 ore di assenza annuali, il certificato diventa obbligatorio anche senza giorni consecutivi. In assenza di giustificazioni valide, le assenze possono essere considerate non giustificate e comportare l'assegnazione di ore arbitrarie. Per gli studenti minorenni è richiesta la conferma dell'assenza da parte di un genitore o tutore. Anche i ritardi sono soggetti a regole specifiche e devono essere registrati e tracciati.

Per operare correttamente in questo dominio è necessario conoscere le regole scolastiche sulla gestione di assenze e ritardi, i ruoli degli utenti coinvolti e i principi base di tracciabilità delle operazioni amministrative.

2.2 Analisi e specifica dei requisiti

ID: REQ-01	
Nome	Autenticazione utenti
Priorità	1
Versione	1.0
Note	Il sistema deve permettere l'accesso tramite credenziali.
Sotto requisiti	
001	Si necessita una maschera di login
002	Logout
003	Recupero password via email

ID: REQ-02	
Nome	Gestione ruoli
Priorità	1
Versione	1.0
Note	Il sistema deve gestire diversi ruoli utente.
Sotto requisiti	
001	Allievo
002	Docente di classe
003	Capo laboratorio
004	Amministratore

ID: REQ-03	
Nome	Dashboard allievo
Priorità	1
Versione	1.0
Note	L'allievo deve visualizzare una dashboard personale.
Sotto requisiti	
001	Totale ore di assenza
002	Ore conteggiate nelle 40 ore
003	Numero di ritardi
004	Stato delle richieste (in attesa, approvate, rifiutate)

ID: REQ-04	
Nome	Dashboard docente di classe
Priorità	1
Versione	1.0
Note	Il docente di classe deve visualizzare una dashboard personale.
Sotto requisiti	
001	Totale ore di assenza per allievo
002	Visualizzare ritardi allievi
003	Visualizzare le 40 ore dell'allievo
004	Ricerca e filtri

ID: REQ-05	
Nome	Dashboard capo laboratorio
Priorità	1
Versione	1.0
Note	Il capo laboratorio deve visualizzare una dashboard personale.
Sotto requisiti	
001	Visualizzare le richieste di congedo
002	Gestire le richieste di congedo

ID: REQ-06	
Nome	Segnalazione assenza
Priorità	1
Versione	1.0
Note	L'allievo deve poter segnalare un'assenza.
Sotto requisiti	
001	Inserimento dati
002	Inserimento certificati

ID: REQ-07	
Nome	Gestione firma genitore/tutore
Priorità	1
Versione	1.0
Note	Le assenze degli studenti minorenni devono essere confermate.
Sotto requisiti	
001	Invio email automatico
002	Link con token sicuro
003	Firma grafica

ID: REQ-08	
Nome	Countdown giustificazioni
Priorità	1
Versione	1.0
Note	Il sistema deve gestire le scadenze di giustificazione.
Sotto requisiti	
001	Countdown visibile all'allievo
002	Notifica prima della scadenza

ID: REQ-09	
Nome	Upload certificato medico
Priorità	1
Versione	1.0
Note	L'allievo deve poter caricare certificati medici.
Sotto requisiti	
001	Upload file
002	Associazione all'assenza

ID: REQ-10	
Nome	Validazione assenza docente
Priorità	1
Versione	1.0
Note	Il docente deve poter validare le assenze.
Sotto requisiti	
001	Giustificare l'assenza
002	Inserire osservazioni
003	Approvare anche senza firma tutore (con commento)

ID: REQ-11	
Nome	Assenze arbitrarie
Priorità	1
Versione	1.0
Note	Il sistema deve gestire le assenze non giustificate.
Sotto requisiti	
001	Cambio stato automatico a "Arbitraria"
002	Notifica a allievo e docente

ID: REQ-12	
Nome	Segnalazione ritardo
Priorità	1
Versione	1.0
Note	L'allievo deve poter segnalare un ritardo.
Sotto requisiti	
001	Inserimento durata
002	Rispetto automatico delle regole

ID: REQ-13	
Nome	Gestione congedi
Priorità	1
Versione	1.0
Note	Il sistema deve gestire richieste di congedo.
Sotto requisiti	
001	Creazione richiesta
002	Rifiuto con commento
003	Approvazione

ID: REQ-14	
Nome	Gestione classi
Priorità	1
Versione	1.0
Note	Il sistema deve gestire classi e assegnazioni.
Sotto requisiti	
001	Importazione da file
002	Assegnazione studenti
003	Assegnazione docente di classe

ID: REQ-15	
Nome	Creazione automatica utenti
Priorità	1
Versione	1.0
Note	Durante l'import il sistema deve creare utenti.
Sotto requisiti	
001	Creazione account studenti
002	Assegnazione classe
003	Generazione credenziali iniziali

ID: REQ-16	
Nome	Storico operazioni
Priorità	2
Versione	1.0
Note	Il sistema deve mantenere uno storico completo.
Sotto requisiti	
001	Tracciamento approvazioni
002	Tracciamento eccezioni
003	Tracciamento commenti e decisioni

ID: REQ-17	
Nome	Dashboard amministrativa
Priorità	1
Versione	1.0
Note	L'amministratore deve disporre di una dashboard di gestione globale del sistema.
Sotto requisiti	
001	Visualizzazione riepilogo generale:
002	Visualizzazione stato del sistema (visualizzazione errori successi)
003	Gestione utenti

ID: REQ-18	
Nome	Gestione notifiche email
Priorità	1
Versione	1.0
Note	Il sistema deve notificare automaticamente gli utenti sugli eventi rilevanti.
Sotto requisiti	
001	Email segnalazione
002	Possibilità di reinviare manualmente una notifica

ID: REQ-19	
Nome	Invio report tutore
Priorità	2
Versione	1.0
Note	Il sistema deve inviare i report ogni mese al tutore dell’allievo e approvarlo
Sotto requisiti	
001	Email con il report
002	Possibilità di firmare

ID: REQ-20	
Nome	Impostazioni regole
Priorità	2
Versione	1.0
Note	Il sistema deve avere la possibilità di settare le regole come le 40 ore, o le regole per i ritardi.
Sotto requisiti	
001	Impostazioni per le regole
002	Gestione regole custom

ID: REQ-21	
Nome	Gestione documenti
Priorità	2
Versione	1.0
Note	Il sistema deve gestire i documenti caricati.
Sotto requisiti	
001	Visualizzazione certificati caricati
002	Download documenti

ID: REQ-22	
Nome	Esportazione dati
Priorità	2
Versione	1.0
Note	Il sistema deve permettere l'esportazione dei dati.
Sotto requisiti	
001	Esportazione elenco assenze
002	Esportazione ritardi
003	Esportazione report per classe

ID: REQ-23	
Nome	Gestione calendario scolastico e vacanze
Priorità	2
Versione	1.0
Note	Il sistema deve permettere la gestione del calendario scolastico e delle date di vacanza utilizzate nelle regole applicative.
Sotto requisiti	
001	Importazione calendario da file PDF
002	Estrazione automatica delle date di vacanza
003	Salvataggio delle date nel sistema
004	Modifica manuale delle date importate
005	Eliminazione delle date non corrette

ID: REQ-24	
Nome	Notifiche interne applicative
Priorità	2
Versione	1.0
Note	Il sistema deve mostrare notifiche interne agli utenti per segnalare eventi rilevanti direttamente nell'applicazione.
Sotto requisiti	
001	Visualizzazione notifiche non lette
002	Conteggio notifiche da leggere
003	Collegamento rapido alla schermata collegata all'evento
004	Aggiornamento dello stato letto/non letto
005	Notifiche dedicate ai ruoli coinvolti

ID: REQ-24	
Nome	Archiviazione documentale per studente
Priorità	2
Versione	1.0
Note	Il sistema deve archiviare i documenti degli studenti in modo strutturato e recuperabile.
Sotto requisiti	
001	Salvataggio documenti per categoria
002	Conteggio notifiche da leggere
003	Collegamento rapido alla schermata collegata all'evento
004	Aggiornamento dello stato letto/non letto
005	Notifiche dedicate ai ruoli coinvolti

2.2.1 Spiegazione elementi tabella dei requisiti:

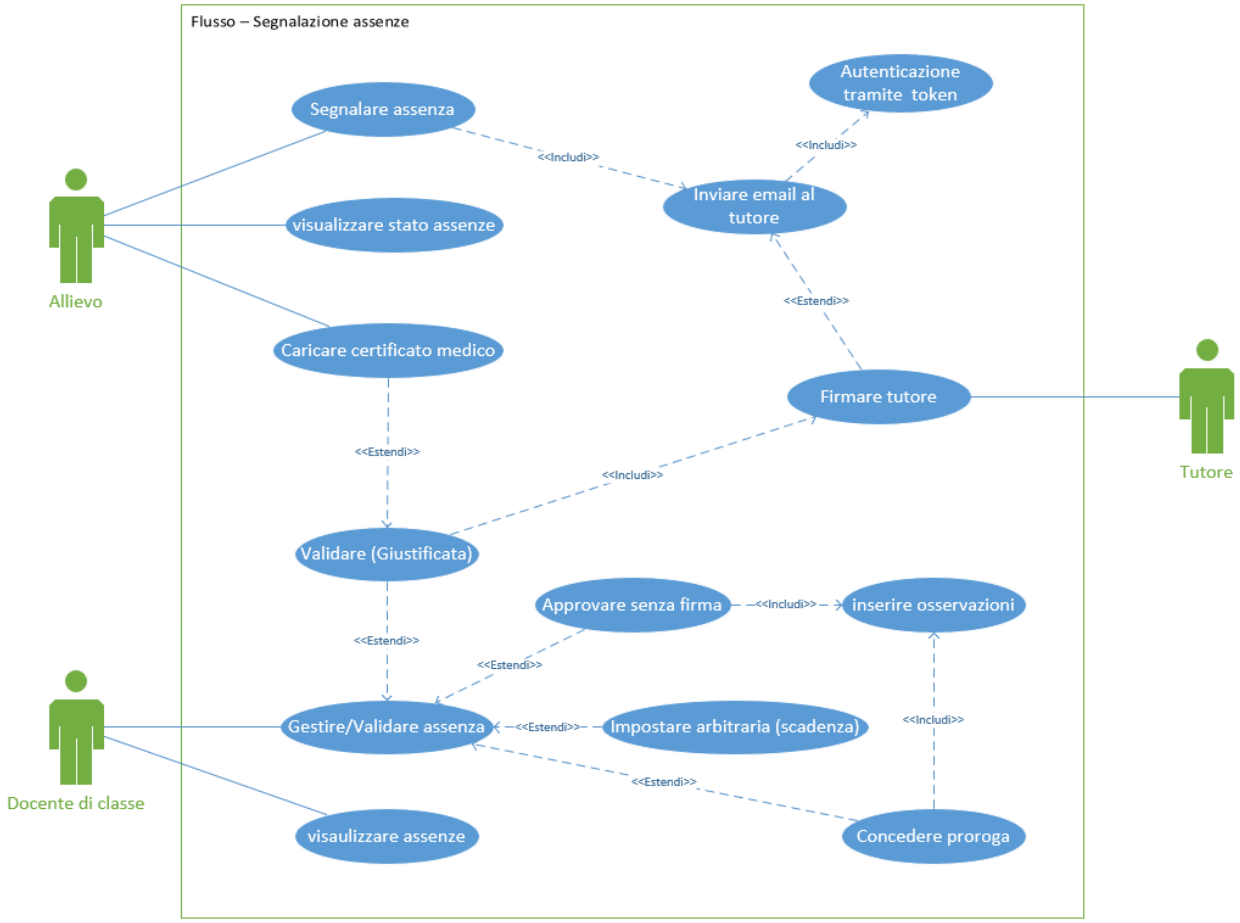
ID: identificativo univoco del requisito
Nome: breve descrizione del requisito
Priorità: indica l'importanza di un requisito nell'insieme del progetto, definita assieme al committente. Ad esempio, poter disporre di report con colonne di colori diversi ha priorità minore rispetto al fatto di avere un database con gli elementi al suo interno. Solitamente si definiscono al massimo di 2-3 livelli di priorità.
Versione: indica la versione del requisito. Ogni modifica del requisito avrà una versione aggiornata.
Sulla documentazione apparirà solamente l'ultima versione, mentre le vecchie dovranno essere inserite nei diari.
Note: eventuali osservazioni importanti o riferimenti ad altri requisiti.
Sotto requisiti: elementi che compongono il requisito.

2.3 Use case

2.3.1 Use case richieste di congedo

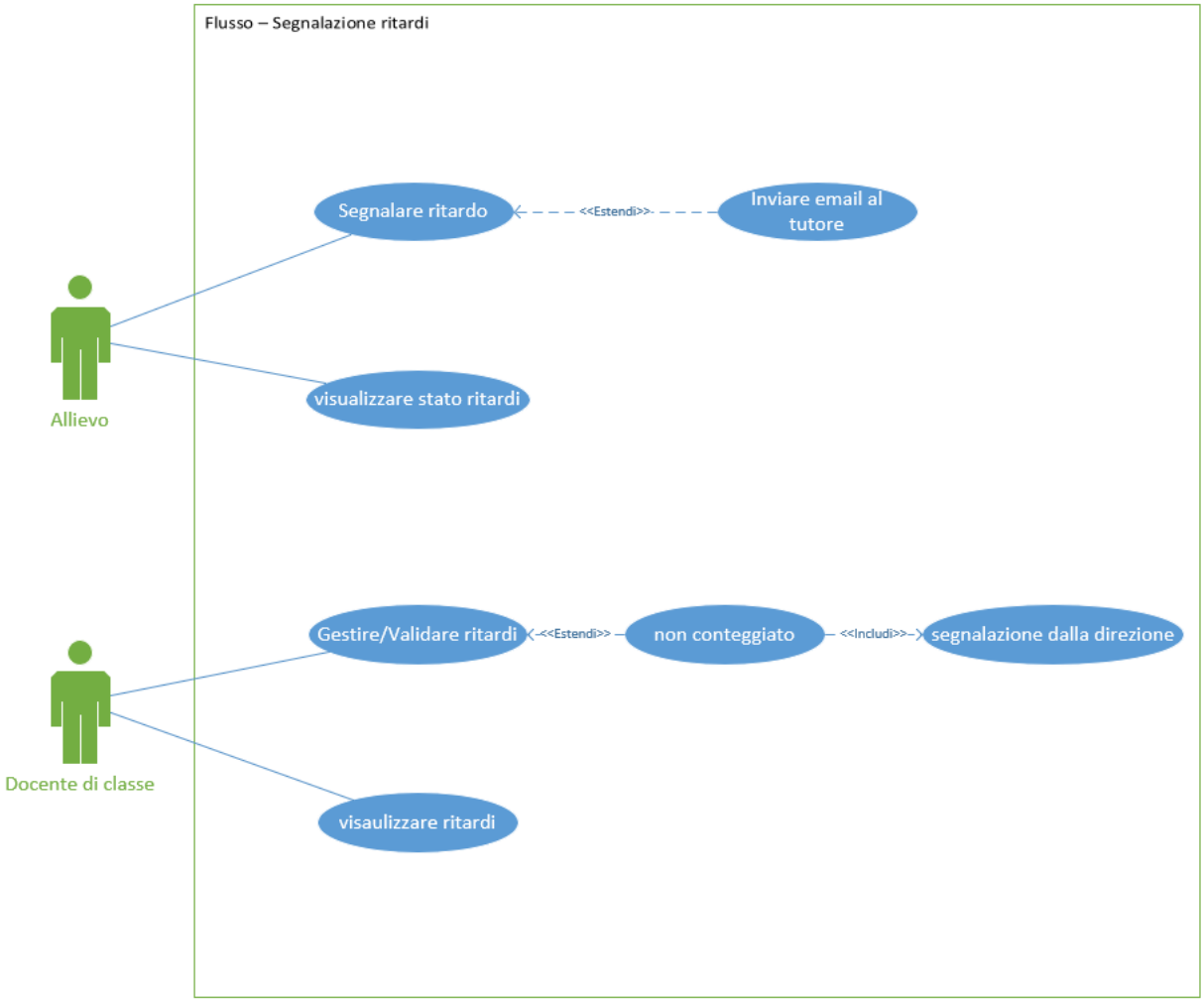
Questo Use Case descrive come funziona la richiesta di congedo all'interno dell'applicazione e come i diversi utenti interagiscono tra loro.
L'allievo può inserire una richiesta di congedo compilando i dati necessari e controllarne lo stato in qualsiasi momento. Dopo l'invio, il sistema manda una email al genitore o tutore, che può confermare la richiesta tramite un link sicuro.
Il docente di classe e il capo laboratorio possono visualizzare e gestire le richieste, decidendo se approvarle, rifiutarle o chiedere documentazione aggiuntiva. In situazioni particolari, possono anche approvare la richiesta senza la firma del tutore, inserendo un commento.

2.3.2 Use case segnalazione assenze



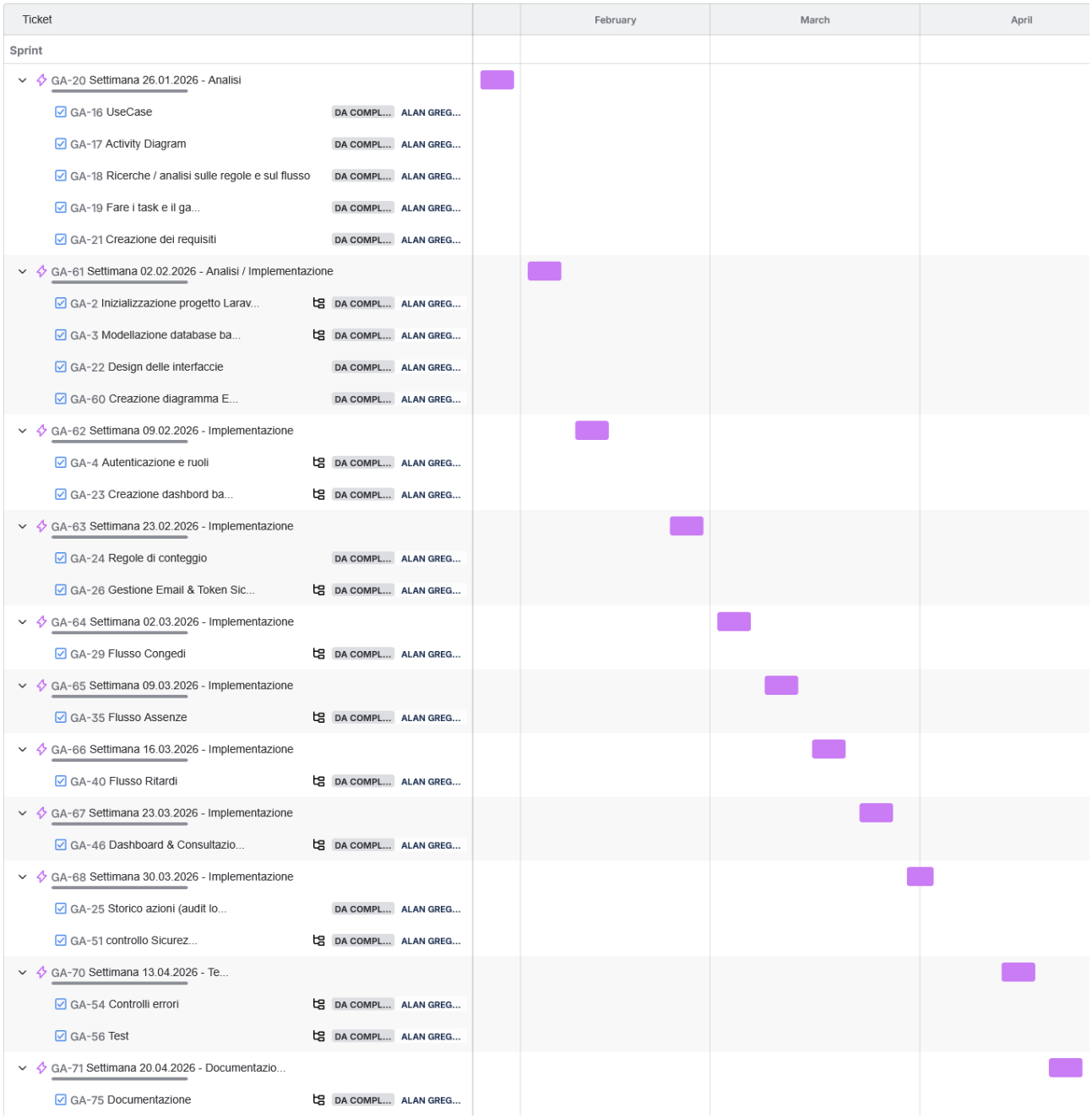
Questo Use Case descrive il processo di segnalazione e gestione delle assenze degli allievi.
L'allievo può segnalare un'assenza, visualizzarne lo stato e, se necessario, caricare un certificato medico.
Dopo la segnalazione, il sistema invia automaticamente una email al genitore o tutore, che può confermare l'assenza tramite un link.
Il docente di classe può visualizzare e gestire le assenze, decidendo se giustificarle, concedere una proroga o approvarle anche senza la firma del tutore, inserendo un'osservazione. Se i termini non vengono rispettati, l'assenza può diventare arbitraria.

2.3.3 Use case segnalazione ritardo



Questo Use Case descrive il processo di segnalazione e gestione dei ritardi.
L'allievo può segnalare un ritardo inserendo i dati richiesti e visualizzare lo stato dei propri ritardi. In seguito alla segnalazione, il sistema può inviare una email al genitore o tutore nei casi previsti.
Il docente di classe può visualizzare i ritardi segnalati e gestirli. Nei casi di ritardo globale comunicato dalla direzione, il ritardo viene contrassegnato come non conteggiato.
Il sistema registra tutte le operazioni effettuate, garantendo una gestione chiara e tracciabile dei ritardi.

2.4 Pianificazione



Il diagramma di Gantt mostra la pianificazione temporale del progetto, suddivisa in fasi settimanali.

La prima fase è dedicata all'analisi, con la definizione dei requisiti, delle use case e dei diagrammi. Segue la fase di implementazione, nella quale vengono sviluppate le principali funzionalità del sistema, come l'autenticazione, la gestione delle assenze, dei ritardi e dei congedi.

Il progetto si conclude con le attività di test, controllo della sicurezza e documentazione finale, garantendo uno sviluppo ordinato e progressivo.

2.4.1 Sprint 1

Lo sprint 1 ha una durata di circa 2 settimane (4 febbraio – 27 febbraio) di dal 13 al 22 è vacanza.
In questa fase vengono sviluppate le funzionalità fondamentali per l'avvio dell'applicativo.

Le attività principali sono:

- Autenticazione e gestione ruoli, con sistema di login, controllo accessi e distinzione tra allievo, docente di classe, capo laboratorio e amministratore.
- Creazione dashboard base, con struttura iniziale delle interfacce e visualizzazione dei primi dati.
- Implementazione del flusso assenze, comprensivo di segnalazione da parte dell'allievo e gestione dello stato.
- Implementazione delle regole di conteggio, incluse le basi per la gestione dinamica delle 40 ore annuali.

Al termine dello sprint 1 il sistema dispone quindi di autenticazione funzionante, struttura delle dashboard e primo flusso completo operativo (assenze).

2.4.2 Sprint 2

Lo sprint 2 copre il periodo 2 marzo – 20 marzo.
In questa fase vengono sviluppati i moduli mancanti legati ai flussi principali e alla comunicazione tramite email.

Le attività previste sono:

- Flusso congedi, con creazione richiesta, gestione stati, approvazioni e rifiuti.
- Gestione email e token sicuri, per conferma tutore tramite link temporaneo e tracciamento delle notifiche.
- Flusso ritardi, con registrazione durata, gestione soglie e stati (giustificato, arbitrario, globale).

Al termine dello sprint 2 tutti i flussi principali dell'applicativo (assenze, congedi e ritardi) risultano implementati.

2.4.3 Sprint 3

Lo sprint 3 si svolge dal 23 marzo al 3 aprile.
Questa fase è dedicata al consolidamento del sistema, al controllo e alla tracciabilità delle operazioni.

Le attività principali sono:

- Dashboard e consultazione avanzata, con miglioramento filtri, visualizzazione dati per classe e consultazione completa dello storico.
- Controlli di sicurezza, inclusa verifica permessi, protezione rotte e validazioni lato server.
- Storiche azioni (audit log), con implementazione completa della tracciabilità tramite LOG_OPERAZIONI.

Al termine dello sprint 3 l'applicativo risulta completo dal punto di vista funzionale, sicuro e dotato di uno storico verificabile di tutte le operazioni rilevanti.

2.5 Analisi dei mezzi

Per la realizzazione dell'applicativo web di gestione di assenze, ritardi e congedi, sono stati utilizzati i mezzi hardware e software messi a disposizione dalla Scuola Arti e Mestieri di Trevano.
Tutti gli strumenti impiegati sono compatibili con lo sviluppo di applicazioni web moderne e consentono una gestione completa del progetto.

2.5.1 Software

Durante lo sviluppo del progetto sono stati utilizzati i seguenti strumenti software e librerie:

- **PHP 8.2.12**
  Linguaggio server-side utilizzato per implementare la logica applicativa.
  Nel progetto gestisce le regole di business (assenze, ritardi, congedi, scadenze), la validazione dei dati inseriti dagli utenti, l'elaborazione dei certificati medici e l'interazione con il database.

- **Laravel 12.49.0**
  Framework PHP utilizzato come base del backend dell'applicazione.
  Si occupa della gestione degli utenti e dei ruoli (studente, docente, amministratore, tutore), dell'autenticazione, delle autorizzazioni, delle notifiche email e del flusso delle richieste.
  Permette inoltre di strutturare il codice in modo ordinato.

- **Inertia.js 2.0.19**
  Libreria utilizzata per collegare il backend Laravel con il frontend React.
  Consente di sviluppare un'applicazione a pagina singola senza la necessità di creare un'API REST separata, permettendo lo scambio diretto dei dati tra server e interfaccia utente.

- **React 18.3.1**
  Libreria JavaScript utilizzata per lo sviluppo dell'interfaccia utente.
  Viene impiegata per la realizzazione delle dashboard e dei moduli interattivi, permettendo un aggiornamento dinamico dei contenuti senza ricaricare la pagina.

- **Tailwind CSS 3.4.19**
  Framework CSS utilizzato per la realizzazione dell'interfaccia grafica.
  Permette di creare layout semplici, coerenti e responsive, mantenendo uno stile uniforme in tutte le sezioni dell'applicazione.

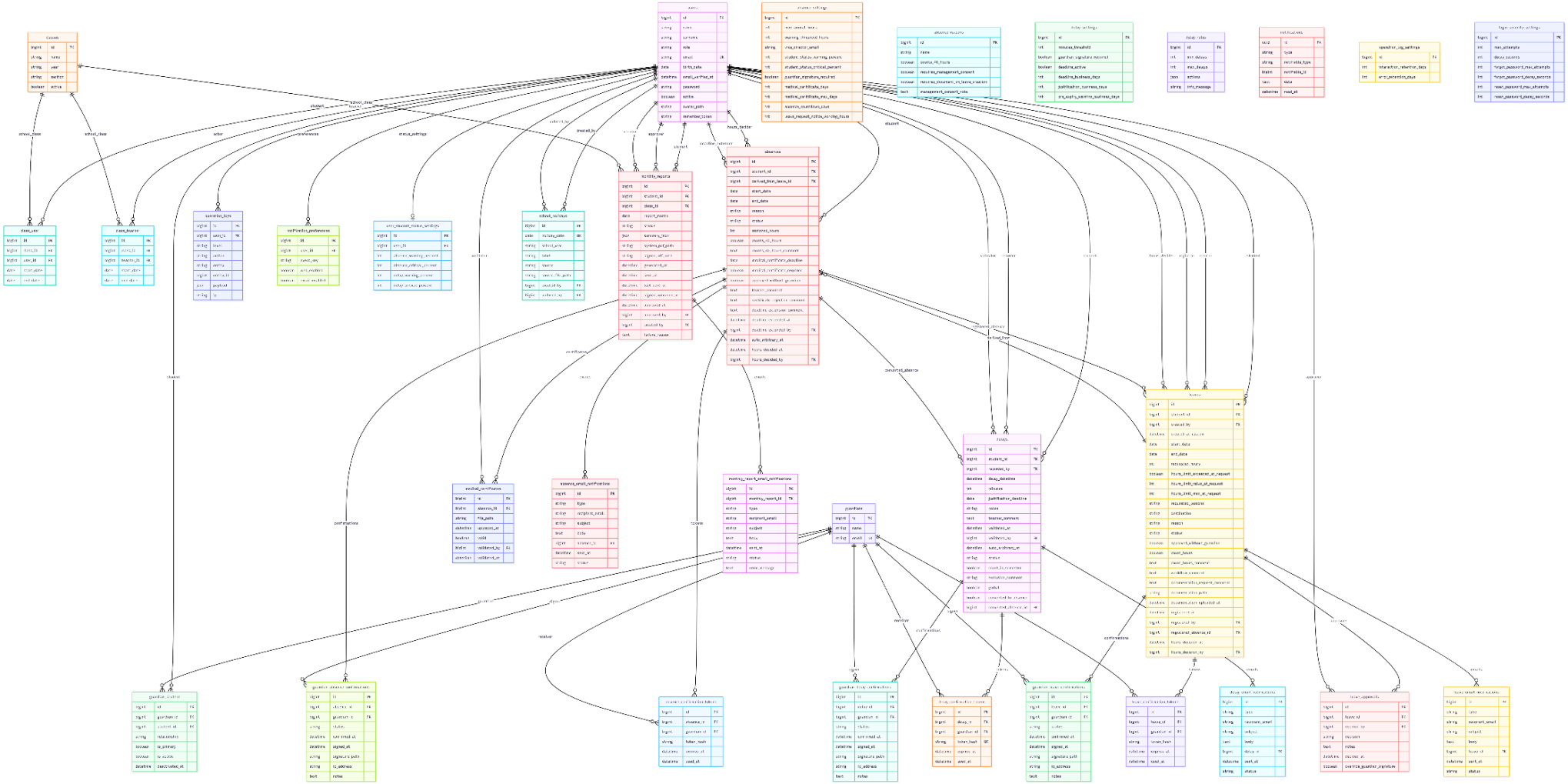
- **MySQL 8.0**
  Sistema di gestione di database relazionali utilizzato per la memorizzazione e gestione dei dati persistenti (utenti, assenze, ritardi e congedi).

2.5.2 Hardware

Per lo sviluppo del progetto è stato utilizzato un PC fornito dalla Scuola Arti e Mestieri di Trevano, con sistema operativo e software di base già installati, utilizzato per lo sviluppo, l'esecuzione e il test dell'applicazione.

3 Progettazione

3.1 Design dei dati e database



Il database del progetto è stato progettato per supportare un gestionale scolastico dedicato alla gestione di assenze, ritardi, richieste di congedo, firme dei tutori, approvazioni, notifiche e report mensili.
L'elemento centrale del sistema è la tabella **users**, che rappresenta tutti gli utenti dell'applicazione: studenti, docenti, capi laboratorio e amministratori.

Il ruolo viene distinto tramite il campo **role**, permettendo di usare un'unica tabella per autenticazione, permessi e collegamenti con le altre entità del sistema.
La struttura scolastica è rappresentata dalla tabella **classes**.
Il collegamento tra studenti e classi viene gestito tramite la tabella ponte **class_user**, che permette di mantenere anche lo storico delle iscrizioni grazie ai campi **start_date** ed **end_date**.
L'associazione tra docenti e classi viene invece gestita tramite **class_teacher**, che collega un docente a una classe e permette di tracciare nel tempo le assegnazioni dei docenti.

I tutori sono salvati nella tabella **guardians** e collegati agli studenti tramite la tabella **guardian_student**.
Questa relazione permette di gestire più tutori per uno stesso studente, indicando il grado di parentela, il tutore principale, lo stato attivo del collegamento e l'eventuale data di disattivazione.

Le assenze sono memorizzate nella tabella **absences**, collegata allo studente tramite **student_id**.
Ogni assenza contiene le date di inizio e fine, il motivo, lo stato, le ore assegnate, l'eventuale conteggio nelle 40 ore annuali, i commenti del docente e le informazioni legate a certificati medici o proroghe.
La firma del tutore per le assenze viene gestita tramite due tabelle dedicate:

- absence_confirmation_tokens, che salva i token sicuri con scadenza;
- guardian_absence_confirmations, che registra la conferma, la firma, la data e il tutore che ha firmato.

I certificati medici sono gestiti dalla tabella **medical_certificates**, collegata ad **absences**.
Ogni certificato contiene il percorso del file, la data di caricamento, lo stato di validità e l'utente che lo ha eventualmente validato.

I ritardi sono salvati nella tabella **delays**.
Ogni ritardo è collegato allo studente e al docente che lo ha registrato. La tabella contiene la data e ora del ritardo, i minuti, lo stato, la scadenza per la giustificazione, l'eventuale conteggio nel semestre, i commenti del docente e l'eventuale collegamento a un'assenza generata.
Anche i ritardi supportano la firma del tutore tramite:

- delay_confirmation_tokens, per la gestione dei token di firma;
- guardian_delay_confirmations, per registrare la firma effettiva;
- delay_email_notifications, per tracciare le email inviate.

Le richieste di congedo sono memorizzate nella tabella **leaves**.
Questa tabella contiene lo studente, l'utente che ha creato la richiesta, il periodo richiesto, le ore, la destinazione, il motivo, lo stato della pratica, la documentazione allegata e le decisioni relative al conteggio nelle 40 ore.
Il workflow dei congedi viene supportato da più tabelle:

- guardian_leave_confirmations, per la firma del tutore;
- leave_confirmation_tokens, per i token di conferma;
- leave_approvals, per registrare approvazioni, rifiuti e override;
- leave_email_notifications, per tracciare le email inviate;
- registered_absence_id, che collega un congedo approvato all'eventuale assenza generata.

Il conteggio delle 40 ore annuali non viene salvato come semplice contatore fisso, ma viene calcolato in base ai dati registrati: assenze, congedi, certificati validati, esclusioni manuali e decisioni dei docenti.
Le regole relative ai motivi di assenza sono configurate nella tabella **absence_reasons**, dove viene indicato se un motivo conta nelle 40 ore, se richiede consenso della direzione o se richiede documentazione già in fase di richiesta congedo.

Immagine ben visibile caricata in 10 Allegati.

Le impostazioni generali del sistema sono divise in tabelle dedicate:

- absence_settings, per soglie annuali, certificati medici, firme dei tutori e preavviso minimo per i congedi;
- delay_settings, per soglie e scadenze legate ai ritardi;
- delay_rules, per configurare le regole applicative sui ritardi;
- login_security_settings, per i limiti di sicurezza su login e recupero password;
- operation_log_settings, per la conservazione dei log;
- user_student_status_settings, per personalizzare le soglie di stato degli studenti.

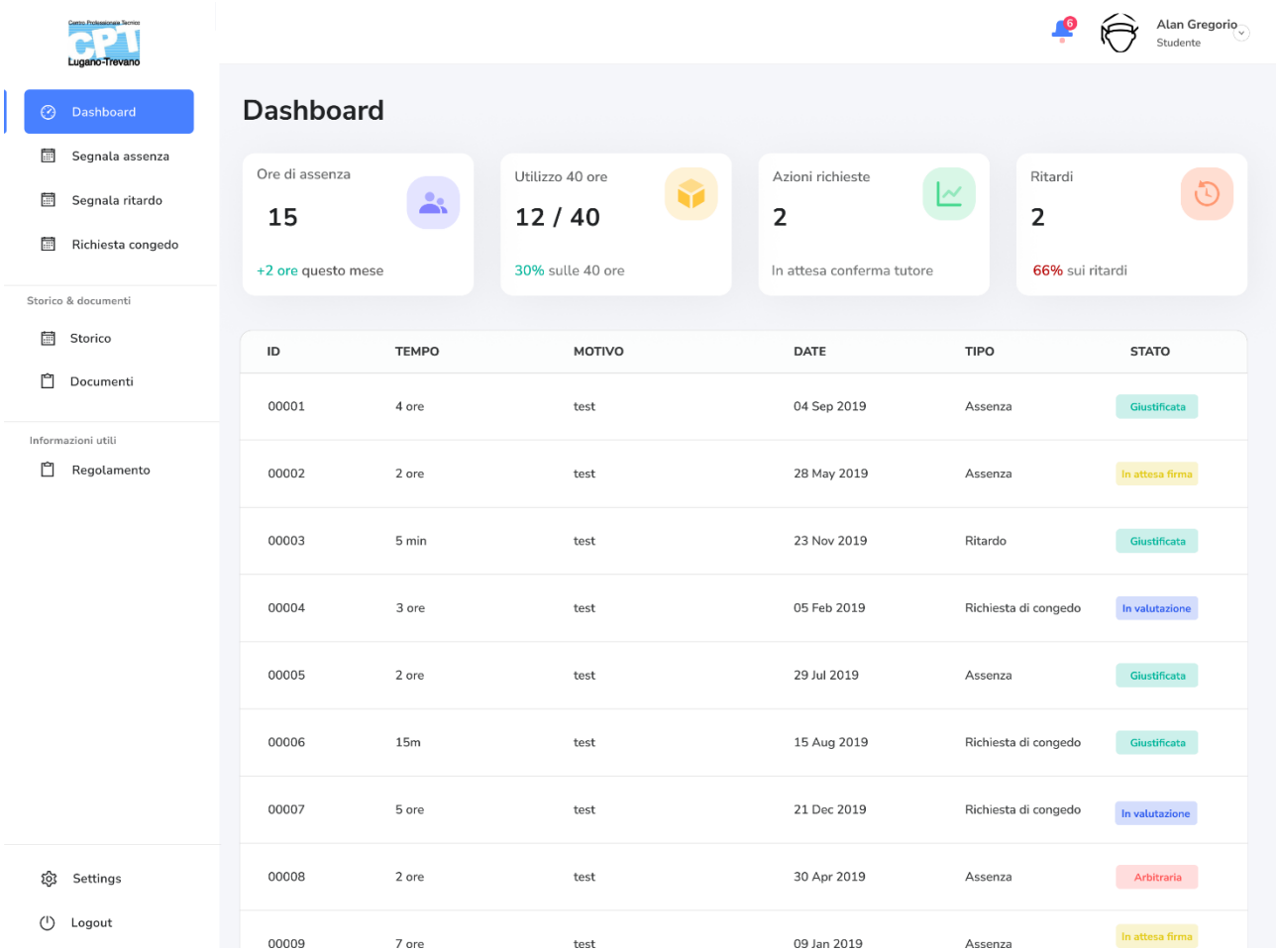
Il sistema include anche la gestione dei report mensili tramite **monthly_reports**, collegata a studenti e classi.
Le email relative ai report sono tracciate in **monthly_report_email_notifications**.
Le notifiche interne dell'applicazione sono salvate nella tabella **notifications**, mentre le preferenze di notifica degli utenti sono gestite tramite **notification_preferences**, dove è possibile abilitare o disabilitare notifiche email e notifiche web per specifici eventi.
Le operazioni rilevanti vengono registrate nella tabella **operation_logs**.
Questa tabella permette di mantenere uno storico delle azioni eseguite, come approvazioni, modifiche, errori, override, invii email e attività amministrative.

Infine, la tabella **school_holidays** permette di registrare i giorni festivi scolastici. Questi dati vengono utilizzati nei calcoli delle scadenze lavorative, ad esempio per certificati medici, ritardi e richieste di congedo.
Nel complesso, il database è strutturato per separare chiaramente utenti, classi, tutori, pratiche scolastiche, configurazioni, notifiche e log, mantenendo uno schema ordinato e coerente con il funzionamento reale dell'applicazione.
.

3.2 Design delle interfacce

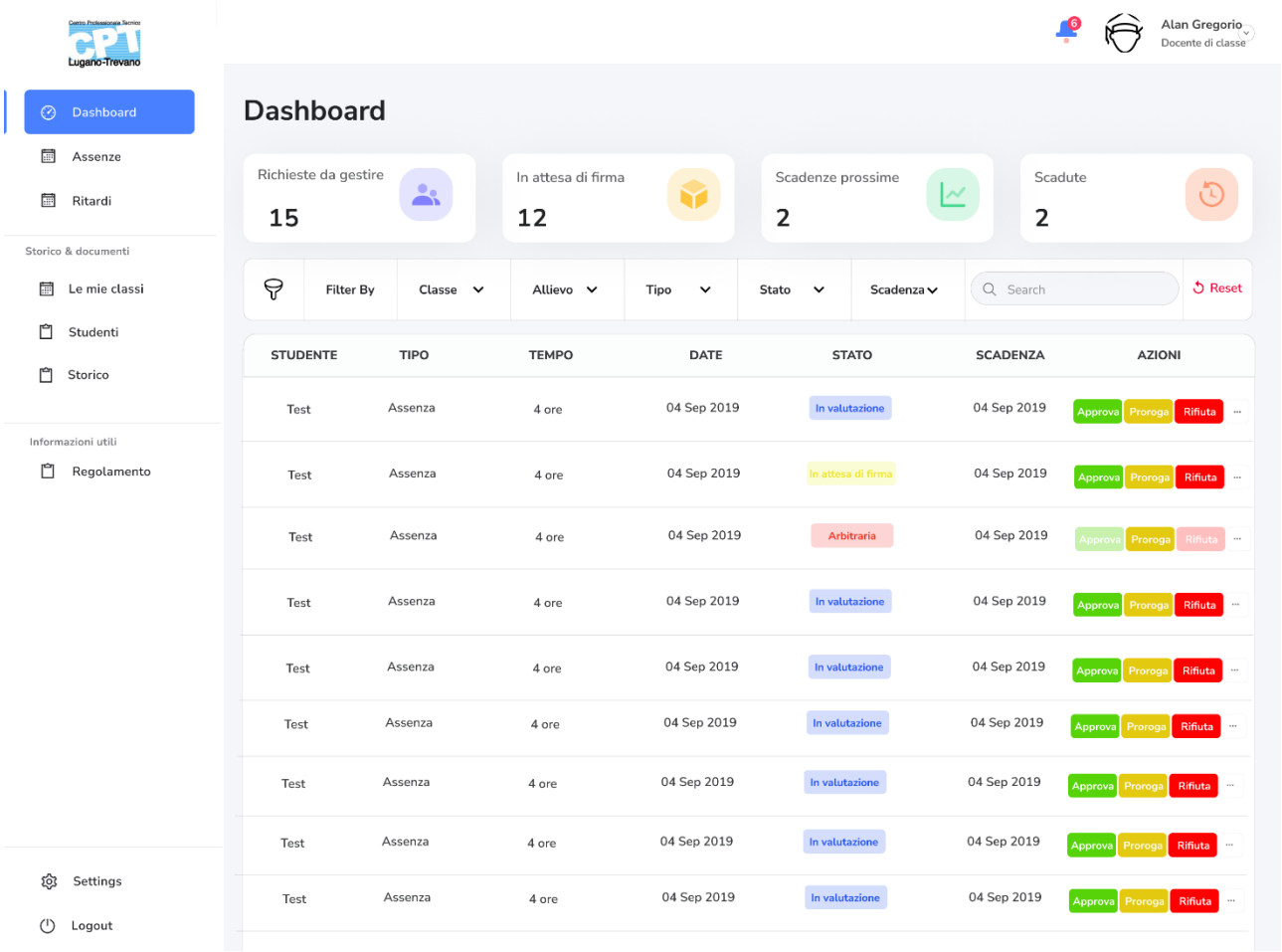
La progettazione delle interfacce è stata realizzata basandosi sulle informazioni ricavate durante la fase di analisi e le abbiamo progettate tramite il sito figma.
Tutte le interfacce sono state pensate per essere utilizzate su Browser Web.

3.2.1 Interfaccia dashboard allievo



Ho progettato la dashboard dell'allievo in modo che sia semplice e intuitivo, per permettere subito la visione di tutte le informazioni imporanti.
Le card iniziali mostrano le informazioni più importanti, come ore di assenza, utilizzo delle 40 ore e richieste in sospeso.
La tabella sottostante permette di consultare lo storico di assenze, ritardi e congedi con il relativo stato.

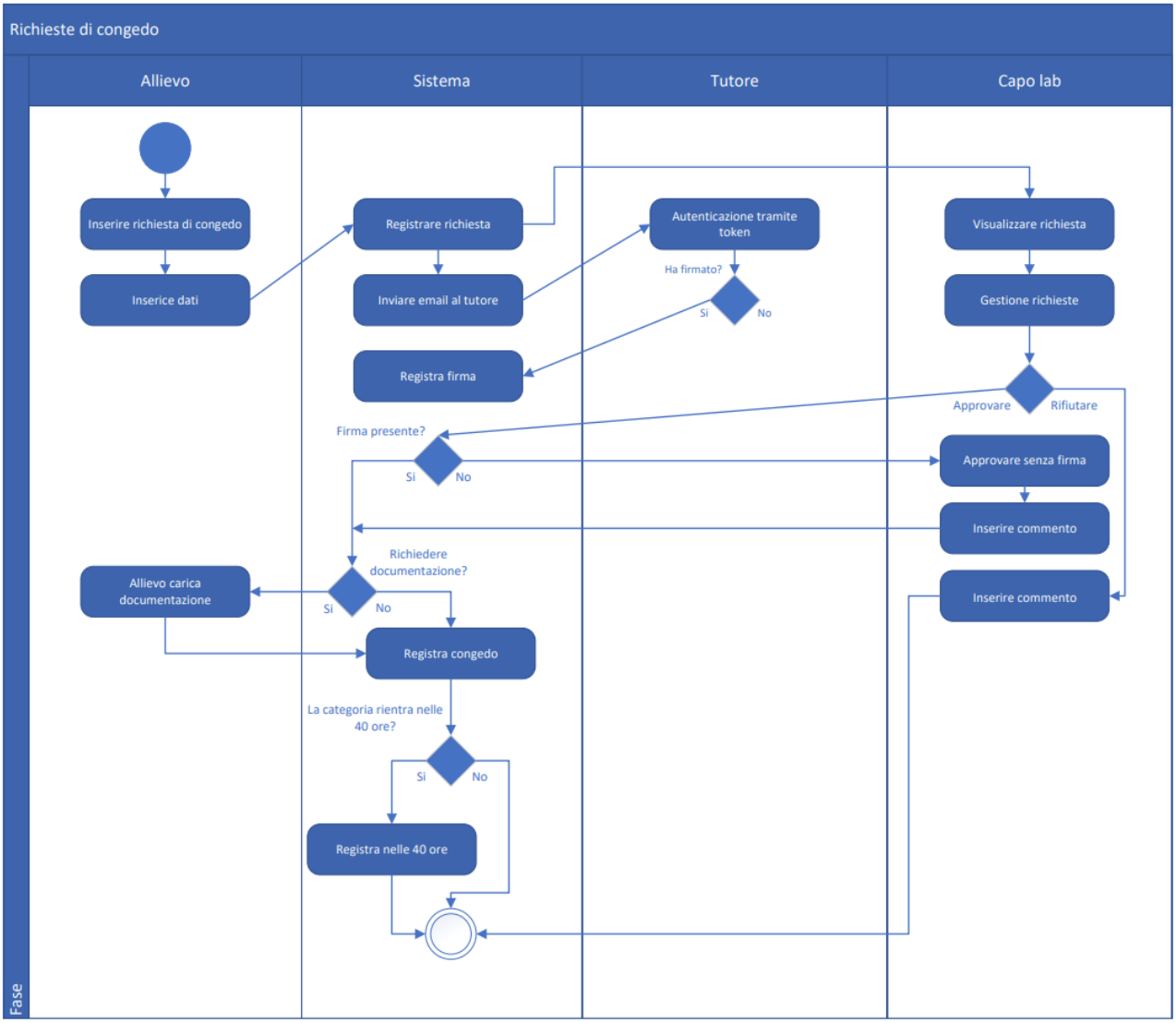
3.2.2 Interfaccia dashboard docente di classe



La dashboard del docente di classe è stata progettata per facilitare la gestione quotidiana delle assenze e ritardi degli allievi.
Le card iniziali permettono di individuare subito le richieste da gestire, quelle in attesa di firma e le scadenze imminenti.
La tabella centrale consente di filtrare e visualizzare rapidamente le segnalazioni per classe, allievo, tipo e stato.
I pulsanti di azione permettono di approvare, prorogare o rifiutare una richiesta in modo rapido.

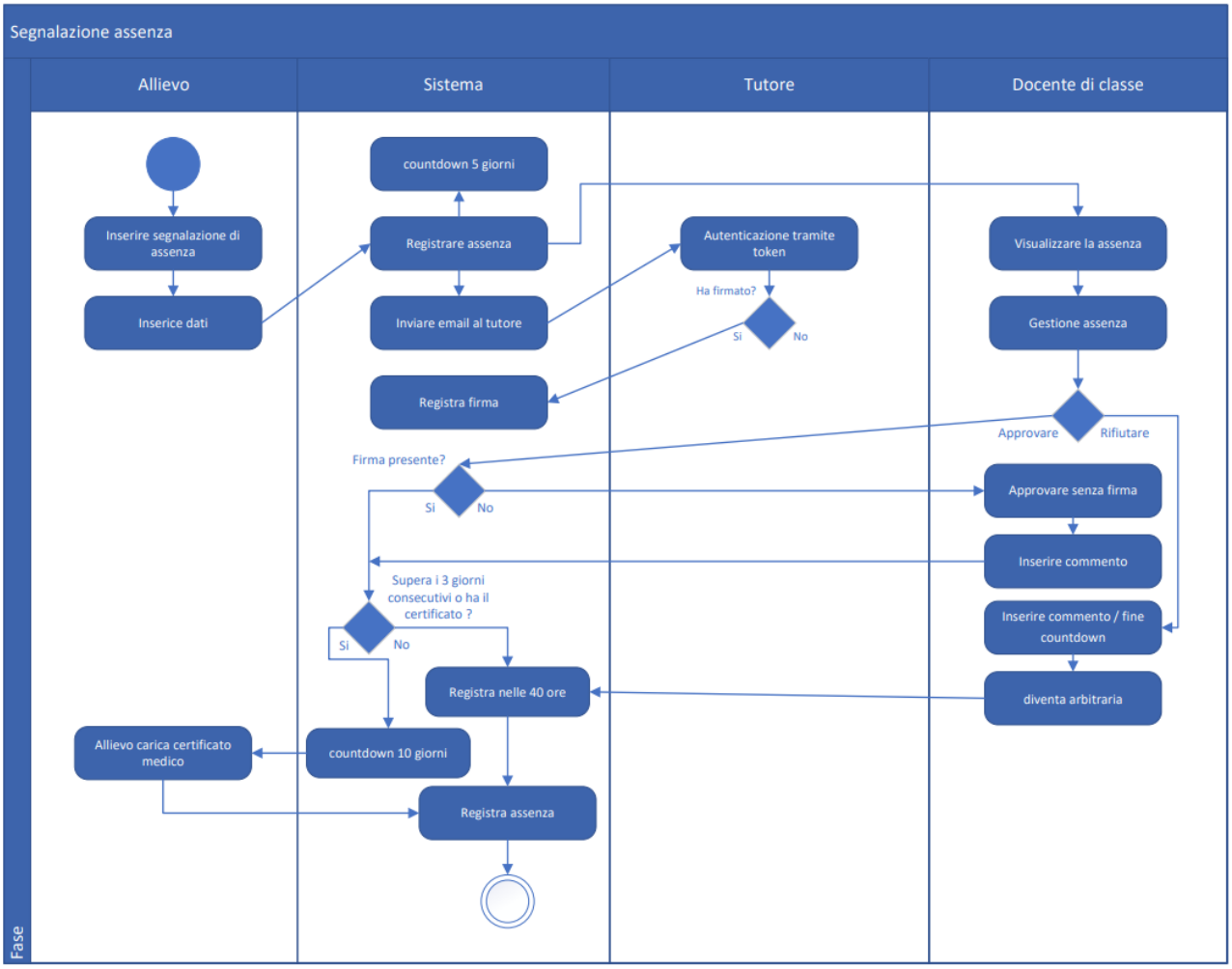
3.3 Design procedurale

3.3.1 Activity diagram richieste di congedo



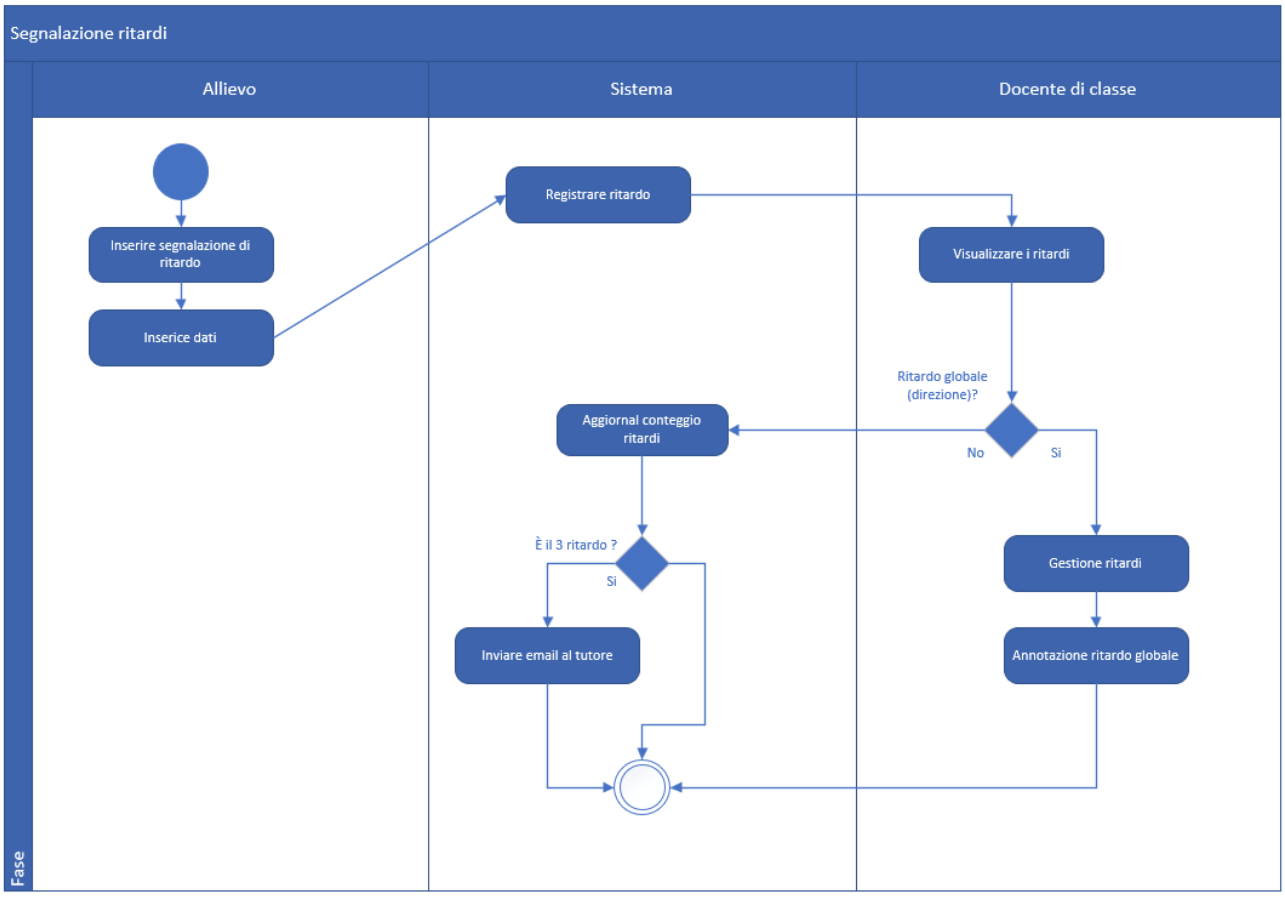
Questo activity diagram rappresenta il flusso operativo della richiesta di congedo, mostrando le attività svolte dai diversi attori e dal sistema.
Il processo inizia quando l'allievo inserisce una richiesta di congedo, compilando i dati necessari. Il sistema registra la richiesta e invia automaticamente una email al genitore o tutore per la conferma.
Il tutore può accedere tramite token e decidere se firmare o meno la richiesta. La firma, se presente, viene registrata dal sistema, ma l'assenza della firma non blocca il processo.
Il capo laboratorio può in qualsiasi momento visualizzare e gestire la richiesta, scegliendo se approvarla, rifiutarla o approvarla senza firma del tutore, inserendo un commento obbligatorio. Se necessario, può anche richiedere documentazione aggiuntiva, che l'allievo può caricare successivamente.
Prima della chiusura del processo, il sistema verifica se il congedo rientra nel conteggio delle 40 ore. In base a questa decisione, il congedo viene registrato correttamente e il flusso si conclude.

3.3.2 Activity diagram segnalazione assenza



Questo activity diagram rappresenta il flusso operativo della segnalazione e gestione delle assenze, mostrando le attività svolte dall'allievo, dal sistema, dal tutore e dal docente di classe.
Il processo inizia quando l'allievo inserisce una segnalazione di assenza, compilando i dati richiesti. Il sistema registra l'assenza, avvia un countdown di 5 giorni e invia automaticamente una email al genitore o tutore per la conferma.
Il tutore può autenticarsi tramite token e decidere se firmare o meno l'assenza. La firma, se presente, viene registrata dal sistema, ma l'assenza rimane comunque visibile e gestibile dal docente di classe anche in assenza di firma.
Il sistema verifica se l'assenza supera i 3 giorni consecutivi o se è richiesto un certificato medico. In tal caso, l'allievo può caricare il certificato, che viene associato all'assenza. In assenza di certificato valido, le ore vengono conteggiate nelle 40 ore.
Il docente di classe può visualizzare e gestire l'assenza, scegliendo se approvarla, rifiutarla o approvarla senza la firma del tutore, inserendo un commento. Può inoltre concedere una proroga. Se il termine scade senza le conferme necessarie, l'assenza diventa arbitraria.

3.3.3 Activity diagram segnalazione ritardi



Questo activity diagram descrive il flusso di segnalazione e gestione dei ritardi.
Il processo ha inizio quando l'allievo inserisce una segnalazione di ritardo compilando i dati richiesti. Il sistema registra il ritardo e aggiorna il conteggio complessivo.
Successivamente il docente di classe può visualizzare i ritardi segnalati e verificare se il ritardo rientra in un caso di ritardo globale comunicato dalla direzione. In tal caso il ritardo viene annotato come ritardo globale e non influisce sul conteggio.
Se il ritardo non è globale, il sistema verifica se si tratta del terzo ritardo: in questo caso viene inviata automaticamente una email al tutore. Il processo si conclude con la registrazione definitiva del ritardo.

4 Implementazione

4.1 Struttura generale dell'applicazione

L'applicazione è stata sviluppata come sistema web basato su Laravel per la parte backend e React con Inertia per la parte frontend. Laravel gestisce le rotte, i controller, l'autenticazione, l'accesso al database e la logica principale dell'applicazione. React viene invece utilizzato per realizzare le schermate visualizzate dagli utenti.

La struttura del progetto è stata organizzata separando le diverse responsabilità. I controller ricevono le richieste provenienti dalle rotte, i model rappresentano le entità salvate nel database, le request gestiscono la validazione dei dati inseriti dagli utenti e i service contengono le parti di logica più complesse. Le pagine dell'interfaccia si trovano invece nella cartella dedicata al frontend.

4.1.1 Cartelle principali del progetto

Cartella	Scopo
routes/web.php	Definizione delle rotte principali dell'applicazione
app/Http/Controllers	Gestione delle richieste e collegamento tra frontend e logica
app/Http/Requests	Validazione dei dati inseriti nei form
app/Models	Rappresentazione delle tabelle e dei dati principali
app/Services	Logica applicativa più complessa
app/Support	Classi di supporto usate in più parti del sistema
app/Console/Commands	Comandi automatici eseguiti dal sistema
resources/js/Pages	Pagine React mostrate agli utenti
resources/js/Layouts	Layout comuni dell'interfaccia
scripts	Script esterni di supporto, come l'analisi dei PDF

4.1.2 Inizializzazione del frontend con Inertia

```
createInertiaApp({
    title: (title) => `${title} - ${appName}`,
    resolve: (name) =>
        resolvePageComponent(
            `./Pages/${name}.jsx`,
            import.meta.glob('./Pages/**/*.jsx'),
        ),
    setup({ el, App, props }) {
        const root = createRoot(el);

        root.render(<App {...props} />);
    },
    progress: {
        color: '#4B5563',
    },
});
```

Questo codice inizializza Inertia e permette a Laravel di caricare le pagine React presenti nella cartella **resources/js/Pages**. In questo modo il backend può restituire direttamente la schermata corretta senza dover creare un frontend completamente separato tramite API. Questa struttura è stata scelta perché consente di mantenere insieme backend e frontend, rendendo più semplice lo sviluppo del progetto.

4.2 Gestione delle rotte

4.2.1 Organizzazione delle rotte

Le rotte principali dell'applicazione sono definite nel file routes/web.php. Ogni rotta collega un URL a un metodo specifico di un controller. In questo modo le richieste degli utenti vengono indirizzate alla parte corretta dell'applicazione.

La maggior parte delle rotte è protetta tramite autenticazione, quindi può essere utilizzata solo da utenti che hanno effettuato il login. Alcune rotte, come quelle per la firma del tutore, sono invece accessibili senza login ma richiedono un token sicuro.

```
Route::redirect('/', '/login');

Route::get('/dashboard', [DashboardController::class, 'index'])-
middleware(['auth', 'verified'])->name('dashboard');
      Route::get('/regole', [RulesController::class, 'index'])
          ->name('rules.index');
      Route::get('/regole/pdf', [RulesController::class, 'downloadPdf'])
          ->name('rules.download');

      Route::middleware('auth')->group(function () {
});
```

4.2.2 Rotte dell'allievo

Le rotte dell'allievo permettono di creare assenze, ritardi, congedi e consultare storico, documenti e report mensili. Queste rotte sono disponibili solo dopo il login.

```
Route::get('/studente/assenze/nuova', [AbsenceController::class, 'create'])
    ->name('student.absences.create');
Route::post('/studente/assenze', [AbsenceController::class,'NewRequestAbsence'])
    ->name('student.absences.store');

Route::get('/studente/congedi/nuovo', [StudentLeaveController::class, 'create'])
    ->name('student.leaves.create');
Route::post('/studente/congedi', [StudentLeaveController::class, 'store'])
    ->name('student.leaves.store');

Route::get('/studente/ritardi/nuovo', [StudentDelayController::class, 'create'])
    ->name('student.delays.create');
Route::post('/studente/ritardi', [StudentDelayController::class, 'store'])
    ->name('student.delays.store');
```

In questo esempio si vede il collegamento tra le pagine dell'allievo e i controller. Le rotte GET mostrano le maschere, mentre le rotte POST salvano i dati inviati dai form.

4.2.3 Rotte del docente

Le rotte del docente gestiscono la consultazione e la validazione delle richieste. Il docente può visualizzare il dettaglio di assenze e ritardi, approvare, rifiutare o inserire osservazioni.

```
Route::get('/docente/assenze/{absence}', [DashboardTeacherController::class,
'showAbsence'])
    ->name('teacher.absences.show');

Route::post('/docente/assenze/{absence}/approva',[TeacherAbsenceController::clas
s, 'approve'])
    ->name('teacher.absences.approve');

Route::post('/docente/assenze/{absence}/rifiuta',[TeacherAbsenceController::clas
s, 'reject'])
    ->name('teacher.absences.reject');

Route::post('/docente/ritardi/{delay}/approva-senza-
firma',[TeacherDelayController::class, 'approveWithoutGuardian'])
    ->name('teacher.delays.approve-without-guardian');
```

Queste rotte mostrano il flusso di validazione delle assenze. L'identificativo dell'assenza viene passato nell'URL tramite {absence}, così il controller può recuperare la richiesta corretta e modificarne lo stato.

4.2.4 Rotte del capo laboratorio

Le rotte del capo laboratorio sono dedicate alla gestione dei congedi. Il sistema permette di visualizzare le richieste, approvarle, rifiutarle, richiedere documentazione o inoltrarle alla direzione.

```
Route::get('/laboratorio/congedi', [LeaveWorkflowController::class, 'index'])
    ->name('lab.leaves');

Route::get('/congedi/{leave}', [LeaveWorkflowController::class, 'show'])
    ->name('leaves.show');

Route::post('/congedi/{leave}/approva', [LeaveWorkflowController::class,
'approve'])
    ->name('leaves.approve');

Route::post('/congedi/{leave}/rifiuta', [LeaveWorkflowController::class,
'reject'])
    ->name('leaves.reject');
```

I congedi hanno un flusso separato rispetto alle assenze normali, perché richiedono più azioni di gestione. Per questo motivo sono stati collegati a un controller dedicato al workflow dei congedi.

4.2.5 Rotte amministrative

Le rotte amministrative permettono la gestione globale del sistema, come utenti, classi, configurazione, log errori e storiche interazioni.

```
Route::get('/admin/utenti', [DashboardAdminController::class,
'UserManagement'])->name('admin.users');

Route::get('/admin/classi', [DashboardAdminController::class,
'ClassesManagement'])->name('admin.classes');

Route::get('/admin/configurazione', [AdminSettingsController::class, 'edit'])-
>name('admin.settings');

Route::get('/admin/storico-interazioni', [DashboardAdminController::class,
'InteractionManagement'])->name('admin.interactions');
```

Le rotte amministrative sono separate dalle altre aree perché permettono operazioni più delicate, come la gestione degli utenti, delle classi e delle impostazioni generali.

4.2.6 Rotte pubbliche con token per il tutore

Alcune operazioni devono essere eseguite dal tutore senza obbligarlo ad avere un account nel sistema. Per questo motivo sono state create rotte pubbliche basate su token. Il token identifica la richiesta e permette al sistema di verificare se il link è valido.

```
Route::get('/tutore/assenze/firma/{token}',
[GuardianAbsenceConfirmationController::class, 'show'])
    ->name('guardian.absences.signature.show');

Route::post('/tutore/assenze/firma/{token}',
[GuardianAbsenceConfirmationController::class, 'store'])
    ->name('guardian.absences.signature.store');

Route::get('/tutore/congedi/firma/{token}',
[GuardianLeaveConfirmationController::class, 'show'])
    ->name('guardian.leaves.signature.show');

Route::post('/tutore/congedi/firma/{token}',
[GuardianLeaveConfirmationController::class, 'store'])
    ->name('guardian.leaves.signature.store');
```

Le rotte GET mostrano la pagina di firma, mentre le rotte POST salvano la decisione del tutore. L'accesso non dipende dal login, ma dalla validità del token ricevuto via email.

4.2.7 Riepilogo delle rotte principali

Area	Esempio rotta	Controller	Scopo
Dashboard	/dashboard	DashboardController	Reindirizzamento alla dashboard corretta
Allievo	/studente/assenze/nuova	AbsenceController	Creazione assenza
Docente	/docente/assenze/{absence}/approva	TeacherAbsenceController	Approvazione assenza
Laboratorio	/laboratorio/congedi	LeaveWorkflowController	Gestione congedi
Admin	/admin/utenti	DashboardAdminController	Gestione utenti
Tutore	/tutore/assenze/firma/{token}	GuardianAbsenceConfirmationController	Firma tramite token

Questa organizzazione delle rotte permette di mantenere separate le diverse aree dell'applicazione e rende più chiaro il collegamento tra URL, controller e funzionalità. I controlli specifici sui ruoli vengono poi gestiti nei controller e nei middleware dedicati.

4.3 Gestione ruoli e accesso alle dashboard

4.3.1 Ruoli previsti dal sistema

Il sistema prevede diversi ruoli utente, ognuno con funzionalità differenti. Il ruolo viene salvato direttamente nell'utente e viene utilizzato per stabilire quale dashboard mostrare e quali operazioni permettere.

I ruoli principali gestiti dall'applicazione sono: allievo, docente di classe, capo laboratorio e amministratore. Il tutore non accede tramite una dashboard interna, ma tramite link sicuro con token per confermare assenze, ritardi o congedi.

Ruolo	Codice nel sistema	Funzione principale
Allievo	student	Segnalazione assenze, ritardi, congedi e consultazione dati personali
Docente	teacher	Validazione assenze e ritardi degli allievi
Capo laboratorio	laboratory_manager	Gestione delle richieste di congedo
Amministratore	admin	Gestione globale di utenti, classi, impostazioni e log
Tutore	Accesso tramite token	Firma o rifiuto delle richieste ricevute via email

4.3.2 Controllo del ruolo utente

```
public function hasRole(string $role): bool
{
    return $this->role === $role;
}
```

Il controllo del ruolo viene centralizzato nel model User tramite il metodo hasRole. Questo metodo confronta il ruolo salvato nell'utente con quello richiesto dalla funzionalità. In questo modo i controller e i middleware possono verificare rapidamente se l'utente ha i permessi corretti.

4.3.3 Reindirizzamento alla dashboard corretta

```
return match (true) {
    $user->hasRole('admin')    => $DashboardAdminController->index(),
    $user->hasRole('teacher') => $TeacherAdminController->index(),
    $user->hasRole('laboratory_manager') => $LaboratoryManagerController-
>index(),
    default                    => $DashboardStudentController->index(),
};
```

Dopo il login, il sistema controlla il ruolo dell'utente autenticato e richiama il controller della dashboard corrispondente. Se l'utente è amministratore viene caricata la dashboard amministrativa, se è docente viene caricata quella del docente, se è capo laboratorio quella dedicata ai congedi. In caso contrario viene mostrata la dashboard dell'allievo.

4.3.4 Dashboard disponibili

Ruolo	Controller utilizzato	Dashboard mostrata
student	DashboardStudentController	Dashboard personale allievo
teacher	DashboardTeacherController	Dashboard docente
laboratory_manager	DashboardLaboratoryManagerController	Dashboard capo laboratorio
admin	DashboardAdminController	Dashboard amministrativa

Ogni dashboard presenta dati e azioni differenti. L'allievo visualizza le proprie richieste, il docente controlla gli allievi della propria classe, il capo laboratorio gestisce i congedi e l'amministratore accede alla gestione globale del sistema.

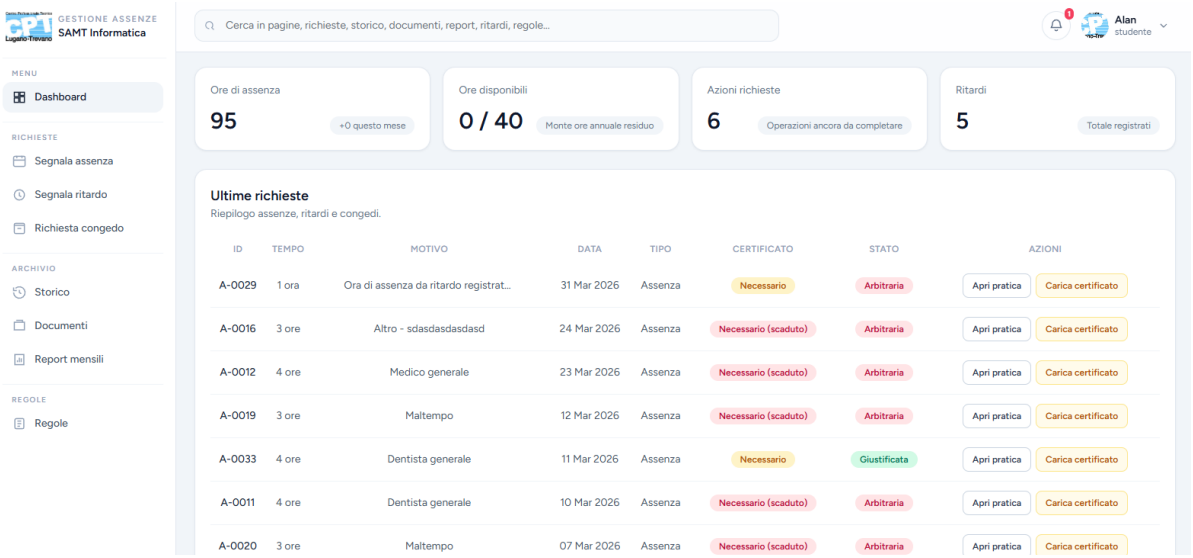


Figura 1 - Dashboard studente

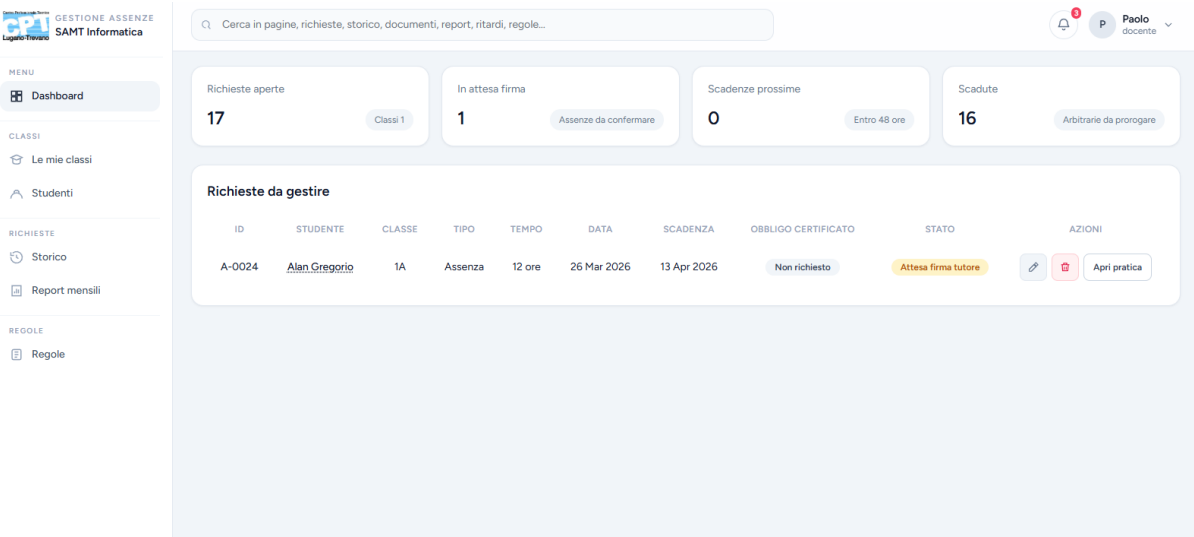


Figura 2 - Dashboard docente

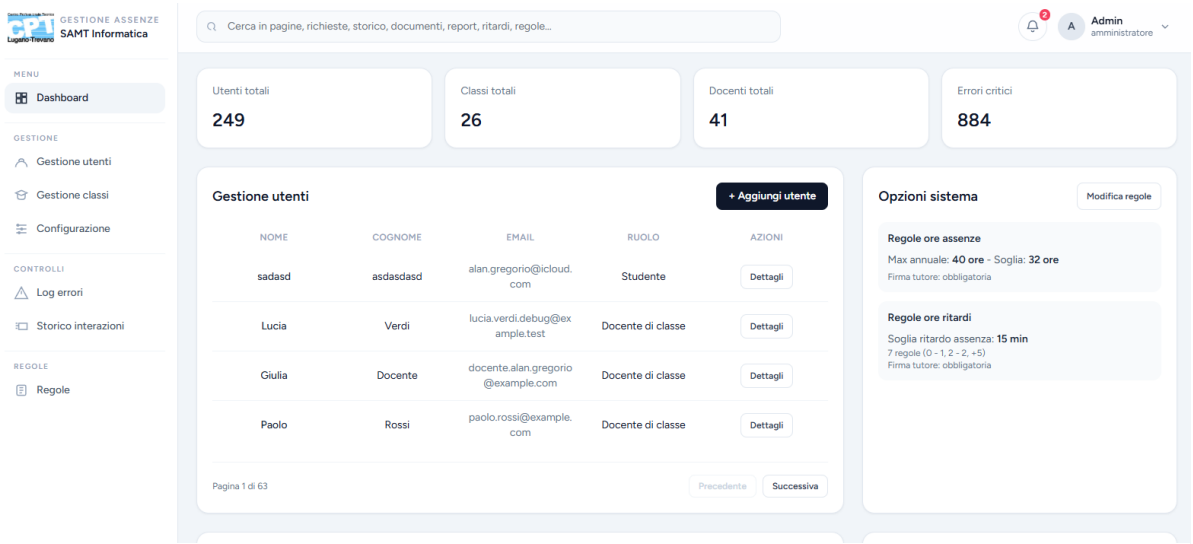


Figura 1 - Dashboard admin

4.3.5 Protezione delle funzionalità per ruolo

```
public function handle(Request $request, Closure $next): Response
{
    $user = $request->user();

    if (! $user || ! $user->hasRole('teacher')) {
        abort(403);
    }

    return $next($request);
}
```

Oltre al reindirizzamento alla dashboard corretta, il sistema utilizza middleware per bloccare l'accesso alle funzionalità riservate. Se l'utente non è autenticato o non possiede il ruolo richiesto, viene restituito un errore 403, impedendo l'accesso alla pagina.

4.3.6 Accesso del tutore tramite token

Il tutore è gestito in modo diverso rispetto agli altri ruoli. Non dispone di una dashboard interna, ma accede solo tramite link sicuri ricevuti via email. Il link contiene un token che permette di identificare la richiesta da confermare o rifiutare.

Questa scelta semplifica l'utilizzo per il tutore e riduce la necessità di creare un account separato per ogni genitore o responsabile legale.

```
private function createToken(Absence $absence, int $guardianId, bool
$forceResend): array
{
    if ($forceResend) {
        AbsenceConfirmationToken::query()
            ->where('absence_id', $absence->id)
            ->where('guardian_id', $guardianId)
            ->whereNull('used_at')
            ->update(['used_at' => now()]);
    }

    $plainToken = Str::random(80);
    $expiresAt = $absence->resolveMedicalCertificateDeadline()->copy()-
>endOfDay();

    if ($expiresAt->lt(now()->addMinutes(15))) {
        $expiresAt = now()->addMinutes(15);
    }

    $token = AbsenceConfirmationToken::create([
        'absence_id' => $absence->id,
        'guardian_id' => $guardianId,
        'token_hash' => hash('sha256', $plainToken),
        'expires_at' => $expiresAt,
    ]);

    return [$plainToken, $token];
}
```

Il sistema genera un token casuale e lo invia al tutore tramite email. Nel database non viene salvato il token in chiaro, ma solo il suo hash tramite sha256. In questo modo, anche se qualcuno accedesse alla tabella dei

token, non potrebbe usare direttamente il link di firma. Il token ha inoltre una scadenza, calcolata in base alla scadenza della giustificazione.

```
public function getTokenState(string $plainToken): array
{
    $token = $this->findTokenByPlainValue($plainToken);

    if (!$token) {
        return ['status' => 'invalid', 'token' => null];
    }

    if ($this->absenceAlreadySigned($token->absence_id)) {
        return ['status' => 'already_signed', 'token' => $token];
    }

    if ($token->used_at) {
        return ['status' => 'used', 'token' => $token];
    }

    if (Carbon::parse($token->expires_at)->isPast()) {
        return ['status' => 'expired', 'token' => $token];
    }

    return ['status' => 'valid', 'token' => $token];
}
```

Quando il tutore apre il link ricevuto via email, il sistema controlla lo stato del token. Il link può essere valido, scaduto, già usato, non valido oppure già firmato da un altro tutore. Solo se lo stato è valid viene mostrata la possibilità di firmare.

4.4 Layout base e dati condivisi con Inertia

4.4.1 Scopo del layout principale

```
<AuthenticatedLayout header="Dashboard">

</AuthenticatedLayout>
```

Il layout principale dell'applicazione viene utilizzato per mantenere una struttura grafica comune tra le varie pagine. Al suo interno sono presenti elementi condivisi come menu laterale, barra superiore, notifiche, ricerca e informazioni dell'utente autenticato.

In questo modo le singole pagine non devono ricreare ogni volta la struttura base dell'interfaccia, ma si concentrano solo sul contenuto specifico della funzionalità.

4.4.2 Dati condivisi da Laravel a React

Per rendere disponibili alcuni dati in tutte le pagine, il sistema utilizza il middleware HandleInertiaRequests. Questo middleware condivide automaticamente con il frontend le informazioni dell'utente autenticato, le notifiche e l'indice della ricerca globale.

Questa scelta evita di dover passare manualmente gli stessi dati da ogni controller.

```
public function share(Request $request): array
{
    return [
        ...parent::share($request),
        'auth' => [
            'user' => fn () => $this->serializeUser($request->user()),
        ],
        'notifications' => fn () => $this->buildNotificationCenter($request),
        'global_search' => fn () => $this->buildGlobalSearchIndex($request),
    ];
}
```

Il metodo share definisce i dati disponibili in tutte le pagine Inertia. La chiave auth contiene l'utente autenticato, notifications contiene le notifiche recenti e global_search contiene i dati usati dalla ricerca globale.

4.4.3 Serializzazione dell'utente autenticato

Prima di inviare i dati dell'utente al frontend, il sistema seleziona solo le informazioni necessarie. Questo permette di evitare l'invio di dati inutili o sensibili alla parte client.

```
private function serializeUser($user): ?array
{
    if (! $user) {
        return null;
    }

    return [
        'id' => $user->id,
        'name' => $user->name,
        'surname' => $user->surname,
        'email' => $user->email,
        'role' => $user->role,
        'email_verified_at' => $user->email_verified_at,
        'avatar_url' => $user->avatar_url,
    ];
}
```

Il metodo restituisce solo i dati necessari per il layout e per le funzionalità comuni. Il ruolo dell'utente viene usato dal frontend per mostrare menu e azioni coerenti con il profilo autenticato.

4.4.4 Utilizzo dei dati nel layout React

Nel layout React i dati condivisi da Laravel vengono letti tramite usePage. In questo modo il layout può accedere all'utente, alle notifiche e alla ricerca globale senza riceverli manualmente da ogni singola pagina.

```
const page = usePage();
const { auth, notifications, global_search: globalSearchIndex = [] } =
page.props;
const user = auth?.user;

const notificationItems = Array.isArray(notifications?.items) ?
notifications.items : [];
const unreadCount = Number(notifications?.unread_count ?? 0);
```

Il layout recupera i dati condivisi da Inertia e li utilizza per mostrare il nome dell'utente, il ruolo, le notifiche non lette e la ricerca globale. Questo permette di mantenere coerente l'interfaccia in tutte le aree dell'applicazione.

4.4.5 Menu laterale in base al ruolo

Il menu laterale viene costruito in base al ruolo dell'utente autenticato. In questo modo ogni utente vede solo i collegamenti utili per il proprio profilo, ad esempio assenze e ritardi per l'allievo, validazioni per il docente, congedi per il capo laboratorio e gestione globale per l'amministratore.

```
if (user?.role === 'student') {
…
}
if (user?.role === 'teacher') {
…
}
if (user?.role === 'laboratory_manager') {
…
}
```

4.4.6 Notifiche nel layout

Le notifiche vengono caricate insieme ai dati condivisi e mostrate direttamente nel layout. Il sistema mostra le notifiche recenti e il numero di notifiche non lette. L'utente può aprire una notifica oppure segnare tutte le notifiche come lette.

```
return [
    'items' => $items,
    'unread_count' => (int) $user->notifications()
        ->whereNull('read_at')
        ->count(),
];
```

Il backend restituisce sia l'elenco delle notifiche recenti sia il numero di notifiche non lette. Il frontend usa questi dati per aggiornare l'icona delle notifiche nella barra superiore.

4.4.7 Ricerca globale

Per l'allievo è stata aggiunta una ricerca globale che raccoglie assenze, ritardi e congedi. I dati vengono preparati lato server e inviati al frontend tramite Inertia, così il layout può cercare rapidamente tra le richieste personali dello studente.

```
return collect($absenceModel->getAbsence($user))
    ->merge($delayModel->getDelay($user))
    ->merge($leaveModel->getLeave($user))
    ->sortByDesc('date')
    ->take(140)
    ->values()
    ->map(function (array $item) {
        $id = trim((string) ($item['id'] ?? ''));
        $type = trim((string) ($item['tipo'] ?? 'Richiesta'));
        $date = trim((string) ($item['data'] ?? $item['date'] ?? ''));
        $rawDate = trim((string) ($item['date'] ?? ''));
        $reason = trim((string) ($item['motivo'] ?? ''));
        $duration = trim((string) ($item['durata'] ?? ''));
        $destination = trim((string) ($item['destinazione'] ??
       …

        return [
            'key' => $id !== '' ? 'request-'.$id : 'request-'.md5($tokenList),
            'type' => 'request',
            'icon' => match (strtolower($type)) {
                'assenza' => 'absence',
                'ritardo' => 'delay',
                'congedo' => 'leave',
                default => 'search',
            },
            'label' => $id !== '' ? $id : ($type !== '' ? $type : 'Richiesta'),
            'subtitle' => trim($type.' '.$date.' '.$reason),
            'tokens' => $tokenList,
            'url' => $id !== ''
                ? route('student.history', ['open' => $id])
                : route('student.history'),
        ];
    })
    ->all();
```

Questo codice costruisce l'indice della ricerca globale per l'allievo. Il sistema unisce assenze, ritardi e congedi in un'unica collezione, li ordina per data e prepara per ogni elemento le informazioni necessarie alla ricerca, come identificativo, tipo, icona, descrizione e link di apertura.

4.5 Segnalazione assenza da parte dell'allievo

4.5.1 Scopo della funzionalità

La segnalazione dell'assenza permette all'allievo di inserire una nuova richiesta indicando periodo, numero di ore e motivazione. Il sistema controlla i dati inseriti, salva l'assenza e imposta lo stato iniziale della richiesta.

Questa funzionalità rappresenta il punto di partenza del flusso di gestione delle assenze, che prosegue poi con eventuale firma del tutore, caricamento del certificato medico e validazione da parte del docente.

4.5.2 Apertura della maschera di segnalazione

Quando l'allievo apre la pagina di segnalazione assenza, il controller recupera le impostazioni configurate nel sistema e l'elenco delle motivazioni disponibili. Questi dati vengono poi inviati alla pagina React tramite Inertia.

```
public function create()
{

    $absenceSetting = AbsenceSetting::query()->firstOrFail();
    $absenceReasons = AbsenceReason::orderBy('name')->get();

    return Inertia::render('Student/AbsenceCreate', [
        'settings' => [
            'max_annual_hours' => $absenceSetting->max_annual_hours,
            'guardian_signature_required' => $absenceSetting-
>guardian_signature_required,
            'medical_certificate_days' => $absenceSetting-
>medical_certificate_days,
            'medical_certificate_max_days' => $absenceSetting-
>medical_certificate_max_days,
            'absence_countdown_days' => $absenceSetting->absence_countdown_days,
        ],
        'reasons' => $absenceReasons->map(fn (AbsenceReason $reason) => [
            'id' => $reason->id,
            'name' => $reason->name,
        ])->values(),
    ]);
}
```

Il metodo create prepara i dati necessari alla maschera. Le impostazioni servono per applicare le regole del sistema, mentre le motivazioni vengono mostrate all'allievo nel form.

4.5.3 Validazione dei dati inseriti

Prima di salvare l'assenza, i dati vengono validati tramite una request dedicata. Questo evita che vengano registrate richieste incomplete, con date non valide o con file non ammessi.

```
public function authorize(): bool
{
    $user = $this->user();

    return $user && $user->hasRole('student');
}

public function rules(): array
{
    return [
        'start_date' => ['required', 'date', 'before_or_equal:today'],
        'end_date' => ['nullable', 'date', 'after_or_equal:start_date',
'before_or_equal:today'],
        'hours' => ['required', 'integer', 'min:1'],
        'reason_choice' => ['required', 'string', 'max:255'],
        'motivation_custom' => ['nullable', 'string', 'max:255'],
        'motivation' => ['required', 'string', 'max:255'],
        'document' => ['nullable', 'file', 'mimes:pdf,jpg,jpeg,png',
'max:4096'],
    ];
}
```

Il metodo authorize permette l'uso della request solo agli utenti con ruolo student. Le regole controllano che le date non siano future, che le ore siano valide, che la motivazione sia presente e che l'eventuale documento caricato sia un PDF o un'immagine.

4.5.4 Controllo di assenze sovrapposte

Oltre alla validazione base, il sistema controlla che l'allievo non abbia già inserito un'assenza nello stesso periodo. Questo evita duplicati o richieste sovrapposte.

```
$startDateString = $startDate->toDateString();
$endDateString = $endDate->toDateString();
$overlapExists = Absence::query()
    ->where('student_id', $user->id)
    ->whereDate('start_date', '<=', $endDateString)
    ->whereDate('end_date', '>=', $startDateString)
    ->exists();
```

Il controllo verifica se esiste già un'assenza dello stesso studente che si sovrappone al periodo selezionato. Se viene trovata, il sistema blocca il salvataggio e restituisce un errore.

4.5.5 Creazione della richiesta di assenza

Dopo la validazione, il controller crea la nuova assenza. Vengono salvati lo studente, il periodo, la motivazione, le ore assegnate, lo stato iniziale e l'eventuale obbligo di certificato medico.

```
$absence = Absence::create([
    'student_id' => $user->id,
    'start_date' => $startDate->toDateString(),
    'end_date' => $endDate->toDateString(),
    'reason' => $motivation,
    'status' => Absence::STATUS_REPORTED,
    'assigned_hours' => $hours,
    'medical_certificate_required' => $certificateRequired,
    'medical_certificate_deadline' => $medicalDeadline->toDateString(),
]);
```

L'assenza viene creata con stato iniziale reported, cioè segnalata. Da questo momento la richiesta entra nel flusso di gestione e può essere controllata dal docente o completata con firma e documentazione.

4.5.6 Registrazione nello storico e invio firma tutore

Dopo il salvataggio, il sistema registra l'operazione nello storico e avvia il processo di firma del tutore, se previsto. In questo modo la creazione dell'assenza non rimane solo nel database, ma viene anche tracciata come operazione significativa.

```
OperationLog::record(
    $user,
    'absence.request.created',
    'absence',
    $absence->id,
    [
        'start_date' => $absence->start_date,
        'end_date' => $absence->end_date,
        'assigned_hours' => $absence->assigned_hours,
        'reason' => $absence->reason,
        'medical_certificate_required' => (bool) $absence-
>medical_certificate_required,
        'medical_certificate_deadline' => $absence-
>medical_certificate_deadline,
    ],
    'INFO',
    $request
);

$emailSummary = $guardianSignatureService->sendConfirmationEmails(
    $absence,
    $user,
    false,
    $request
);
```

perationLog::record salva nello storico i dati principali della richiesta. Successivamente il servizio AbsenceGuardianSignatureService gestisce l'eventuale invio dell'email di firma al tutore.

4.5.7 Flusso riassuntivo

Passaggio	File principale	Scopo
Apertura maschera	AbsenceController.php	Carica impostazioni e motivazioni
Compilazione form	AbsenceCreate.jsx	Inserimento dati da parte dell'allievo
Validazione	AbsenceRequest.php	Controllo dati, date, ore e documento
Controllo duplicati	AbsenceRequest.php	Evita assenze sovrapposte
Salvataggio	AbsenceController.php	Crea la richiesta di assenza
Storico	OperationLog	Registra l'operazione
Firma tutore	AbsenceGuardianSignatureService	Invia email di conferma, se necessaria

La segnalazione dell'assenza è quindi composta da più passaggi: visualizzazione del form, validazione dei dati, controllo delle sovrapposizioni, salvataggio della richiesta e avvio degli eventuali processi successivi. Questa struttura permette di mantenere il flusso ordinato e controllato fin dalla creazione dell'assenza.

4.6 Upload e gestione certificato medico

4.6.1 Scopo della funzionalità

Il sistema permette all'allievo di caricare un certificato medico collegato a un'assenza. Il certificato può essere inserito durante la creazione dell'assenza oppure successivamente dalla sezione documenti, se l'assenza lo richiede e se il termine non è scaduto.

Il file caricato viene validato, salvato nell'archivio dello studente e associato all'assenza corrispondente.

4.6.2 Validazione del file caricato

L'upload del certificato viene controllato tramite una request dedicata. Questa request permette l'operazione solo agli utenti con ruolo allievo e accetta solamente file PDF o immagini entro la dimensione massima prevista.

```
public function authorize(): bool
{
    $user = $this->user();

    return $user && $user->hasRole('student');
}

public function rules(): array
{
    return [
        'document' => ['required', 'file', 'mimes:pdf,jpg,jpeg,png',
'max:4096'],
    ];
}
```

Il metodo authorize impedisce l'upload a utenti che non sono allievi. Le regole di validazione controllano che il file sia obbligatorio, che sia realmente un file e che il formato sia tra quelli ammessi.

4.6.3 Controllo dell'assenza e dei permessi

Prima di salvare il certificato, il controller verifica che l'assenza appartenga allo studente autenticato. In questo modo un allievo non può caricare documenti su assenze di altri utenti.

```
public function uploadMedicalCertificate(UploadMedicalCertificateRequest
$request, Absence $absence)
{
    $absence->refresh();

    if ($absence->student_id !== $request->user()->id) {
        abort(403);
    }

    $statusCode = Absence::normalizeStatus($absence->status);
    if (! in_array($statusCode, [Absence::STATUS_REPORTED,
Absence::STATUS_JUSTIFIED], true)) {
        return back()->withErrors([
            'document' => 'Il certificato puo essere caricato solo su assenze
segnalate o giustificate entro il termine.',
        ]);
    }
```

Il primo controllo confronta l'id dello studente dell'assenza con quello dell'utente autenticato. Il secondo controllo permette l'upload solo per assenze in stato compatibile, cioè segnalate o giustificate.

4.6.4 Controllo della scadenza

Il sistema controlla anche la scadenza entro cui è possibile caricare il certificato. Se il termine è superato, l'assenza può essere aggiornata automaticamente secondo le regole previste e il caricamento viene bloccato.

```
$effectiveDeadline = $absence->syncMedicalCertificateDeadline();
if (Carbon::today()->gt($effectiveDeadline)) {
    $absence->markArbitraryForExpiredDeadlineIfNeeded();

    return back()->withErrors([
        'document' => 'Il termine per il caricamento del certificato e
scaduto.',
    ]);
}
```

Il metodo syncMedicalCertificateDeadline aggiorna la scadenza effettiva del certificato. Se la data attuale supera il termine, il sistema impedisce il caricamento e gestisce l'eventuale cambio di stato dell'assenza.

4.6.5 Prevenzione di certificati duplicati

Per evitare confusione nella gestione dei documenti, il sistema non permette di caricare un nuovo certificato se ne esiste già uno associato alla stessa assenza, salvo i casi in cui il docente lo rifiuti.

```
$existingCertificate = $absence->medicalCertificates()
    ->orderByDesc('uploaded_at')
    ->orderByDesc('id')
    ->first();
if ($existingCertificate) {
    return back()->withErrors([
        'document' => 'E gia presente un certificato per questa assenza. Puoi
caricarne uno nuovo solo se il docente lo rifiuta.',
    ]);
}
```

Il controllo recupera l'ultimo certificato collegato all'assenza. Se ne trova uno già presente, l'upload viene bloccato per evitare più documenti attivi sulla stessa richiesta.

4.6.6 Salvataggio del file nell'archivio studente

Quando tutti i controlli sono superati, il file viene salvato tramite StudentArchivePathBuilder. Questa classe crea un percorso organizzato per studente e categoria, evitando di salvare i documenti in modo disordinato.

```
$filePath = StudentArchivePathBuilder::storeUploadedFileForStudent(
    $request->file('document'),
    $request->user(),
    StudentArchivePathBuilder::CATEGORY_CERTIFICATES,
    [
        'context' => 'certificato_assenza',
        'code' => 'a'.str_pad((string) $absence->id, 4, '0', STR_PAD_LEFT),
    ]
);
```

Il primo controllo confronta l'id dello studente dell'assenza con quello dell'utente autenticato. Il secondo controllo permette l'upload solo per assenze in stato compatibile, cioè segnalate o giustificate.

4.6.7 Associazione del certificato all'assenza

Il sistema controlla anche la scadenza entro cui è possibile caricare il certificato. Se il termine è superato, l'assenza può essere aggiornata automaticamente secondo le regole previste e il caricamento viene bloccato.

```
$certificate = MedicalCertificate::create([
    'absence_id' => $absence->id,
    'file_path' => $filePath,
    'uploaded_at' => now(),
    'valid' => false,
]);
$absence->update([
    'certificate_rejection_comment' => null,
]);
```

Il certificato viene creato con stato iniziale non validato. La validazione viene effettuata successivamente dal docente. Eventuali commenti di rifiuto precedenti vengono rimossi dopo il nuovo caricamento.

4.6.8 Registrazione nello storico

Anche il caricamento del certificato viene registrato nelle storiche operazioni. Questo permette di sapere quando il documento è stato caricato, da chi e a quale assenza è stato associato.

```
OperationLog::record(
    $request->user(),
    'absence.certificate.uploaded',
    'medical_certificate',
    $certificate->id,
    [
        'absence_id' => $absence->id,
        'file_path' => $filePath,
        'source' => 'student_documents',
    ],
    'INFO',
    $request
);
```

Il log registra l'utente che ha caricato il certificato, l'id dell'assenza, il percorso del file e l'origine dell'operazione.

4.6.9 Flusso riassuntivo

Passaggio	File principale	Scopo
Validazione file	UploadMedicalCertificateRequest.php	Controlla formato, dimensione e ruolo utente
Controllo proprietario	AbsenceController.php	Verifica che l'assenza appartenga allo studente
Controllo stato	AbsenceController.php	Permette upload solo in stati compatibili
Controllo scadenza	AbsenceController.php	Blocca upload fuori termine
Salvataggio file	StudentArchivePathBuilder.php	Salva il certificato nell'archivio studente
Associazione dati	MedicalCertificate.php	Collega il certificato all'assenza
Storico	OperationLog	Registra l'operazione

La gestione del certificato medico non si limita al semplice caricamento del file, ma include controlli su ruolo, proprietà dell'assenza, stato della richiesta, scadenza e duplicati. Questo permette di mantenere i documenti ordinati e collegati correttamente alle assenze.

4.7 Firma tutore tramite token

4.7.1 Scopo della funzionalità

Per gli allievi minorenni, alcune richieste devono essere confermate dal tutore. Per evitare di creare un account dedicato per ogni tutore, il sistema utilizza un link sicuro inviato via email.

Il link contiene un token che identifica la richiesta da firmare. Il tutore può quindi aprire la pagina di firma, controllare i dati principali dell'assenza e confermare la richiesta tramite firma grafica.

4.7.2 Generazione del token

Quando viene richiesta la firma del tutore, il sistema genera un token casuale. Il token viene inviato al tutore tramite email, mentre nel database viene salvato solo il suo hash. Questa scelta evita di memorizzare direttamente il token utilizzabile nel link.

```
private function createToken(Absence $absence, int $guardianId, bool
$forceResend): array
{
    if ($forceResend) {
        AbsenceConfirmationToken::query()
            ->where('absence_id', $absence->id)
            ->where('guardian_id', $guardianId)
            ->whereNull('used_at')
            ->update(['used_at' => now()]);
    }

    $plainToken = Str::random(80);
    $expiresAt = $absence->resolveMedicalCertificateDeadline()->copy()-
>endOfDay();

    if ($expiresAt->lt(now()->addMinutes(15))) {
        $expiresAt = now()->addMinutes(15);
    }

    $token = AbsenceConfirmationToken::create([
        'absence_id' => $absence->id,
        'guardian_id' => $guardianId,
        'token_hash' => hash('sha256', $plainToken),
        'expires_at' => $expiresAt,
    ]);

    return [$plainToken, $token];
}
```

Il token in chiaro viene usato solo per costruire il link da inviare via email. Nel database viene salvato token_hash, cioè il valore cifrato tramite hash sha256. Il token contiene anche una scadenza, oltre la quale il link non è più valido.

4.7.3 Invio dell'email al tutore

```
[$plainToken, $token] = $this->createToken($absence, $guardian->id,
$forceResend);
$signatureUrl = route('guardian.absences.signature.show', ['token' =>
$plainToken]);
$subject = $this->buildSubject($absence);

try {
    Mail::to($guardian->email)
        ->send(new GuardianAbsenceSignatureMail($absence, $guardian,
$signatureUrl, $token->expires_at));
```

Il link inviato al tutore contiene il token in chiaro. Quando il tutore apre il link, il sistema può verificare il token confrontandone l'hash con quello salvato nel database.

4.7.4 Verifica dello stato del token

Quando il tutore apre il link ricevuto, il sistema controlla se il token è valido. Il link può essere non valido, già usato, scaduto oppure già firmato da un altro tutore.

```
public function getTokenState(string $plainToken): array
{
    $token = $this->findTokenByPlainValue($plainToken);

    if (! $token) {
        return ['status' => 'invalid', 'token' => null];
    }

    if ($this->absenceAlreadySigned($token->absence_id)) {
        return ['status' => 'already_signed', 'token' => $token];
    }

    if ($token->used_at) {
        return ['status' => 'used', 'token' => $token];
    }

    if (Carbon::parse($token->expires_at)->isPast()) {
        return ['status' => 'expired', 'token' => $token];
    }

    return ['status' => 'valid', 'token' => $token];
}
```

Il metodo restituisce lo stato del token. Solo se lo stato è valid il tutore può firmare la richiesta. In tutti gli altri casi la pagina mostra un messaggio di errore o di link non più utilizzabile.

4.7.5 Apertura della pagina di firma

Il controller riceve il token presente nell'URL e chiede al service di verificarne lo stato. Successivamente invia alla pagina React i dati necessari per mostrare la richiesta e decidere se la firma è permessa.

```
public function show(string $token, AbsenceGuardianSignatureService
$signatureService): Response
{
    $state = $signatureService->getTokenState($token);
    /** @var AbsenceConfirmationToken|null $tokenRecord */
    $tokenRecord = $state['token'];

    $absence = $tokenRecord?->absence;
    $guardian = $tokenRecord?->guardian;

    return Inertia::render('Guardian/AbsenceSignature', [
        'token' => $token,
        'status' => $state['status'],
        'signatureSuccess' => (bool) session('signature_success', false),
        'canSign' => $state['status'] === 'valid',
        'alreadySigned' => $state['status'] === 'already_signed',
        'signedAt' => $signedAt?->format('d/m/Y H:i'),
        'signedBy' => $signedBy,
        'absence' => $absence ? [
            'id' => 'A-'.str_pad((string) $absence->id, 4, '0', STR_PAD_LEFT),
            'student_name' => $studentName !== '' ? $studentName : '-',
            'start_date' => $absence->start_date?->format('d/m/Y'),
            'end_date' => $absence->end_date?->format('d/m/Y'),
            'hours' => (int) $absence->assigned_hours,
            'reason' => (string) ($absence->reason ?? '-'),
            'certificate_requirement' => $certificateRequirement,
        ] : null,
        'guardian' => $guardian ? [
            'name' => (string) ($guardian->name ?? '-'),
        ] : null,
        'oldInput' => [
            'full_name' => (string) old('full_name', $guardian?->name ?? ''),
            'consent' => (bool) old('consent', false),
            'signature_data' => (string) old('signature_data', ''),
        ],
    ]);
}
```

La pagina di firma riceve lo stato del token e i dati principali dell'assenza. Il campo canSign viene usato dal frontend per abilitare o bloccare la firma.

4.7.6 Salvataggio della firma grafica

Quando il tutore conferma la richiesta, il sistema valida i dati inseriti, decodifica la firma grafica e la salva come file PNG nell'archivio dello studente.

```
$disk = Storage::disk(config('filesystems.default', 'local'));
$signaturePath = StudentArchivePathBuilder::storeBinaryForStudent(
    $signatureBinary,
    $absence->student,
    StudentArchivePathBuilder::CATEGORY_SIGNATURES,
    'png',
    [
        'context' => 'firma_assenza',
        'code' => 'a'.str_pad((string) $tokenRecord->absence_id, 4, '0',
STR_PAD_LEFT)
            .'_g'.str_pad((string) $tokenRecord->guardian_id, 4, '0',
STR_PAD_LEFT),
        'guardian' => (string) ($tokenRecord->guardian?->name ??
$validated['full_name']),
    ]
);
```

Il sistema richiede nome del firmatario, consenso e firma grafica. La firma viene convertita da base64 a contenuto binario e salvata nell'archivio dello studente nella categoria dedicata alle firme.

4.7.7 Conferma della firma e utilizzo del token

Dopo aver salvato il file della firma, il sistema registra la conferma del tutore e marca il token come utilizzato. In questo modo lo stesso link non può essere usato più volte.

```
$confirmation = GuardianAbsenceConfirmation::query()->firstOrNew([
    'absence_id' => $lockedToken->absence_id,
    'guardian_id' => $lockedToken->guardian_id,
]);

$signedAt = now();
$confirmation->status = 'confirmed';
$confirmation->confirmed_at = $signedAt;
$confirmation->signed_at = $signedAt;
$confirmation->signature_path = $signaturePath;
$confirmation->ip_address = $request->ip();
$confirmation->notes = json_encode([
    'signer_name' => trim((string) $validated['full_name']),
    'signature_mode' => 'drawn_canvas',
    'user_agent' => (string) ($request->userAgent() ?? ''),
], JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES);
$confirmation->save();

$signatureService->markTokenAsUsed($lockedToken);
```

La conferma viene salvata con stato confirmed, data della firma, percorso del file e indirizzo IP. Dopo il salvataggio, il token viene marcato come usato, impedendo ulteriori firme con lo stesso link.

4.7.8 Registrazione nello storico

La firma del tutore viene registrata anche nello storico delle operazioni. Questo permette di ricostruire quando è stata confermata una richiesta e quale firma è stata salvata.

```
OperationLog::record(
    null,
    'absence.guardian.signature.confirmed',
    'absence',
    $lockedToken->absence_id,
    [
        'guardian_id' => $lockedToken->guardian_id,
        'signature_path' => $signaturePath,
    ],
    'INFO',
    $request
);
```

Il log registra l'id del tutore e il percorso della firma salvata. L'attore è null perché il tutore non accede con un account interno, ma tramite link sicuro.

4.7.9 Flusso riassuntivo

Passaggio	File principale	Scopo
Generazione token	AbsenceGuardianSignatureService.php	Crea il token e salva l'hash
Invio email	AbsenceGuardianSignatureService.php	Invia il link sicuro al tutore
Apertura link	GuardianAbsenceConfirmationController.php	Verifica il token e mostra la pagina
Firma grafica	Guardian/AbsenceSignature.jsx	Permette al tutore di firmare
Salvataggio firma	GuardianAbsenceConfirmationController.php	Salva la firma nell'archivio
Conferma richiesta	GuardianAbsenceConfirmationController.php	Registra la conferma del tutore
Token usato	AbsenceGuardianSignatureService.php	Impedisce il riutilizzo del link
Storico	OperationLog	Registra l'operazione

La firma tramite token permette al tutore di confermare una richiesta senza accedere al sistema con un account. Il controllo su hash, scadenza e utilizzo del token rende il link limitato alla singola operazione e impedisce riutilizzi non autorizzati.

4.8 Validazione assenza da parte del docente

4.8.1 Scopo della funzionalità

Dopo la segnalazione da parte dell'allievo, l'assenza deve essere controllata dal docente di classe. Il docente può visualizzare il dettaglio della richiesta, verificare eventuale firma del tutore e certificato medico, approvare l'assenza, rifiutarla oppure approvarla senza firma del tutore inserendo una motivazione.

Questa fase è importante perché determina lo stato finale dell'assenza e stabilisce se l'assenza deve essere conteggiata nelle 40 ore.

4.8.2 Accesso alle assenze della propria classe

Il docente non può validare qualsiasi assenza presente nel sistema. Prima di eseguire le operazioni, il controller recupera solo le assenze appartenenti agli allievi delle classi assegnate al docente.

```
private function resolveTeacherAbsence(User $teacher, int $absenceId): Absence
{
    return Absence::query()
        ->whereKey($absenceId)
        ->whereIn('student_id', function ($subQuery) use ($teacher) {
            $subQuery
                ->select('class_user.user_id')
                ->from('class_user')
                ->join('class_teacher', 'class_teacher.class_id', '=',
'class_user.class_id')
                ->where('class_teacher.teacher_id', $teacher->id);
        })
        ->with(['medicalCertificates', 'guardianConfirmations'])
        ->firstOrFail();
}
```

Questo metodo limita la ricerca alle assenze degli studenti appartenenti alle classi collegate al docente. Se l'assenza non appartiene a una sua classe, non viene trovata e l'operazione non può continuare.

4.8.3 Approvazione dell'assenza

L'approvazione standard è possibile solo per assenze ancora segnalate e con firma del tutore presente, quando richiesta. Dopo l'approvazione, lo stato dell'assenza viene aggiornato a giustificata.

```
public function approve(TeacherOptionalCommentRequest $request, Absence
$absence)
{

    $teacher = $request->user();
    $absence = $this->resolveTeacherAbsence($teacher, $absence->id);
    $statusCode = Absence::normalizeStatus($absence->status);

    if ($statusCode !== Absence::STATUS_REPORTED) {
        return back()->withErrors([
            'absence' => 'Puoi approvare solo assenze segnalate.',
        ]);
    }

    if (! $this->hasGuardianSignature($absence)) {
        return back()->withErrors([
            'absence' => 'Firma tutore non presente: usa approvazione senza
firma.',
        ]);
    }

    $absence->update([
        'status' => Absence::STATUS_JUSTIFIED,
        'approved_without_guardian' => false,
        'teacher_comment' => isset($validated['comment'])
            ? trim((string) $validated['comment'])
            : null,
        'counts_40_hours' => $counts40Hours,
        'counts_40_hours_comment' => $counts40Comment,
        'hours_decided_at' => now(),
        'hours_decided_by' => $teacher->id,
    ]);
    $absence->refresh();
    $this->syncMedicalCertificateRequirement($absence);



    return back()->with('success', 'Assenza approvata.');
}
```

Il metodo controlla che l'assenza sia ancora in stato reported e che sia presente la firma del tutore. Se i controlli sono superati, l'assenza passa allo stato justified.

4.8.4 Approvazione senza firma del tutore

In alcuni casi il docente può approvare un'assenza anche senza firma del tutore. Questa operazione richiede obbligatoriamente un commento, perché rappresenta un'eccezione rispetto al flusso normale.

```
public function rules(): array
{
    return [
        'comment' => ['required', 'string', 'max:1000'],
        'counts_40_hours' => ['required', 'boolean'],
        'counts_40_hours_comment' => [
            'nullable',
            'string',
            'max:1000',
            Rule::requiredIf(fn () => $this->boolean('counts_40_hours') ===
false),
        ],
    ];
}
```

La request obbliga il docente a inserire un commento quando approva senza firma. Inoltre richiede di specificare se l'assenza deve essere conteggiata nelle 40 ore.

```
$absence->update([
    'status' => Absence::STATUS_JUSTIFIED,
    'approved_without_guardian' => true,
    'teacher_comment' => trim((string) $validated['comment']),
    'counts_40_hours' => $counts40Hours,
    'counts_40_hours_comment' => $counts40Comment,
    'hours_decided_at' => now(),
    'hours_decided_by' => $teacher->id,
]);
```

L'assenza viene comunque giustificata, ma viene marcata con approved_without_guardian. In questo modo il sistema mantiene traccia del fatto che l'approvazione è avvenuta senza firma del tutore.

4.8.5 Rifiuto dell'assenza

Se il docente rifiuta l'assenza, la richiesta viene trasformata in assenza arbitraria. Il commento del docente è obbligatorio, così da motivare la decisione presa.

```
$absence->update([
    'status' => Absence::STATUS_ARBITRARY,
    'approved_without_guardian' => false,
    'teacher_comment' => trim((string) $validated['comment']),
    'counts_40_hours' => true,
    'counts_40_hours_comment' => 'Assenza rifiutata dal docente.',
    'hours_decided_at' => now(),
    'hours_decided_by' => $teacher->id,
]);
```

Il rifiuto porta l'assenza allo stato arbitrary. In questo caso l'assenza viene conteggiata nelle 40 ore e viene salvata la motivazione inserita dal docente.

4.8.6 Modifica dei dati dell'assenza

Il docente può anche rettificare alcuni dati dell'assenza, come date, ore, motivazione, stato e conteggio nelle 40 ore. Questa possibilità è utile quando la richiesta contiene errori o richiede una correzione manuale.

```
return [
    'start_date' => ['required', 'date', 'before_or_equal:today'],
    'end_date' => ['nullable', 'date', 'after_or_equal:start_date',
'before_or_equal:today'],
    'hours' => ['required', 'integer', 'min:1'],
    'motivation' => ['required', 'string', 'max:255'],
    'status' => [
        'nullable',
        'string',
        Rule::in([Absence::STATUS_REPORTED, Absence::STATUS_JUSTIFIED,
Absence::STATUS_ARBITRARY]),
    ],
    'counts_40_hours' => ['required', 'boolean'],
    'counts_40_hours_comment' => [
        'nullable',
        'string',
        'max:1000',
        Rule::requiredIf(fn () => $this->boolean('counts_40_hours') === false),
    ],
    'comment' => ['required', 'string', 'max:1000'],
];
```

La modifica richiede un commento obbligatorio e permette solo stati ammessi. Questo evita cambiamenti non controllati sui dati dell'assenza.

4.8.7 Validazione del certificato medico

Se l'allievo ha caricato un certificato medico, il docente può accettarlo o rifiutarlo. Quando il certificato viene accettato, viene segnato come valido. Se viene rifiutato, il file viene eliminato e viene salvato il commento di rifiuto.

```
$certificate->update([
    'valid' => true,
    'validated_by' => $teacher->id,
    'validated_at' => now(),
]);

$absence->update([
    'certificate_rejection_comment' => null,
]);

$comment = trim((string) $validated['comment']);
$disk = Storage::disk(config('filesystems.default', 'local'));
$filePath = (string) $certificate->file_path;
if ($filePath !== '' && $disk->exists($filePath)) {
    $disk->delete($filePath);
}
$certificateId = $certificate->id;
$certificate->delete();
$absence->update([
    'certificate_rejection_comment' => $comment,
]);
```

L'accettazione mantiene il certificato e lo marca come valido. Il rifiuto elimina il file e il record del certificato, lasciando però un commento visibile all'allievo per spiegare il motivo.

4.8.8 Registrazione nello storico

Ogni decisione del docente viene registrata nelle storiche operazioni. Questo permette di ricostruire chi ha approvato, rifiutato, modificato o validato un certificato.

```
OperationLog::record(
    $teacher,
    'absence.certificate.rejected',
    'medical_certificate',
    $certificateId,
    [
        'absence_id' => $absence->id,
        'comment' => $comment,
        'deleted_file_path' => $filePath,
    ],
    'INFO',
    $request
);
```

Il log salva il docente che ha effettuato l'azione, il tipo di operazione, l'assenza coinvolta e il cambio di stato eseguito.

4.8.9 Flusso riassuntivo

Azione	Stato iniziale	Stato finale	Note
Approvazione con firma	reported	justified	Richiede firma tutore
Approvazione senza firma	reported	justified	Richiede commento docente
Rifiuto	reported	arbitrary	Richiede commento docente
Modifica dati	reported, justified, arbitrary	Variabile	Richiede commento docente
Accetta certificato	Certificato presente	Certificato valido	Può escludere dalle 40 ore
Rifiuta certificato	Certificato presente	Certificato rimosso	Richiede commento

La validazione docente è il punto in cui l'assenza viene controllata e chiusa dal punto di vista amministrativo. I controlli su ruolo, classe, firma, stato e certificato permettono di evitare approvazioni non corrette e mantengono tracciate tutte le decisioni prese.

4.9 Gestione ritardi

4.9.1 Scopo della funzionalità

Il sistema permette all'allievo di segnalare un ritardo indicando data, durata in minuti e motivazione. Dopo la segnalazione, il docente può valutare il ritardo e decidere se giustificarlo oppure registrarlo nel conteggio dei ritardi.

La gestione dei ritardi è separata da quella delle assenze perché segue regole specifiche, come il conteggio nel semestre, eventuali notifiche automatiche e possibili scadenze per la giustificazione.

4.9.2 Validazione dei dati del ritardo

I dati inseriti dall'allievo vengono validati tramite una request dedicata. Il sistema accetta solo ritardi con data non futura, durata valida e motivazione obbligatoria.

```
public function authorize(): bool
{
    $user = $this->user();

    return $user && $user->hasRole('student');
}

public function rules(): array
{
    return [
        'delay_date' => ['required', 'date', 'before_or_equal:today'],
        'delay_minutes' => ['required', 'integer', 'min:1', 'max:480'],
        'motivation' => ['required', 'string', 'max:255'],
    ];
}
```

Il metodo authorize permette la creazione del ritardo solo agli utenti con ruolo allievo. Le regole impediscono l'inserimento di date future, durate non valide o motivazioni mancanti.

4.9.3 Apertura della maschera ritardo

Quando l'allievo apre la pagina per segnalare un ritardo, il controller recupera le impostazioni attive, come la soglia dei minuti e l'eventuale obbligo di firma del tutore.

```
public function create()
{
    $setting = $this->resolveDelaySetting();

    return Inertia::render('Student/DelayCreate', [
        'settings' => [
            'minutes_threshold' => (int) $setting->minutes_threshold,
            'guardian_signature_required' => (bool) $setting-
>guardian_signature_required,
        ],
    ]);
}
```

Le impostazioni vengono passate alla pagina React, così il frontend può mostrare all'allievo le informazioni corrette sulla gestione del ritardo.

4.9.4 Creazione della segnalazione ritardo

Dopo la validazione, il sistema crea il ritardo associandolo allo studente autenticato. Il ritardo viene inizialmente salvato con stato reported, cioè segnalato, e non viene ancora conteggiato nel semestre.

```
$delay = Delay::query()->create([
    'student_id' => $user->id,
    'recorded_by' => $user->id,
    'delay_datetime' => $delayDate,
    'minutes' => $delayMinutes,
    'justification_deadline' => $justificationDeadline?->toDateString(),
    'notes' => $motivation,
    'status' => Delay::STATUS_REPORTED,
    'count_in_semester' => false,
    'global' => false,
]);
```

Il ritardo viene salvato come segnalazione iniziale. Il campo count_in_semester viene impostato a false, perché il ritardo non viene conteggiato finché il docente non lo valuta.

4.9.5 Registrazione nello storico

Anche la segnalazione del ritardo viene registrata nello storico, così da poter ricostruire quando è stata creata e quali dati sono stati inseriti.

```
OperationLog::record(
    $user,
    'delay.request.created',
    'delay',
    $delay->id,
    [
        'delay_datetime' => $delayDate->toDateString(),
        'minutes' => $delayMinutes,
        'reason' => $motivation,
        'justification_deadline' => $justificationDeadline?->toDateString(),
        'guardian_signature_required' => $guardianSignatureRequired,
        'deadline_mode_active' => $deadlineModeActive,
        'delay_minutes_in_request' => $delayMinutes,
    ],
    'INFO',
    $request
);
```

Il log salva data, durata, motivazione, eventuale scadenza e informazione sull’obbligo di firma del tutore.

4.9.6 Validazione del ritardo da parte del docente

Il docente può giustificare il ritardo direttamente. In questo caso il ritardo passa allo stato justified e non viene conteggiato nel semestre.

```
$delay->update([
    'status' => Delay::STATUS_JUSTIFIED,
    'count_in_semester' => false,
    'teacher_comment' => $comment,
    'validated_at' => now(),
    'validated_by' => $teacher->id,
]);
```

Quando il ritardo viene approvato, viene marcato come giustificato. Il sistema salva anche il docente che ha effettuato la validazione e la data dell'operazione.

4.9.7 Registrazione del ritardo nel conteggio

Se il docente decide che il ritardo deve essere registrato, il sistema lo inserisce nel conteggio del semestre. In questo caso il ritardo passa allo stato registered e può generare notifiche o azioni automatiche in base alle regole configurate.

```
$delay->update([
    'status' => Delay::STATUS_REGISTERED,
    'count_in_semester' => true,
    'justification_deadline' => $registeredDeadline?->toDateString(),
    'teacher_comment' => $comment,
    'validated_at' => now(),
    'validated_by' => $teacher->id,
    'auto_arbitrary_at' => null,
]);
```

Lo stato registered indica che il ritardo è stato registrato dal docente e viene conteggiato. Se la gestione delle scadenze è attiva, viene anche impostato un termine per la giustificazione.

4.9.8 Approvazione senza firma del tutore

Se il ritardo richiede la firma del tutore ma la firma non è presente, il docente può approvare il ritardo senza firma solo in casi specifici e inserendo un commento obbligatorio.

```
$delay->update([
    'status' => Delay::STATUS_JUSTIFIED,
    'count_in_semester' => false,
    'teacher_comment' => $comment,
    'validated_at' => now(),
    'validated_by' => $teacher->id,
]);
```

Anche in questo caso il ritardo viene giustificato, ma la decisione viene tracciata nello storico come approvazione senza firma del tutore.

4.9.9 Applicazione delle regole sui ritardi

Quando un ritardo viene registrato nel conteggio, il sistema calcola quanti ritardi risultano nel semestre e verifica se esistono regole configurate da applicare. Le regole possono generare notifiche o altre azioni previste.

```
public static function evaluateForCount(int $count): array
{
    $applicableRules = self::resolveApplicableRules($count);
    $primaryRule = self::resolvePrimaryRule($applicableRules);
    $actions = self::normalizeActions($applicableRules);
    $infoMessages = $applicableRules
        ->map(fn (DelayRule $rule) => trim((string) ($rule->info_message ??
''')))
        ->filter(fn (string $message) => $message !== '')
        ->unique()
        ->values()
        ->all();

    return [
        'primary_rule' => $primaryRule,
        'applicable_rules' => $applicableRules,
        'actions' => $actions,
        'info_messages' => $infoMessages,
    ];
}
```

Il metodo riceve il numero di ritardi dello studente nel semestre e recupera le regole applicabili. Il risultato contiene la regola principale, le azioni da eseguire e i messaggi informativi da mostrare o inviare.

4.9.10 Accesso solo ai ritardi della propria classe

Come per le assenze, il docente può gestire solo i ritardi degli allievi appartenenti alle proprie classi.

```
private function resolveTeacherDelay(User $teacher, int $delayId): Delay
{
    return Delay::query()
        ->whereKey($delayId)
        ->whereIn('student_id', function ($subQuery) use ($teacher) {
            $subQuery
                ->select('class_user.user_id')
                ->from('class_user')
                ->join('class_teacher', 'class_teacher.class_id', '=',
'class_user.class_id')
                ->where('class_teacher.teacher_id', $teacher->id);
        })
        ->with(['student.guardians', 'guardianConfirmations'])
        ->firstOrFail();
}
```

Il metodo limita la ricerca ai ritardi degli studenti assegnati al docente. Questo evita che un docente possa modificare ritardi di classi non di sua competenza.

4.9.11 Flusso riassuntivo

Passaggio	Stato	Effetto
Allievo segnala ritardo	reported	Ritardo creato ma non conteggiato
Docente giustifica	justified	Ritardo approvato e non conteggiato
Docente registra	registered	Ritardo conteggiato nel semestre
Regole ritardi	variabile	Eventuali notifiche o azioni automatiche
Firma tutore	se richiesta	Conferma esterna tramite token
Proroga scadenza	registered	Estende il termine di giustificazione
Scadenza superata	registered / arbitrario	Il ritardo può diventare arbitrario

La gestione dei ritardi segue quindi un flusso separato rispetto alle assenze. L'allievo può segnalare il ritardo, mentre il docente decide se giustificarlo o registrarlo nel conteggio. Le regole configurate permettono di automatizzare notifiche e azioni in base al numero di ritardi accumulati dallo studente.

4.10 Gestione congedi

4.10.1 Scopo della funzionalità

La gestione dei congedi permette all'allievo di richiedere un'assenza pianificata, indicando periodo, ore o lezioni interessate, motivazione e destinazione. A differenza di una normale assenza, il congedo richiede un flusso di approvazione separato, gestito dal capo laboratorio.

Dopo l'invio, la richiesta può richiedere firma del tutore, documentazione aggiuntiva, approvazione, rifiuto o inoltro alla direzione.

4.10.2 Validazione della richiesta di congedo

I dati del congedo vengono validati tramite una request dedicata. Il sistema controlla date, ore, lezioni selezionate, motivazione, destinazione ed eventuale documento allegato.

```
class LeaveRequest extends FormRequest
{
    public function authorize(): bool
    {
        $user = $this->user();

        return $user && $user->hasRole('student');
    }

    public function rules(): array
    {
        return [
            'start_date' => ['required', 'date'],
            'end_date' => ['nullable', 'date', 'after_or_equal:start_date'],
            'hours' => ['nullable', 'integer', 'min:1'],
            'lessons_start' => ['nullable', 'array'],
            'lessons_start.*' => ['integer', 'between:1,11', 'distinct'],
            'lessons_end' => ['nullable', 'array'],
            'lessons_end.*' => ['integer', 'between:1,11', 'distinct'],
            'reason_choice' => ['required', 'string', 'max:255'],
            'motivation_custom' => ['nullable', 'string', 'max:255'],
            'motivation' => ['required', 'string', 'max:255'],
            'management_consent_confirmed' => ['nullable', 'boolean'],
            'destination' => ['required', 'string', 'max:255'],
            'document' => ['nullable', 'file', 'mimes:pdf,jpg,jpeg,png',
'max:4096'],
        ];
    }
```

La request permette la creazione solo agli allievi. Oltre ai campi principali, viene controllato anche l'eventuale documento allegato, accettando solo PDF o immagini.

4.10.3 Controllo di congedi sovrapposti

Prima di accettare la richiesta, il sistema verifica che lo studente non abbia già un altro congedo nello stesso periodo.

```
$startDateString = $startDate->toDateString();
$endDateString = $endDate->toDateString();
$overlappingLeaves = Leave::query()
    ->where('student_id', $user->id)
    ->whereNotIn('status', [
        Leave::STATUS_REJECTED,
        Leave::STATUS_FORWARDED_TO_MANAGEMENT,
    ])
    ->whereDate('start_date', '<=', $endDateString)
    ->whereDate('end_date', '>=', $startDateString)
    ->exists();

if ($overlappingLeaves) {
    $validator->errors()->add(
        'start_date',
        'Hai gia una richiesta congedo su uno o piu giorni selezionati.'
    );
}
```

Il controllo evita che lo stesso allievo inserisca più richieste di congedo sovrapposte, escludendo solo quelle già rifiutate o inoltrate alla direzione.

4.10.4 Creazione della richiesta

Dopo la validazione, il controller crea il congedo associandolo allo studente autenticato. La richiesta viene salvata con stato iniziale in attesa di firma del tutore.

```
$leave = Leave::create([
    'student_id' => $user->id,
    'created_by' => $user->id,
    'created_at_custom' => now(),
    'start_date' => $startDate->toDateString(),
    'end_date' => $endDate->toDateString(),
    'requested_hours' => max($requestedHours, 1),
    'hours_limit_exceeded_at_request' => $hoursLimitExceededAtRequest,
    'hours_limit_value_at_request' => $hoursOn40AtRequest,
    'hours_limit_max_at_request' => $maxAnnualHours,
    'requested_lessons' => $requestedLessonsPayload,
    'reason' => $motivation,
    'destination' => $destination,
    'status' => Leave::STATUS_AWAITING_GUARDIAN_SIGNATURE,
    'approved_without_guardian' => false,
    'count_hours' => $countHours,
    'count_hours_comment' => $countHoursComment,
    'documentation_path' => null,
    'documentation_uploaded_at' => null,
    'workflow_comment' => null,
    'hours_decision_at' => null,
    'hours_decision_by' => null,
]);
```

Il congedo viene salvato con periodo, ore richieste, eventuali lezioni selezionate, motivazione e destinazione. Lo stato iniziale indica che la richiesta attende la firma del tutore.

4.10.5 Upload della documentazione

Se necessario, l'allievo può allegare un documento alla richiesta. Il file viene salvato nell'archivio dello studente e collegato al congedo.

```
if ($request->hasFile('document')) {
    $documentationPath = StudentArchivePathBuilder::storeUploadedFileForStudent(
        $request->file('document'),
        $user,
        StudentArchivePathBuilder::CATEGORY_SLIPS,
        [
            'context' => 'talloncino_congedo',
            'code' => 'c'.str_pad((string) $leave->id, 4, '0', STR_PAD_LEFT),
        ]
    );

    $leave->update([
        'documentation_path' => $documentationPath,
        'documentation_uploaded_at' => now(),
        'workflow_comment' => 'Documentazione allegata in fase di creazione
richiesta.',
    ]);
}
```

Il documento viene salvato nella categoria dedicata ai talloncini/congedi. Il percorso viene poi registrato nel congedo, così il capo laboratorio può visualizzarlo durante la valutazione.

4.10.6 Invio firma tutore

Dopo la creazione del congedo, il sistema avvia il flusso di firma del tutore. Il funzionamento è simile a quello delle assenze: viene generato un token e inviato un link sicuro via email.

```
$emailSummary = $signatureService->sendConfirmationEmails(
    $leave,
    $user,
    false,
    $request
);
```

Il servizio LeaveGuardianSignatureService si occupa dell'invio dell'email di firma. Il controller usa il riepilogo restituito per mostrare un messaggio all'allievo.

4.10.7 Gestione da parte del capo laboratorio

Le richieste di congedo vengono gestite dal capo laboratorio tramite un workflow dedicato. Il capo laboratorio può visualizzare le richieste aperte, aprire il dettaglio e decidere quale azione eseguire.

Azione	Scopo
Approvare	Accetta il congedo
Approvare senza firma	Permette un override della firma tutore
Rifiutare	Respinge la richiesta con commento
Richiedere documentazione	Chiede un allegato aggiuntivo
Rifiutare documentazione	Richiede un nuovo documento
Inoltrare alla direzione	Passa la richiesta a valutazione superiore
Modificare	Corregge alcuni dati del congedo

4.10.8 Approvazione del congedo

Per approvare un congedo, il sistema controlla che la richiesta sia ancora aperta e che sia presente la firma del tutore oppure un'approvazione senza firma. Durante l'approvazione viene anche deciso se il congedo deve essere conteggiato nelle ore annuali.

```
$leave->update([
    'status' => Leave::STATUS_APPROVED,
    'approved_without_guardian' => ! $guardianSigned,
    'workflow_comment' => $comment !== '' ? $comment : null,
    'count_hours' => $countHours,
    'count_hours_comment' => $countHoursComment,
    'hours_decision_at' => now(),
    'hours_decision_by' => $actor->id,
]);
```

Il congedo passa allo stato approvato. Vengono salvati anche l'eventuale commento, la decisione sul conteggio ore e l'utente che ha preso la decisione.

4.10.9 Registrazione del congedo come assenza

Dopo l'approvazione, il congedo viene registrato e può generare una bozza di assenza. Questo passaggio collega il workflow del congedo alla gestione normale delle assenze.

```
$absence = Absence::create([
    'student_id' => $leave->student_id,
    'derived_from_leave_id' => $leave->id,
    'start_date' => $leave->start_date?->toDateString(),
    'end_date' => $leave->end_date?->toDateString(),
    'reason' => (string) $leave->reason,
    'status' => $status,
    'assigned_hours' => (int) $leave->requested_hours,
    'counts_40_hours' => (bool) $leave->count_hours,
    'counts_40_hours_comment' => $countHoursComment,
    'approved_without_guardian' => false,
    'teacher_comment' => $teacherComment,
    'medical_certificate_required' => false,
    'medical_certificate_deadline' => $medicalDeadline,
    'hours_decided_at' => null,
    'hours_decided_by' => null,
]);
```

L'assenza generata resta inizialmente in stato bozza. Lo studente dovrà poi completarla e inviarla come assenza ufficiale.

4.10.10 Rifiuto del congedo

Se il capo laboratorio rifiuta il congedo, la richiesta passa allo stato rifiutato e viene salvato il commento con la motivazione.

```
$leave->update([
    'status' => Leave::STATUS_REJECTED,
    'workflow_comment' => $comment,
    'approved_without_guardian' => false,
]);
```

```
OperationLog::record(
    $actor,
    'leave.rejected',
    'leave',
    $leave->id,
    [
        'comment' => $comment,
        'status' => $leave->status,
    ],
    'INFO',
    $request
);
```

Il rifiuto viene salvato sia nel congedo sia nello storico operazioni. In questo modo rimane tracciata la decisione presa dal capo laboratorio.

4.10.11 Flusso riassuntivo

Passaggio	File principale	Scopo
Compilazione richiesta	Student/LeaveCreate.jsx	Inserimento dati da parte dell'allievo
Validazione	LeaveRequest.php	Controllo date, motivazione, lezioni e documento
Creazione congedo	StudentLeaveController.php	Salva la richiesta
Firma tutore	LeaveGuardianSignatureService.php	Invia link sicuro al tutore
Gestione workflow	LeaveWorkflowController.php	Valutazione capo laboratorio
Documentazione	StudentArchivePathBuilder.php	Salvataggio allegati
Approvazione	LeaveWorkflowController.php	Approva e registra il congedo
Bozza assenza	LeaveAbsenceDraftService.php	Crea assenza derivata dal congedo
Storico	OperationLog	Traccia le decisioni

La gestione dei congedi è stata separata dalle assenze normali perché richiede più passaggi decisionali. Il workflow permette di controllare firma, documentazione, approvazione, rifiuto e successiva trasformazione del congedo in bozza di assenza.

4.11 Invio email e notifiche interne

4.11.1 Scopo della funzionalità

Il sistema utilizza notifiche interne ed email per informare gli utenti sugli eventi principali dell'applicazione. Le notifiche interne vengono mostrate direttamente nell'interfaccia, mentre le email vengono usate per comunicazioni esterne o importanti, come la firma del tutore, i report mensili e gli aggiornamenti sui congedi.

Per evitare di ripetere la stessa logica nei controller, parte della gestione delle notifiche è stata centralizzata in servizi dedicati.

4.11.2 Invio notifiche interne

Le notifiche interne vengono gestite tramite il servizio NotificationDispatcher. Questo servizio riceve l'utente destinatario, il tipo di evento e il contenuto della notifica. Prima di inviare la notifica controlla che l'utente esista e sia attivo.

```
public function notifyUser(?User $user, string $eventKey, array $content): void
{
    if (! $user || ! (bool) $user->active) {
        return;
    }

    $user->notify(new SystemMessageNotification($eventKey, $content));
}
```

I metodo evita l'invio di notifiche a utenti non validi o disattivati. Se l'utente è valido, viene creata una SystemMessageNotification, che Laravel salva o invia in base ai canali configurati.

4.11.3 Invio a più destinatari

Per alcuni eventi è necessario notificare più utenti, ad esempio docenti, capi laboratorio o amministratori. In questi casi il sistema normalizza i destinatari, rimuove duplicati e permette anche di escludere l'utente che ha generato l'azione.

```
public function notifyUsers(iterable $users, string $eventKey, array $content,
?int $excludeUserId = null): void
{
    $this->normalizeRecipients($users)
        ->reject(fn (User $user) => $excludeUserId !== null && (int) $user->id
=== $excludeUserId)
        ->each(function (User $user) use ($eventKey, $content) {
            $user->notify(new SystemMessageNotification($eventKey, $content));
        });
}
```

Questo metodo viene usato quando una stessa notifica deve essere inviata a più persone. La normalizzazione evita invii duplicati e limita la notifica agli utenti attivi.

4.11.4 Risoluzione dei destinatari

Per sapere a chi inviare una notifica, il sistema usa il servizio NotificationRecipientResolver. Questo servizio recupera i destinatari corretti in base al contesto, ad esempio i docenti di uno studente, i docenti di una classe, i capi laboratorio o gli amministratori.

```
public function teachersForStudent(int $studentId): Collection
{
    return User::query()
        ->select('users.*')
        ->join('class_teacher', 'class_teacher.teacher_id', '=', 'users.id')
        ->join('class_user', 'class_user.class_id', '=',
'class_teacher.class_id')
        ->where('class_user.user_id', $studentId)
        ->where('users.role', 'teacher')
        ->where('users.active', true)
        ->distinct()
        ->get();
}
```

Il metodo recupera i docenti collegati alla classe dello studente. In questo modo una notifica relativa a uno studente viene inviata solo ai docenti competenti.

4.11.5 Scelta del canale di notifica

La classe SystemMessageNotification decide se una notifica deve essere salvata nel database, inviata via email o entrambe le cose. La scelta dipende dalle preferenze configurate per l'utente e dal tipo di evento.

```
public function via(object $notifiable): array
{
    if (! $notifiable instanceof User) {
        return ['database'];
    }

    $channels = [];

    if (NotificationPreference::webEnabledFor($notifiable, $this->eventKey)) {
        $channels[] = 'database';
    }

    if (
        (bool) $notifiable->active
        && filled($notifiable->email)
        && NotificationPreference::emailEnabledFor($notifiable, $this->eventKey)
    ) {
        $channels[] = 'mail';
    }

    return $channels;
}
```

Il metodo via stabilisce i canali da usare. Se le notifiche web sono abilitate, la notifica viene salvata nel database. Se l'utente ha email attiva e le preferenze lo permettono, viene inviata anche via email.

4.11.6 Contenuto della notifica interna

Il contenuto della notifica viene trasformato in un array utilizzato dal frontend per mostrare titolo, testo, icona, link e azione associata.

```
public function toArray(object $notifiable): array
{
    return [
        'event_key' => $this->eventKey,
        'title' => (string) ($this->content['title'] ?? 'Nuova notifica'),
        'body' => (string) ($this->content['body'] ?? ''),
        'url' => $this->content['url'] ?? null,
        'icon' => (string) ($this->content['icon'] ?? 'system'),
        'action_label' => (string) ($this->content['action_label'] ?? 'Apri'),
        'action_type' => (string) ($this->content['action_type'] ?? 'open'),
    ];
}
```

Questi dati vengono poi letti dal layout dell'applicazione per mostrare le notifiche nella barra superiore.

4.11.7 Invio email specifiche

Alcune email hanno un contenuto specifico e vengono gestite tramite classi Mail dedicate. Un esempio è l'email inviata al tutore per firmare un'assenza. In questo caso il sistema passa alla mail l'assenza, il tutore, il link di firma e la scadenza del token.

```
public function __construct(
    public Absence $absence,
    public Guardian $guardian,
    public string $signatureUrl,
    public ?CarbonInterface $expiresAt
) {}
```

Il costruttore riceve tutti i dati necessari per costruire l'email. Il link di firma viene generato prima dal servizio e poi inserito nel messaggio inviato al tutore.

4.11.8 Oggetto e contenuto dell'email

L'oggetto dell'email viene costruito in base allo studente associato all'assenza. Il contenuto viene invece caricato da una vista Blade dedicata.

```
public function envelope(): Envelope
{
    $studentName = trim((string) ($this->absence->student?->name ?? '').'
'.(string) ($this->absence->student?->surname ?? ''));

    return new Envelope(
        subject: $studentName !== ''
            ? 'Conferma assenza - '.$studentName
            : 'Conferma assenza studente',
    );
}

public function content(): Content
{
    return new Content(
        view: 'mail.guardian-absence-signature',
        with: [
            'absence' => $this->absence,
            'guardian' => $this->guardian,
            'signatureUrl' => $this->signatureUrl,
            'expiresAt' => $this->expiresAt,
            'studentName' => trim((string) ($this->absence->student?->name ??
'').' '.(string) ($this->absence->student?->surname ?? '')),
        ],
    );
}
```

La mail utilizza una vista dedicata e riceve i dati necessari per mostrare al tutore il riferimento dell'assenza e il link sicuro di firma.

4.11.9 Tracciamento degli invii email

Per alcune comunicazioni, oltre all'invio dell'email, il sistema salva anche un record con destinatario, oggetto, corpo, stato e data di invio. Questo permette di distinguere email inviate correttamente da email fallite.
Nel flusso di firma del tutore, dopo l'invio dell'email viene creato un record nella tabella delle notifiche email dell'assenza. In caso di errore viene registrato lo stato failed e viene salvato anche un log operativo.

```
AbsenceEmailNotification::create([
    'type' => $forceResend ? 'guardian_signature_resend' :
'guardian_signature_request',
    'recipient_email' => $guardian->email,
    'subject' => $subject,
    'body' => $this->buildNotificationBody($absence, $token->expires_at,
$forceResend),
    'absence_id' => $absence->id,
    'sent_at' => now(),
    'status' => 'sent',
]);
```

Questo record permette di sapere quale email è stata inviata, a quale destinatario e per quale assenza.

4.11.10 Flusso riassuntivo

Elemento	File principale	Scopo
Dispatcher notifiche	NotificationDispatcher.php	Invio notifiche interne
Risoluzione destinatari	NotificationRecipientResolver.php	Recupero utenti da notificare
Notifica generica	SystemMessageNotification.php	Gestione canali database/email
Email firma assenza	GuardianAbsenceSignatureMail.php	Invio link firma tutore
Preferenze notifiche	NotificationPreference	Abilitazione/disabilitazione canali
Storico email	AbsenceEmailNotification, LeaveEmailNotification, DelayEmailNotification	Tracciamento invii

La gestione centralizzata di notifiche ed email permette di separare la logica di comunicazione dai controller. In questo modo il sistema può notificare gli utenti corretti, rispettare le preferenze configurate e mantenere traccia degli invii effettuati.

4.12 Archiviazione documenti per studente

4.12.1 Scopo dell'archiviazione

Il sistema gestisce diversi tipi di documenti collegati agli studenti, come certificati medici, firme del tutore, documentazione dei congedi e report mensili. Per evitare una gestione disordinata dei file, è stata creata una struttura di archiviazione basata sull'identificativo dello studente.

Ogni documento viene salvato in una cartella specifica in base allo studente e alla categoria del file. In questo modo è più semplice recuperare i documenti collegati a una determinata persona.

4.12.2 Struttura dell'archivio

L'archivio viene organizzato partendo dall'id dello studente. All'interno della cartella dello studente vengono poi create sottocartelle per le diverse categorie di documenti.

```
archivio/
  260/
    certificati_medici/
    firme/
    talloncini/
    report_mensili/
      2026/
        03/
          report_mensile_2026_03.pdf
          firmati/
            report_mensile_2026_03_firmato.pdf
```

Nell'esempio, 260 rappresenta l'id dello studente. Le cartelle interne separano certificati medici, firme, talloncini e report mensili.

4.12.3 Categorie dei documenti

Le categorie principali dell'archivio sono definite nella classe StudentArchivePathBuilder.

```
class StudentArchivePathBuilder
{
    public const CATEGORY_CERTIFICATES = 'certificati_medici';
    public const CATEGORY_SIGNATURES = 'firme';
    public const CATEGORY_SLIPS = 'talloncini';
}
```

Queste costanti permettono di usare sempre gli stessi nomi di cartella quando il sistema salva certificati, firme o documentazione dei congedi.

4.12.4 Costruzione del percorso di salvataggio

Il percorso del file viene costruito automaticamente usando l'id dello studente e la categoria del documento. Questo evita di scrivere manualmente i percorsi nei controller.

```
private static function buildDirectory(User $student, string $category): string
{
    $normalizedCategory = static::normalizeCategory($category);

    return 'archivio/'.max((int) $student->id, 0).'/'.$normalizedCategory;
}
```

Il metodo restituisce un percorso del tipo archivio/{id_studente}/{categoria}. La categoria viene normalizzata prima di essere usata, così il sistema salva i file solo in cartelle previste.

4.12.5 Salvataggio di un file caricato dall'utente

Quando l'utente carica un file, il sistema usa il metodo storeUploadedFileForStudent. Questo metodo sceglie la cartella corretta, costruisce un nome file leggibile e salva il documento.

```
public static function storeUploadedFileForStudent(
    UploadedFile $file,
    User $student,
    string $category,
    array $meta = []
): string {
    $disk = Storage::disk(config('filesystems.default', 'local'));
    $directory = static::buildDirectory($student, $category);
    $baseName = static::buildBaseName($student, $meta);
    $extension = static::normalizeExtension(
        (string) ($file->getClientOriginalExtension() ?: $file->extension() ?:
'bin')
    );
    $fileName = static::resolveUniqueFileName($disk, $directory, $baseName,
$extension);

    return $file->storeAs($directory, $fileName);
}
```

Il metodo riceve il file, lo studente, la categoria e alcuni metadati. Da questi dati genera il percorso e il nome del file, poi salva il documento nel disco configurato da Laravel.

4.12.6 Generazione del nome del file

Il nome del file non viene lasciato casuale, ma viene costruito includendo informazioni utili come contesto, codice della richiesta, studente, classe, tutore e data di salvataggio.

```
$parts = [
    $contextLabel,
    $codeLabel,
    $studentCode,
    $studentName,
    $classLabel,
    $guardianLabel,
    $timestampLabel,
];

return collect($parts)
    ->filter(fn ($part) => $part !== '')
    ->implode('_');
```

Il nome viene composto usando solo le parti disponibili. Questo rende i file più riconoscibili e riduce il rischio di nomi duplicati.

4.12.7 Prevenzione dei file duplicati

Se un file con lo stesso nome esiste già, il sistema aggiunge automaticamente un numero progressivo al nome del file.

```
private static function resolveUniqueFileName($disk, string $directory, string
$baseName, string $extension): string
{
    $candidate = $baseName.'.'.$extension;
    $increment = 1;

    while ($disk->exists($directory.'/'.$candidate)) {
        $candidate = $baseName.'_'.$increment.'.'.$extension;
        $increment++;
    }

    return $candidate;
}
```

Questo metodo evita che un nuovo caricamento sovrascriva un file già presente. Se il nome è già occupato, viene aggiunto _1, _2 e così via.

4.12.8 Archiviazione dei report mensili

I report mensili usano una struttura leggermente diversa, perché devono essere ordinati per anno e mese. Per questo è stata creata la classe MonthlyReportArchivePathBuilder.

```
public static function originalPath(int $studentId, Carbon $month): string
{
    return static::baseDirectory($studentId, $month)
        .'/report_mensile_'
        .$month->format('Y_m')
        .'.pdf';
}

public static function signedPath(int $studentId, Carbon $month, string
$extension = 'pdf'): string
{
    return static::baseDirectory($studentId, $month)
        .'/firmati/report_mensile_'
        .$month->format('Y_m')
        .'_firmato.'
        .static::normalizeExtension($extension);
}
```

Il report originale e quello firmato vengono salvati in percorsi separati. Il report firmato viene inserito nella sottocartella firmati, mantenendo ordinati i documenti mensili.

4.12.9 Esempi di utilizzo dell'archivio

Tipo documento	Categoria / percorso	Esempio utilizzo
Certificato medico	certificati_medici	Upload certificato per assenza
Firma tutore	firme	Salvataggio firma grafica
Documentazione congedo	talloncini	Allegato per richiesta congedo
Report mensile	report_mensili/anno/mese	PDF mensile generato dal sistema
Report firmato	report_mensili/anno/mese/firmati	PDF restituito firmato

4.12.10 Flusso riassuntivo

Passaggio	File principale	Scopo
Scelta categoria	StudentArchivePathBuilder.php	Decide la cartella del documento
Costruzione percorso	StudentArchivePathBuilder.php	Crea archivio/{studente}/{categoria}
Generazione nome	StudentArchivePathBuilder.php	Crea un nome file riconoscibile
Controllo duplicati	StudentArchivePathBuilder.php	Evita sovrascritture
Salvataggio file	Storage Laravel	Scrive il documento nel filesystem
Report mensili	MonthlyReportArchivePathBuilder.php	Organizza i report per anno e mese

L'archiviazione per studente permette di mantenere ordinati i documenti generati o caricati nel sistema. La costruzione automatica dei percorsi riduce la duplicazione di codice nei controller e rende più semplice collegare ogni file allo studente e alla richiesta corrispondente.

4.13 Report mensili

4.13.1 Scopo del report mensile

Il sistema genera un report mensile per ogni allievo attivo. Il report contiene un riepilogo delle informazioni principali del mese, come assenze, ritardi, congedi, ore conteggiate nelle 40 ore, certificati mancanti e richieste non ancora firmate.

Questa funzionalità è stata realizzata lato backend, perché i dati devono essere calcolati in modo automatico e coerente per tutti gli studenti. Il frontend viene usato principalmente per consultare i report generati, caricare eventuali versioni firmate e approvarle.

4.13.2 Raccolta dei dati del report

I dati del report vengono preparati dal servizio MonthlyReportDataService. Questo servizio riceve lo studente e il mese di riferimento, poi recupera dal database tutte le informazioni utili.

```
$absences = Absence::query()
    ->where('student_id', $student->id)
    ->whereDate('start_date', '<=', $monthEnd->toDateString())
    ->whereDate('end_date', '>=', $monthStart->toDateString())
    ->with([
        'medicalCertificates',
        'guardianConfirmations',
        'derivedFromLeave',
    ])
    ->get();

$delays = Delay::query()
    ->where('student_id', $student->id)
    ->whereBetween('delay_datetime', [
        $monthStart->copy()->startOfDay(),
        $monthEnd->copy()->endOfDay(),
    ])
    ->with('guardianConfirmations')
    ->get();

$leaves = Leave::query()
    ->where('student_id', $student->id)
    ->whereDate('start_date', '<=', $monthEnd->toDateString())
    ->whereDate('end_date', '>=', $monthStart->toDateString())
    ->get();
```

Il servizio non si limita a leggere i dati, ma calcola anche i valori riassuntivi usati nel PDF, ad esempio:

```
return [
    'class' => $class,
    'summary' => [
        'month' => $monthStart->toDateString(),
        'month_label' => $this->monthLabel($monthStart),
        'absence_hours' => $absenceHours,
        'absence_count' => (int) $absences->count(),
        'delay_count' => $delayCount,
        'leave_count' => $leaveCount,
        'leave_hours' => $leaveHours,
        'hours_40' => [
            'month_counted' => $monthly40Hours,
            'annual_counted' => $annual40Hours,
            'limit' => $max40Hours,
            'remaining' => $remaining40Hours,
        ],
        'medical_certificates' => [
            'required' => (int) $requiredCertificates->count(),
            'uploaded' => (int) $uploadedCertificates->count(),
            'missing' => (int) max($missingCertificates, 0),
            'missing_absence_codes' => $missingCertificateCodes,
        ],
        'unsigned_absence_count' => $unsignedAbsences,
        'unsigned_delay_count' => $unsignedDelays,
        'arbitrary_hours' => $arbitraryHours,
        'penalties' => $penalties,
        'generated_at' => now()->toDateTimeString(),
    ],
];
```

In questo modo il report viene generato partendo dai dati reali presenti nel sistema e non da valori inseriti manualmente.

4.13.3 Generazione dei report per gli studenti

La generazione viene gestita dal servizio MonthlyReportService. Quando viene avviata la procedura mensile, il sistema recupera tutti gli studenti attivi e mette in coda la generazione del report per ognuno.

```
$students = User::query()
    ->where('role', 'student')
    ->where('active', true)
    ->pluck('id');

foreach ($students as $studentId) {
    \App\Jobs\GenerateMonthlyReportForStudentJob::dispatch(
        (int) $studentId,
        $month->copy()->startOfMonth()->toDateString(),
        $actorUserId
    );
}
```

Questa scelta evita di generare tutti i PDF in una sola richiesta. Ogni report viene elaborato separatamente tramite job, rendendo il processo più stabile anche quando ci sono molti studenti.

4.13.4 Creazione del PDF

Dopo aver raccolto i dati, il sistema genera il PDF tramite MonthlyReportPdfService.

```
public function generate(
    MonthlyReport $report,
    User $student,
    ?SchoolClass $class,
    array $summary
): string {
    $month = Carbon::parse($report->report_month)->startOfMonth();
    $path = $this->buildPath((int) $student->id, $month);
    $payload = $this->buildPayload($student, $class, $summary, $report, $month);
    $content = $this->pdfBuilder->buildSchoolMonthlyReport($payload);

    Storage::disk('local')->put($path, $content);

    return $path;
}
```

Il PDF viene quindi creato a partire da un payload strutturato, che contiene i dati dello studente, della classe e del riepilogo mensile.

4.13.5 Dati inseriti nel PDF

Nel PDF vengono mostrate diverse sezioni riassuntive. Una parte del payload viene costruita in questo modo:

```
'summary_rows' => [
    ['label' => 'Ore di assenza', 'value' => (string) ((int)
($summary['absence_hours'] ?? 0))],
    ['label' => 'Numero assenze', 'value' => (string) ((int)
($summary['absence_count'] ?? 0))],
    ['label' => 'Numero congedi', 'value' => (string) ((int)
($summary['leave_count'] ?? 0))],
    ['label' => 'Ore arbitrarie', 'value' => (string) ((int)
($summary['arbitrary_hours'] ?? 0))],
    ['label' => 'Ritardi nel mese', 'value' => (string) ((int)
($summary['delay_count'] ?? 0))],
],
```

Questa parte permette di trasformare i dati calcolati dal backend in righe leggibili nel documento PDF.

4.13.6 Archiviazione del report

Il report generato viene salvato nell'archivio dello studente, nella cartella dedicata ai report mensili.

archivio/
260/
report_mensili/
2026/
03/
report_mensile_2026_03.pdf
firmati/
report_mensile_2026_03_firmato.pdf

Il percorso viene costruito tramite MonthlyReportArchivePathBuilder, così ogni report viene salvato in modo ordinato per studente, anno e mese.

4.13.7 Invio email del report

Dopo la generazione, il sistema può inviare il report via email allo studente e ai suoi tutori.

```
$recipientEmails = collect()
    ->push($report->student?->email)
    ->merge(
        $report->student?->guardians
            ?->pluck('email')
            ->all() ?? []
    )
    ->map(fn ($email) => strtolower(trim((string) $email)))
    ->filter(fn (string $email) => $email !== '')
    ->unique()
    ->values();

foreach ($recipientEmails as $recipientEmail) {
…
Mail::to($recipientEmail)->send(new MonthlyReportMail($report));
…
}
```

Ogni invio viene anche registrato nel database tramite una notifica email, così il sistema mantiene traccia dei report inviati e di eventuali errori.

4.13.8 Upload del report firmato

Quando il report firmato viene ricevuto, può essere caricato nel sistema. Il file viene salvato nella cartella dei report firmati dello studente.

```
$storedPath = $file->storeAs(
    dirname(MonthlyReportArchivePathBuilder::signedPath(
        (int) $report->student_id,
        $month,
        $extension
    )),
    basename(MonthlyReportArchivePathBuilder::signedPath(
        (int) $report->student_id,
        $month,
        $extension
    )),
    'local'
);
if (! is_string($storedPath) || trim($storedPath) === '') {
    throw new \RuntimeException('Impossibile salvare il report firmato.');
}

$report->signed_pdf_path = $storedPath;
$report->signed_uploaded_at = now();
$report->status = MonthlyReport::STATUS_SIGNED_UPLOADED;
$report->approved_at = null;
$report->approved_by = null;
$report->save();
```

Il sistema salva il report firmato nella cartella corretta dello studente usando MonthlyReportArchivePathBuilder. Dopo il salvataggio, aggiorna il record del report impostando il percorso del file firmato, la data di caricamento e lo stato signed_uploaded. Inoltre vengono azzerati eventuali dati di approvazione precedente, perché un nuovo file firmato deve essere ricontrollato.

4.13.9 Approvazione del report

Dopo il caricamento della versione firmata, il report può essere approvato. L'approvazione aggiorna lo stato del report e registra quale utente ha eseguito l'operazione.

```
$report->status = MonthlyReport::STATUS_APPROVED;
$report->approved_at = now();
$report->approved_by = $actor->id;
$report->save();
```

Quando il report firmato viene approvato, il sistema aggiorna lo stato del report a approved, salva la data di approvazione e registra l'utente che ha eseguito l'operazione. L'approvazione viene inoltre salvata nello storico tramite OperationLog, così l'azione rimane tracciabile.

4.13.10 Generazione tramite comando

La generazione dei report mensili può essere avviata anche tramite comando Artisan.

```
protected $signature = 'reports:generate-monthly {--month= : Mese riferimento
formato YYYY-MM}';

protected $description = 'Genera i report mensili per tutti gli studenti e invia
email a studenti/tutori';
```

Il comando permette di specificare il mese oppure, se non viene indicato, usa il mese precedente.

```
$summary = $service->queueGenerationForMonth($month);

$this->info('Generazione report mensili messa in coda.');
$this->line('Mese: '.$summary['month']);
$this->line('Studenti processati: '.$summary['students']);
```

Questo rende la funzionalità utilizzabile sia manualmente sia tramite scheduler automatico.

4.13.11 Tabella riassuntiva

Parte	File principale	Scopo
Raccolta dati	MonthlyReportDataService	Recupera assenze, ritardi, congedi e calcola i valori mensili
Generazione report	MonthlyReportService	Coordina la creazione e l'invio dei report
Creazione PDF	MonthlyReportPdfService	Genera il documento PDF
Archiviazione	MonthlyReportArchivePathBuilder	Costruisce i percorsi dei report
Invio email	MonthlyReportMail	Invia il report a studente e tutori
Upload firmato	MonthlyReportService	Salva la versione firmata del report
Comando automatico	GenerateMonthlyReports	Avvia la generazione mensile

La gestione dei report mensili è stata implementata come processo automatico lato server. Il sistema raccoglie i dati dello studente, genera il PDF, lo archivia nella cartella corretta e lo invia ai destinatari previsti. In questo modo il report mensile rimane riproducibile, tracciabile e collegato ai dati reali presenti nell'applicazione.

4.14 Comandi automatici e scheduler

4.14.1 Scopo dei comandi automatici

Alcune operazioni del sistema non devono dipendere dall'azione manuale di un utente. Per questo sono stati creati comandi automatici che controllano periodicamente lo stato delle assenze, dei ritardi, dei congedi, dei report mensili e dei log.

Le operazioni principali automatizzate sono:

Comando	Scopo
absences:mark-arbitrary	Imposta come arbitrarie le assenze scadute
delays:mark-arbitrary	Registra automaticamente i ritardi scaduti
delays:resend-expired-signature-tokens	Reinvia le email per firme tutore dei ritardi con token scaduto
leaves:register-due-absences	Crea bozze di assenza dai congedi approvati
reports:generate-monthly	Genera i report mensili degli studenti
logs:prune-retention	Elimina i log più vecchi secondo la retention
students:promote-adult-guardian	Aggiorna il tutore quando lo studente diventa maggiorenne

4.14.2 Pianificazione nello scheduler

La pianificazione dei comandi si trova nel file routes/console.php.

```
$timezone = 'Europe/Zurich';
$schoolWindowStart = '06:00';
$schoolWindowEnd = '20:00';

Schedule::command('absences:mark-arbitrary')
    ->hourlyAt(5)
    ->between($schoolWindowStart, $schoolWindowEnd)
    ->timezone($timezone)
    ->withoutOverlapping();

Schedule::command('delays:mark-arbitrary')
    ->hourlyAt(10)
    ->between($schoolWindowStart, $schoolWindowEnd)
    ->timezone($timezone)
    ->withoutOverlapping();

Schedule::command('delays:resend-expired-signature-tokens')
    ->hourlyAt(20)
    ->between($schoolWindowStart, $schoolWindowEnd)
    ->timezone($timezone)
    ->withoutOverlapping();
```

Questi comandi vengono eseguiti ogni ora, ma solo nella fascia oraria scolastica dalle 06:00 alle 20:00.
Il metodo withoutOverlapping() evita che lo stesso comando venga eseguito più volte contemporaneamente.

4.14.3 Generazione automatica dei report mensili

La generazione dei report mensili viene pianificata una volta al mese.

```
Schedule::command('reports:generate-monthly')
    ->monthlyOn(1, '05:30')
    ->timezone($timezone)
    ->withoutOverlapping();
```

Il comando viene eseguito il primo giorno del mese alle 05:30. In questo modo il sistema genera i report del mese precedente prima dell'inizio della giornata scolastica.

```
protected $signature = 'reports:generate-monthly {--month= : Mese riferimento
formato YYYY-MM}';

protected $description = 'Genera i report mensili per tutti gli studenti e invia
email a studenti/tutori';
```

Nel metodo handle, il comando calcola il mese da elaborare e avvia la generazione tramite MonthlyReportService.

```
public function handle(MonthlyReportService $service): int
{
    $month = $this->resolveMonth();
    if (! $month) {
        return self::FAILURE;
    }

    $summary = $service->queueGenerationForMonth($month);

    $this->info('Generazione report mensili messa in coda.');
    $this->line('Mese: '.$summary['month']);
    $this->line('Studenti processati: '.$summary['students']);

    return self::SUCCESS;
}
```

Il comando può anche ricevere un mese specifico tramite opzione, ad esempio 2026-03.

4.14.4 Uso dei job per i report mensili

Per evitare di generare tutti i report in una sola esecuzione, il sistema usa un job dedicato per ogni studente.

```
class GenerateMonthlyReportForStudentJob implements ShouldQueue
{
    use Queueable;

    public function __construct(
        public int $studentId,
        public string $month,
        public ?int $actorUserId = null
    ) {}

    public function handle(MonthlyReportService $service): void
    {
        $month = Carbon::parse($this->month)->startOfMonth();
        $actor = $this->actorUserId
            ? User::query()->find($this->actorUserId)
            : null;

        $service->generateAndSendForStudent($this->studentId, $month, $actor);
    }
}
```

Questa scelta rende il processo più stabile, perché ogni report viene elaborato separatamente.

4.14.5 Comando per assenze arbitrarie

Il comando per le assenze arbitrarie si trova in: MarkAbsencesArbitrary.php

```
protected $signature = 'absences:mark-arbitrary';

protected $description = 'Imposta come arbitrarie le assenze scadute ancora
aperte';

public function handle(): int
{
    $updated = Absence::applyAutomaticArbitrary();

    $this->info('Assenze impostate come arbitrarie: '.$updated);

    return self::SUCCESS;
}
```

Il comando richiama il metodo applyAutomaticArbitrary() del model Absence, che controlla le assenze ancora aperte e scadute, aggiornandone automaticamente lo stato.$

4.14.6 Comando per ritardi scaduti

Il comando dei ritardi si trova in: MarkDelaysArbitrary.php

```
protected $signature = 'delays:mark-arbitrary';

protected $description = 'Registra automaticamente i ritardi scaduti ancora
aperti';

public function handle(): int
{
    $updated = Delay::applyAutomaticArbitrary();

    $this->info('Ritardi registrati automaticamente per scadenza: '.$updated);

    return self::SUCCESS;
}
```

Questo comando serve per evitare che i ritardi restino indefinitamente in attesa. Se la scadenza viene superata, il sistema li aggiorna automaticamente secondo le regole configurate.

4.14.7 Pulizia dei log

La pulizia dei log viene pianificata ogni giorno.

```
Schedule::command('logs:prune-retention')
    ->dailyAt('01:45')
    ->timezone($timezone);
```

Il comando si trova in: PruneOperationLogs.php

```
protected $signature = 'logs:prune-retention';

protected $description = 'Elimina i log operativi oltre la retention
configurata';

public function handle(): int
{
    $result = OperationLog::pruneByConfiguredRetention();

    $this->info(
        'Prune completato. INFO eliminati: '.$result['info_deleted']
        .' | WARNING/ERROR eliminati: '.$result['error_deleted']
    );
    $this->line(
        'Cutoff INFO: '.$result['interaction_cutoff']
        .' | Cutoff WARNING/ERROR: '.$result['error_cutoff']
    );

    return self::SUCCESS;
}
```

Questo comando evita che la tabella dei log cresca senza limite, mantenendo comunque le informazioni necessarie per la tracciabilità.

4.14.8 Aggiornamento tutore al compimento dei 18 anni

Il sistema gestisce anche il caso in cui uno studente diventa maggiorenne.

```
Schedule::command('students:promote-adult-guardian')
    ->dailyAt('02:15')
    ->timezone($timezone)
    ->withoutOverlapping();
```

Il comando si trova in: PromoteAdultStudentGuardians.php
Questo comando controlla gli studenti che hanno compiuto 18 anni, assegna lo studente come proprio tutore e invia le notifiche necessarie.
La scelta è importante perché, dopo la maggiore età, non è più necessario far firmare le richieste al tutore precedente.

4.14.9 Tabella riassuntiva

File	Comando	Frequenza	Frequenza
MarkAbsencesArbitrary.php	absences:mark-arbitrary	Ogni ora al minuto 5	Assenze scadute
MarkDelaysArbitrary.php	delays:mark-arbitrary	Ogni ora al minuto 10	Ritardi scaduti
ResendExpiredDelaySignatureTokens.php	delays:resend-expired-signature-tokens	Ogni ora al minuto 20	Reinvia token firma ritardi
RegisterDueLeaveAbsences.php	leaves:register-due-absences	Ogni ora al minuto 30	Crea assenze da congedi
GenerateMonthlyReports.php	reports:generate-monthly	Primo giorno del mese alle 05:30	Report mensili
PruneOperationLogs.php	logs:prune-retention	Ogni giorno alle 01:45	Pulizia log
PromoteAdultStudentGuardians.php	logs:prune-retention	Ogni giorno alle 02:15	Gestione studenti maggiorenni

L’utilizzo dei command e dello scheduler permette al sistema di eseguire automaticamente controlli e operazioni ripetitive. In questo modo assenze, ritardi, congedi, report mensili e log vengono gestiti in modo regolare, senza dipendere da interventi manuali degli utenti.

4.15 Script Python di supporto

4.15.1 Scopo dello script Python

Lo script Python è stato aggiunto per analizzare il PDF del calendario scolastico e ricavare automaticamente le date evidenziate come vacanze o giorni particolari.

Questa parte è stata realizzata in Python perché la lettura interna del PDF richiede l'analisi dei contenuti grafici, come colori, rettangoli e celle della tabella. Gestire questa logica direttamente in Laravel sarebbe stato più complesso e meno pratico.

La dipendenza usata è definita nel file scripts/requirements.txt:
pypdf>=6.0.0

4.15.2 Analisi del PDF

Lo script apre il PDF tramite PdfReader e legge sia il testo sia le operazioni grafiche presenti nelle pagine.

```
def analyze_pdf(pdf_path: Path) -> dict:
    reader = PdfReader(str(pdf_path))

    op_counter: Counter[str] = Counter()
    rgb_color_counter: Counter[tuple[float, float, float]] = Counter()
    gray_color_counter: Counter[float] = Counter()
    scn_counter: Counter[str] = Counter()
    filled_rectangles_by_fill: Counter[str] = Counter()
    fill_rect_samples: dict[str, list[tuple[float, float, float, float]]] = {}
    fill_rectangles_all: dict[str, list[tuple[float, float, float, float]]] = {}
    text_items: list[tuple[float, float, str]] = []
    page_previews: list[dict] = []
```

Lo script non legge solo il testo visibile, ma controlla anche le forme disegnate nel PDF. Questo è necessario perché i giorni del calendario sono riconosciuti in base ai colori presenti nelle celle.

4.15.3 Riconoscimento dei colori

```
if op_name in {"scn", "sc"}:
    parts: list[str] = []
    for value in operands:
        if isinstance(value, (int, float)):
            parts.append(f"{float(value):.4f}")
        else:
            parts.append(str(value))
    signature = ",".join(parts)
    current_fill = f"{op_name}:{signature}"
    scn_counter[current_fill] += 1
    continue
```

In questo modo il sistema può distinguere i colori usati nel calendario, ad esempio il verde scuro per le date da importare e il rosso per le celle non valide del mese.

4.15.4 Conversione delle celle in date

Dopo aver trovato i rettangoli colorati, lo script li associa alla riga del mese e alla colonna del giorno.

```
def map_fill_rectangles_to_cells(
    rectangles: list[tuple[float, float, float, float]],
    unique_x: list[float],
    unique_y: list[float],
    cell_height: float,
) -> set[tuple[int, int]]:
```

Il risultato di questa funzione è una lista di celle nel formato:
(row_index, day_number)

Successivamente lo script trasforma queste informazioni in date reali, ad esempio:
date_obj = dt.date(year, month_number, day_number)
dark_dates.append(date_obj.isoformat())

Il formato finale della data è YYYY-MM-DD, così può essere letto facilmente da Laravel.

4.15.5 Output JSON dello script

Lo script restituisce sempre un output JSON. Questo è importante perché Laravel deve poter leggere il risultato in modo strutturato.

```
payload = {
    "dates": result["dark_dates"],
    "metadata": {
        "month_order": result["month_order"],
        "month_order_source": result["month_order_source"],
        "columns": len(result["unique_x"]),
        "rows": len(result["unique_y"]),
        "dark_signature": result["dark_green_signature"],
        "red_signature": result["red_signature"],
        "red_by_row": {
            str(k): sorted(v) for k, v in result["red_by_row"].items()
        },
        "dark_by_row": {
            str(k): sorted(v) for k, v in result["dark_by_row"].items()
        },
    },
}
print(json.dumps(payload, ensure_ascii=False))
```

Nel JSON vengono restituite le date trovate e alcune informazioni tecniche utili per controllare come è stato interpretato il PDF.

4.15.6 Collegamento con Laravel

Laravel esegue lo script tramite il servizio SchoolHolidayPdfExtractor.

```
$process = new Process([
    ...$this->pythonCommand(),
    $scriptPath,
    '--json',
    $absolutePdfPath,
], base_path());
$process->setTimeout(90);
$process->run();
```

Il servizio passa allo script il percorso assoluto del PDF caricato. Dopo l'esecuzione, legge l'output JSON.

```
$output = trim($process->getOutput());

$decoded = json_decode($output, true);
if (! is_array($decoded)) {
    throw new RuntimeException('Output parser calendario non valido.');
}
```

Le date vengono poi filtrate e normalizzate:

```
$dates = collect($rawDates)
    ->map(fn ($value) => trim((string) $value))
    ->filter(fn (string $value) => preg_match('/^\d{4}-\d{2}-\d{2}$/', $value)
=== 1)
    ->unique()
    ->sort()
    ->values()
    ->all();
```

Questo controllo evita di importare valori non validi nel database.

4.15.7 Importazione nel database

L'import viene avviato dal controller amministrativo, nel metodo importHolidaysFromPdf.

```
$uploadedFile = $validated['calendar_pdf'];
$disk = Storage::disk(config('filesystems.default', 'local'));
$storedPath = $disk->putFile('private/configurazione/calendari', $uploadedFile);
$absolutePath = $disk->path($storedPath);

$parsed = $extractor->extract($absolutePath);
```

Dopo l'analisi del PDF, le date vengono preparate per il salvataggio.

```
foreach ($importedDates as $date) {
    $normalizedDate = Carbon::parse((string) $date)->toDateString();
    $schoolYear = SchoolHoliday::schoolYearFromDate($normalizedDate);
    $schoolYears[$schoolYear] = true;
    $upsertPayload[] = [
        'holiday_date' => $normalizedDate,
        'school_year' => $schoolYear,
        'label' => null,
        'source' => SchoolHoliday::SOURCE_PDF_IMPORT,
        'source_file_path' => $storedPath,
        'created_by' => $actor?->id,
        'updated_by' => $actor?->id,
        'created_at' => $timestamp,
        'updated_at' => $timestamp,
    ];
}
```

Infine il sistema elimina le vecchie date importate da PDF per lo stesso anno scolastico e salva quelle nuove.

```
DB::transaction(function () use ($schoolYears, $upsertPayload): void {
    SchoolHoliday::query()
        ->whereIn('school_year', array_keys($schoolYears))
        ->where('source', SchoolHoliday::SOURCE_PDF_IMPORT)
        ->delete();

    SchoolHoliday::query()->upsert(
        $upsertPayload,
        ['holiday_date'],
        [
            'school_year',
            'label',
            'source',
            'source_file_path',
            'updated_by',
            'updated_at',
        ]
    );
});
```

4.16 Storiche operazioni e log

Il servizio Laravel controlla se esiste l'ambiente Python del progetto e, se necessario, prova a prepararlo automaticamente.

```
private function ensureProjectVirtualEnvironment(): void
{
    $venvPython = $this->venvPythonBinary();
    if ($venvPython !== null && $this->commandIsUsable([$venvPython], true)) {
        return;
    }

    $bootstrapCommand = $this->resolveUsableCommand(
        $this->systemPythonCandidates(),
        false
    );

}
```

Se l'ambiente virtuale non è pronto, il servizio prova a creare .venv e installare le dipendenze presenti in requirements.txt.

```
$this->runBestEffort(
    new Process(
        [
            $createdVenvPython,
            '-m',
            'pip',
            'install',
            '-r',
            $requirementsPath,
        ],
        base_path()
    ),
    240
);
```

Questa scelta rende l'importazione più robusta, perché il sistema tenta di preparare automaticamente l'ambiente necessario allo script.

4.16.1 Tabella riassuntiva

Parte	Tecnologia	Funzione
Lettura PDF	Python + pypdf	Analisi del calendario scolastico
Script	calendar_pdf_probe.py	Estrae le date dal PDF
Output	JSON	Passaggio dati da Python a Laravel
Servizio Laravel	SchoolHolidayPdfExtractor	Esegue lo script e valida l'output
Controller	AdminSettingsController	Salva le date nel database
Database	SchoolHoliday	Memorizza le vacanze importate

Lo script Python è stato usato come supporto esterno per una parte specifica del sistema: l'analisi del calendario scolastico in formato PDF. Laravel rimane responsabile della validazione, del salvataggio e dello storico dell'operazione, mentre Python si occupa solo dell'estrazione tecnica delle date dal documento.

4.16.2 Storico operazioni e log

4.16.3 Scopo dello storico

Lo storico operazioni è stato implementato per rendere tracciabili le azioni più importanti del sistema, come approvazioni, rifiuti, caricamenti di documenti, invii email, login, modifiche amministrative e generazione dei report mensili.

Ogni log contiene:

Campo	Significato
user_id	Utente che ha eseguito l'azione
level	Livello del log: INFO, WARNING, ERROR
action	Codice dell'azione eseguita
entity	Tipo di entità coinvolta
entity_id	ID dell'elemento coinvolto
payload	Dettagli aggiuntivi in formato JSON
ip	Indirizzo IP della richiesta
created_at	Data e ora dell'operazione

4.16.4 Struttura della tabella dei log

La tabella principale è operation_logs.

Codice della migration:

```
Schema::create('operation_logs', function (Blueprint $table) {
    $table->id();

    $table->foreignId('user_id')->nullable()->constrained('users')-
>nullOnDelete();
    $table->string('level');
    $table->string('action');
    $table->string('entity')->nullable(); // absence, delay, leave, user ecc
    $table->unsignedBigInteger('entity_id')->nullable();

    $table->json('payload')->nullable();
    $table->string('ip')->nullable();

    $table->timestamps();
});
```

È stato scelto il campo payload in formato JSON perché ogni operazione può avere dettagli diversi. Ad esempio, un'approvazione contiene informazioni diverse rispetto a un invio email o a un import utenti.

4.16.5 Registrazione di un'operazione

La registrazione dei log viene centralizzata nel metodo record del model OperationLog.

```
public static function record(
    ?User $user,
    string $action,
    ?string $entity = null,
    ?int $entityId = null,
    array $payload = [],
    string $level = 'INFO',
    ?Request $request = null
): void {
    $normalizedAction = trim($action);
    if (str_ends_with($normalizedAction, '.viewed')) {
        return;
    }

    static::query()->create([
        'user_id' => $user?->id,
        'level' => strtoupper(trim($level)),
        'action' => $normalizedAction,
        'entity' => $entity ? trim($entity) : null,
        'entity_id' => $entityId,
        'payload' => $payload,
        'ip' => $request?->ip(),
    ]);
}
```

Il metodo riceve l'utente, il tipo di azione, l'entità coinvolta e gli eventuali dettagli.
Le azioni che terminano con .viewed non vengono salvate, perché servono solo a indicare una visualizzazione e renderebbero lo storico troppo pieno di dati poco utili.

4.16.6 Esempio di log applicativo

Un esempio di log è l'approvazione di un report mensile.

```
OperationLog::record(
    null,
    'annual_hours.limit_reached.notified',
    'user',
    $studentId,
    [
        'student_id' => $studentId,
        'student_name' => $studentName,
        'total_hours' => $totalHours,
        'max_annual_hours' => $maxAnnualHours,
        'school_year' => $schoolYearLabel,
        'teachers_notified' => $teacherRecipients->count(),
        'guardians_notified' => $guardianRecipients->count(),
    ],
    'INFO'
);
```

4.16.7 Traduzione delle azioni

Nel model OperationLog sono presenti delle mappe che trasformano i codici tecnici in testi leggibili.

```
private const ACTION_LABELS = [
    'absence.request.created' => 'Creazione richiesta di assenza',
    'absence.updated' => 'Aggiornamento assenza',
    'absence.quick.updated' => 'Aggiornamento rapido assenza',
    'absence.approved' => 'Approvazione assenza',
    'absence.approved_without_guardian' => 'Approvazione assenza senza firma
tutore',
    'absence.rejected' => 'Rifiuto assenza',
```

Questa scelta permette di salvare nel database codici tecnici stabili, ma mostrare all'amministratore descrizioni più chiare.

4.16.8 Visualizzazione dei log

I log sono divisi in due aree principali:

Area	Livelli mostrati	Scopo
Storico interazioni	INFO	Operazioni normali degli utenti
Log errori	WARNING,ERROR	Errori, avvisi e problemi di sistema

Nel controller amministrativo vengono recuperati i log paginati.

```
$result = $log->getPaginatedLog(
    ['INFO'],
    50,
    $this->nullIfEmpty($filters['query']),
    $this->nullIfEmpty($filters['entity']),
    $this->nullIfEmpty($filters['action']),
    $this->nullIfEmpty($filters['date_from']),
    $this->nullIfEmpty($filters['date_to'])
```

Per i log di errore invece vengono usati i livelli ERROR e WARNING.

```
$allowedLevels = ['ERROR', 'WARNING'];
$levels = in_array($level, $allowedLevels, true)
    ? [$level]
    : $allowedLevels;

$result = $log->getPaginatedLog(
    $levels,
    50,
    $this->nullIfEmpty($filters['query']),
    $this->nullIfEmpty($filters['entity']),
    $this->nullIfEmpty($filters['action']),
    $this->nullIfEmpty($filters['date_from']),
    $this->nullIfEmpty($filters['date_to'])
);
```

4.16.9 Filtri di ricerca

Il model permette di filtrare i log per livello, entità, azione, data e testo cercato.

```
if ($levels) {
    $normalizedLevels = array_map('strtoupper', $levels);
    $query->whereIn('level', $normalizedLevels);
}

$normalizedEntity = strtolower(trim((string) $entity));
if ($normalizedEntity !== '') {
    $query->whereRaw('LOWER(entity) = ?', [$normalizedEntity]);
}

$normalizedAction = strtolower(trim((string) $action));
if ($normalizedAction !== '') {
    $query->whereRaw('LOWER(action) = ?', [$normalizedAction]);
}

$normalizedDateFrom = $this->normalizeDateInput($dateFrom);
if ($normalizedDateFrom) {
    $query->whereDate('created_at', '>=', $normalizedDateFrom);
}

$normalizedDateTo = $this->normalizeDateInput($dateTo);
if ($normalizedDateTo) {
    $query->whereDate('created_at', '<=', $normalizedDateTo);
}
```

Questi filtri rendono più semplice trovare operazioni specifiche, ad esempio tutte le approvazioni di assenze o tutti gli errori di invio email.

4.16.10 Notifica automatica agli amministratori

Quando viene creato un log di livello WARNING o ERROR, l'OperationLogObserver invia una notifica agli amministratori.

```
public function created(OperationLog $log): void
{
    $level = strtoupper(trim((string) $log->level));
    if (! in_array($level, ['WARNING', 'ERROR'], true)) {
        return;
    }

    $eventKey = $level === 'ERROR'
        ? 'admin_system_errors'
        : 'admin_system_warnings';
    $title = $level === 'ERROR'
        ? 'Errore di sistema'
        : 'Warning di sistema';
```

La notifica viene inviata tramite NotificationDispatcher.

```
$this->dispatcher->notifyUsers(
    $this->recipients->admins(),
    $eventKey,
    [
        'title' => $title,
        'body' => $body.'.',
        'url' => route('admin.error-logs'),
        'icon' => 'admin',
        'mail_subject' => $title,
    ]
);
```

In questo modo gli amministratori vengono avvisati quando il sistema registra un problema importante.

4.16.11 Esportazione dei log

I log possono essere esportati in formato CSV. L'esportazione usa gli stessi filtri della schermata di consultazione.

```
$result = $log->getExportRows(
    ['INFO'],
    $this->nullIfEmpty($filters['query']),
    $this->nullIfEmpty($filters['entity']),
    $this->nullIfEmpty($filters['action']),
    $this->nullIfEmpty($filters['date_from']),
    $this->nullIfEmpty($filters['date_to']),
    self::LOG_EXPORT_MAX_ROWS
);
```

Nel CSV vengono inserite le colonne principali.

```
fputcsv($handle, [
    'id',
    'livello',
    'livello_label',
    'attore',
    'azione',
    'azione_code',
    'entita',
    'entita_code',
    'ip',
    'created_at',
    'dettagli_json',
], ';');
```

Questa funzione permette di conservare o analizzare esternamente lo storico delle operazioni.

4.16.12 Retention e pulizia automatica

Per evitare che la tabella dei log cresca senza limite, è stata aggiunta una retention configurabile.

```
Schema::create('operation_log_settings', function (Blueprint $table) {
    $table->id();
    $table->unsignedInteger('interaction_retention_days')->default(425);
    $table->unsignedInteger('error_retention_days')->default(425);
    $table->timestamps();
});
```

La pulizia viene eseguita dal metodo pruneByConfiguredRetention.

```
$infoDeleted = (int) static::query()
    ->where('level', 'INFO')
    ->where('created_at', '<', $interactionCutoff)
    ->delete();

$errorDeleted = (int) static::query()
    ->whereIn('level', ['WARNING', 'ERROR'])
    ->where('created_at', '<', $errorCutoff)
    ->delete();
```

Il comando automatico associato è:

```
protected $signature = 'logs:prune-retention';

protected $description = 'Elimina i log operativi oltre la retention
configurata';
```

Questo comando viene pianificato nello scheduler ed elimina i log più vecchi in base alle impostazioni configurate.

5 Test

5.1 Protocollo di test

Test Case: **Riferimento**:	TC-001 REQ-01.001	**Nome:**	Login utente con credenziali valide e non valide
Descrizione:	Verificare che la maschera di login permetta l'accesso solo con credenziali corrette		
Prerequisiti:	Database demo caricato. Browser non autenticato.		
Procedura:	1. Aprire la pagina http://127.0.0.1:8000/login. 2. Inserire email alan.gregorio@example.com e password errata PasswordErrata. 3. Confermare il login. 4. Inserire email alan.gregorio@example.com e password Admin$00. 5. Confermare il login.		
Risultati attesi:	• con password errata il sistema mostra un errore e resta sulla pagina di login; • con password corretta l'utente accede alla dashboard; • il nome/ruolo dell'utente autenticato e coerente con l'account usato.		

Test Case: **Riferimento**:	TC-002 REQ-01.002	**Nome:**	Logout utente
Descrizione:	Verificare che l'utente possa terminare la sessione.		
Prerequisiti:	Utente allievo autenticato.		
Procedura:	1. Accedere con alan.gregorio@example.com. 2. Aprire il menu utente. 3. Cliccare Logout. 4. Tentare di aprire /dashboard.		
Risultati attesi:	• dopo il logout l'utente viene riportato alla pagina di login; • l'accesso diretto a /dashboard non e consentito senza nuova autenticazione.		

Test Case: **Riferimento**:	TC-003 REQ-01.003	**Nome:**	Recupero password via email
Descrizione:	Verificare l'invio del link di recupero password.		
Prerequisiti:	Email configurata su log o SMTP di test. Browser non autenticato.		
Procedura:	1. Aprire /forgot-password o cliccare password dimenticata. 2. Inserire alan.gregorio@example.com. 3. Inviare la richiesta. 4. Controllare il log email o la casella di test. 5. Aprire il link di reset ricevuto. 6. Impostare una nuova password valida. 7. Effettuare login con la nuova password.		
Risultati attesi:	• il sistema conferma l'invio della mail; • viene generato un link di reset valido; • la password viene aggiornata; • il login funziona con la nuova password e non con quella precedente.		

Test Case: **Riferimento**:	TC-004 REQ-02.001-04	**Nome:**	Accesso differenziato per ruolo
Descrizione:	Verificare che allievo, docente, capo laboratorio e amministratore vedano solo le funzioni previste dal proprio ruolo.		
Prerequisiti:	Database demo caricato.		
Procedura:	1. Accedere come allievo alan.gregorio@example.com. 2. Verificare la dashboard allievo e tentare di aprire /admin/utenti. 3. Uscire. 4. Accedere come docente paolo.rossi@example.com. 5. Verificare la dashboard docente e tentare di aprire /admin/configurazione. 6. Uscire. 7. Accedere come capo laboratorio luca.galli@example.com. 8. Verificare la dashboard capo laboratorio e tentare di aprire /admin/utenti. 9. Uscire. 10. Accedere come admin admin@example.com. 11. Verificare l'accesso alle pagine amministrative.		
Risultati attesi:	• ogni ruolo vede le voci di menu corrette; • le pagine non autorizzate sono bloccate o reindirizzate; • l'amministratore accede alla gestione utenti, classi e configurazione.		

Test Case: **Riferimento**:	TC-005 REQ-03.001-04	**Nome:**	Dashboard allievo
Descrizione:	Verificare che l'allievo visualizzi riepilogo assenze, ore nelle 40 ore, ritardi e stato richieste.		
Prerequisiti:	Utente allievo autenticato. Database demo caricato.		
Procedura:	1. Accedere come alan.gregorio@example.com. 2. Aprire /dashboard. 3. Verificare il totale ore di assenza. 4. Verificare le ore conteggiate nelle 40 ore. 5. Verificare il numero di ritardi. 6. Verificare la lista/stato delle richieste: in attesa, approvate, rifiutate o arbitrarie. 7. Aprire lo storico allievo.		
Risultati attesi:	• i dati sono visibili nella dashboard; • i valori della dashboard sono coerenti con lo storico dell'allievo; • ogni richiesta mostra uno stato comprensibile.		

Test Case: **Riferimento**:	TC-006 REQ-04.001-04	**Nome:**	Dashboard docente di classe
Descrizione:	Verificare che il docente visualizzi dati per allievo, ritardi, 40 ore, ricerca e filtri.		
Prerequisiti:	Utente docente autenticato. Database demo caricato.		
Procedura:	1. Accedere come paolo.rossi@example.com. 2. Aprire /dashboard. 3. Verificare l'elenco degli allievi assegnati. 4. Aprire il dettaglio di un allievo. 5. Verificare assenze totali, ritardi e stato 40 ore. 6. Usare ricerca per cognome Gregorio. 7. Applicare un filtro disponibile, ad esempio classe o stato.		
Risultati attesi:	• il docente vede solo gli allievi delle proprie classi; • il riepilogo per allievo e coerente con i dettagli; • ricerca e filtri restringono correttamente i risultati.		

Test Case: **Riferimento**:	TC-007 REQ-05.001-02	**Nome:**	Dashboard capo laboratorio e gestione congedi
Descrizione:	Verificare che il capo laboratorio visualizzi e gestisca le richieste di congedo.		
Prerequisiti:	Utente capo laboratorio autenticato. Esiste almeno una richiesta di congedo in attesa.		
Procedura:	1. Accedere come luca.galli@example.com. 2. Aprire /laboratorio/congedi. 3. Selezionare una richiesta in attesa. 4. Aprire il dettaglio. 5. Inserire un commento di gestione. 6. Eseguire una decisione prevista dal flusso: pre-approvazione, approvazione, rifiuto o richiesta documentazione.		
Risultati attesi:	• la lista congedi e visibile; • il dettaglio mostra allievo, periodo, motivazione, documenti e stato; • la decisione aggiorna lo stato della richiesta; • l'operazione viene registrata nello storico.		

Test Case: **Riferimento**:	TC-008 REQ-06.001-02	**Nome:**	Segnalazione assenza da parte dell'allievo
Descrizione:	Verificare l'inserimento di un'assenza e la possibilità di allegare certificati.		
Prerequisiti:	Utente allievo autenticato. File PDF di test disponibile: certificato_test.pdf.		
Procedura:	1. Accedere come alan.gregorio@example.com. 2. Aprire /studente/assenze/nuova. 3. Inserire un'assenza con date valide, ad esempio dal 04.05.2026 al 04.05.2026. 4. Selezionare una motivazione valida. 5. Salvare la richiesta. 6. Aprire lo storico o la dashboard. 7. Se il sistema richiede certificato, caricare certificato_test.pdf.		
Risultati attesi:	• la richiesta viene salvata; • l'assenza compare nello storico dell'allievo; • lo stato iniziale e coerente con il flusso previsto; • il certificato viene accettato se formato e dimensione sono validi.		

Test Case: **Riferimento**:	TC-009 REQ-07.001-03	**Nome:**	Firma tutore per allievo minorenne
Descrizione:	Verificare invio email, link sicuro con token e firma grafica del tutore.		
Prerequisiti:	Allievo minorenne con tutore associato. Email configurata su log o SMTP di test. Esiste una nuova assenza o un nuovo ritardo da firmare.		
Procedura:	1. Creare una richiesta di assenza o ritardo come allievo minorenne. 2. Controllare nel log email o nella casella del tutore la presenza del link di firma. 3. Aprire il link in browser non autenticato. 4. Disegnare la firma nell'area grafica. 5. Confermare la firma. 6. Aprire nuovamente lo stesso link.		
Risultati attesi:	• il tutore riceve una email automatica; • il link contiene un token non prevedibile; • il form di firma e accessibile senza login ma solo tramite token valido; • la firma viene salvata; • il token gia usato non permette una seconda firma.		

Test Case: **Riferimento**:	TC-010 REQ-08.001-02	**Nome:**	Countdown giustificazione e notifica scadenza
Descrizione:	Verificare che l'allievo veda la scadenza e riceva notifica prima della scadenza.		
Prerequisiti:	Esiste un'assenza o ritardo con scadenza futura. Scheduler e queue worker attivi.		
Procedura:	1. Accedere come allievo. 2. Aprire la dashboard. 3. Verificare la presenza del countdown o della data limite di giustificazione. 4. Portare la richiesta vicino alla scadenza in ambiente di test, tramite configurazione regole o aggiornamento controllato del dato. 5. Eseguire lo scheduler: php artisan schedule:run 6. Verificare notifiche interne ed email.		
Risultati attesi:	• la scadenza e visibile all'allievo; • il countdown diminuisce coerentemente con la data limite; • prima della scadenza viene generata una notifica; • la notifica contiene riferimento alla richiesta corretta.		

Test Case: **Riferimento**:	TC-011 REQ-09.001-02	**Nome:**	Upload e associazione certificato medico
Descrizione:	Verificare che un certificato medico venga caricato e associato all'assenza corretta.		
Prerequisiti:	Esiste un'assenza che richiede certificato. File certificato_test.pdf disponibile.		
Procedura:	1. Accedere come allievo. 2. Aprire la richiesta di assenza interessata. 3. Caricare certificato_test.pdf. 4. Accedere come docente. 5. Aprire il dettaglio dell'assenza. 6. Visualizzare o scaricare il certificato.		
Risultati attesi:	• il file viene salvato senza errori; • il certificato risulta collegato alla sola assenza selezionata; • il docente puo visualizzarlo/scaricarlo; • file non ammessi o troppo grandi vengono rifiutati.		

Test Case: **Riferimento**:	TC-012 REQ-10.001-03	**Nome:**	Validazione assenza da parte del docente
Descrizione:	Verificare giustificazione, osservazioni e approvazione anche senza firma tutore con commento.		
Prerequisiti:	Esiste un'assenza in attesa associata a un allievo del docente.		
Procedura:	1. Accedere come paolo.rossi@example.com. 2. Aprire il dettaglio dell'assenza. 3. Inserire ore assegnate e osservazioni. 4. Approvare l'assenza. 5. Creare o selezionare un'altra assenza senza firma tutore. 6. Tentare l'approvazione senza firma lasciando vuoto il commento. 7. Ripetere inserendo un commento motivato.		
Risultati attesi:	• il docente puo giustificare l'assenza; • le osservazioni vengono salvate; • l'approvazione senza firma viene bloccata se manca il commento; • con commento valido l'approvazione senza firma viene registrata; • lo storico riporta la decisione.		

Test Case: **Riferimento**:	TC-013 REQ-11.001-02	**Nome:**	Cambio automatico a stato arbitraria
Descrizione:	Verificare che una richiesta non giustificata entro scadenza diventi arbitraria e generi notifiche.		
Prerequisiti:	Esiste un'assenza scaduta non giustificata. Scheduler attivo.		
Procedura:	1. Creare un'assenza come allievo senza completare la giustificazione richiesta. 2. In ambiente di test impostare la scadenza a una data precedente alla data di riferimento. 3. Eseguire: php artisan absences:mark-arbitrary 4. Accedere come allievo e controllare lo stato dell'assenza. 5. Accedere come docente e controllare notifiche e storico.		
Risultati attesi:	• l'assenza passa automaticamente allo stato Arbitraria; • allievo e docente ricevono notifica; • il cambio stato e tracciato; • richieste gia giustificate non vengono modificate.		

Test Case: **Riferimento**:	TC-014 REQ-12.001-02	**Nome:**	Segnalazione ritardo e applicazione regole
Descrizione:	Verificare inserimento durata ritardo e rispetto automatico delle regole configurate.		
Prerequisiti:	Utente allievo autenticato. Regole ritardi presenti in configurazione.		
Procedura:	1. Accedere come alan.gregorio@example.com. 2. Aprire /studente/ritardi/nuovo. 3. Inserire data/ora valida e durata, ad esempio 15 minuti. 4. Salvare il ritardo. 5. Accedere come docente. 6. Aprire il dettaglio del ritardo. 7. Registrare o validare il ritardo secondo il flusso previsto. 8. Verificare se il ritardo viene conteggiato nel semestre o escluso secondo regola.		
Risultati attesi:	• il ritardo viene salvato con durata corretta; • il docente lo vede tra gli elementi da gestire; • le regole configurate vengono applicate automaticamente; • eventuali notifiche/regole di recupero vengono generate solo quando previsto.		

Test Case: **Riferimento**:	TC-015 REQ-13.001-03	**Nome:**	Creazione, approvazione e rifiuto congedo
Descrizione:	Verificare il flusso completo delle richieste di congedo.		
Prerequisiti:	Utente allievo, docente/capo laboratorio e tutore disponibili.		
Procedura:	1. Accedere come allievo. 2. Aprire /studente/congedi/nuovo. 3. Creare una richiesta di congedo con date future, motivo e ore richieste. 4. Verificare invio eventuale firma tutore. 5. Accedere come capo laboratorio. 6. Aprire la richiesta. 7. Rifiutare una copia o una richiesta di test inserendo un commento. 8. Creare una seconda richiesta valida. 9. Approvare la seconda richiesta.		
Risultati attesi:	• la richiesta viene creata; • il rifiuto richiede e salva un commento; • l'approvazione aggiorna lo stato; • lo storico mostra creazione, rifiuto/approvazione e attore; • se previsto, il congedo approvato genera l'assenza collegata.		

Test Case: **Riferimento**:	TC-016 REQ-14.001-03	**Nome:**	Gestione classi e assegnazioni
Descrizione:	Verificare creazione/importazione classi, assegnazione studenti e docente di classe.		
Prerequisiti:	Utente admin autenticato.		
Procedura:	1. Accedere come admin@example.com. 2. Aprire /admin/classi. 3. Creare una nuova classe di test, ad esempio TEST 1A. 4. Modificare la classe appena creata. 5. Assegnare o verificare uno studente associato alla classe. 6. Assegnare o verificare un docente di classe. 7. Aprire la dashboard docente con l'account assegnato.		
Risultati attesi:	• la classe viene creata e modificata; • gli studenti risultano associati alla classe corretta; • il docente vede la classe assegnata; • l'amministratore puo filtrare/ricercare le classi.		

Test Case: **Riferimento**:	TC-017 REQ-15.001-03	**Nome:**	Creazione automatica utenti da import
Descrizione:	Verificare che l'import crei account studenti, assegni la classe e generi credenziali iniziali tramite email di reset.		
Prerequisiti:	Utente admin autenticato. File CSV/XLSX GAGI di test disponibile.		
Procedura:	1. Accedere come admin. 2. Aprire /admin/allievi/aggiunta. 3. Selezionare il file CSV/XLSX di test. 4. Caricare il file. 5. Aprire /admin/utenti. 6. Cercare mario.rossi@student.example.com. 7. Verificare ruolo, classe e stato. 8. Controllare l'email di impostazione password nel log o nella casella di test.		
Risultati attesi:	• viene creato un utente studente; • l'email viene normalizzata con dominio student.* se necessario; • la classe viene creata/associata; • viene inviata l'email per impostare la password iniziale; • l'import e registrato nello storico.		

Test Case: **Riferimento**:	TC-018 REQ-16.001-03	**Nome:**	Storiche operazioni
Descrizione:	Verificare tracciamento approvazioni, eccezioni, commenti e decisioni.		
Prerequisiti:	Utente admin autenticato. Eseguire almeno una decisione su assenza, ritardo o congedo.		
Procedura:	1. Eseguire una decisione, ad esempio approvare un'assenza con commento. 2. Accedere come admin. 3. Aprire /admin/storico-interazioni. 4. Cercare l'operazione appena eseguita. 5. Applicare filtro per livello o azione. 6. Aprire /admin/log-errori. 7. Verificare la presenza degli errori/warning demo o generati in test.		
Risultati attesi:	• le approvazioni sono tracciate con attore, data, entita e azione; • i commenti/decisioni sono consultabili nei dettagli; • errori e warning sono separati dallo storico interazioni; • ricerca e filtri funzionano.		

Test Case: **Riferimento**:	TC-019 REQ-17.001-02	**Nome:**	Dashboard amministrativa
Descrizione:	Verificare riepilogo generale, stato sistema e gestione utenti.		
Prerequisiti:	Utente admin autenticato.		
Procedura:	1. Accedere come admin@example.com. 2. Aprire /dashboard. 3. Verificare riepilogo utenti, classi, docenti ed errori. 4. Aprire /admin/log-errori. 5. Aprire /admin/utenti. 6. Cercare un utente. 7. Modificare un dato consentito di un utente di test. 8. Disattivare e riattivare un utente di test.		
Risultati attesi:	• il riepilogo generale e visibile; • errori e warning sono consultabili; • la gestione utenti permette ricerca, modifica e cambio stato; • il sistema impedisce operazioni pericolose, ad esempio eliminare/disattivare se stessi.		

Test Case: **Riferimento**:	TC-020 REQ-18.001-02	**Nome:**	Notifiche email e reinvio manuale
Descrizione:	Verificare invio automatico email sugli eventi rilevanti e reinvio manuale.		
Prerequisiti:	Email configurata su log o SMTP di test. Esiste una richiesta con email inviata o inviabile.		
Procedura:	1. Creare un'assenza o un ritardo che richiede firma tutore. 2. Controllare che venga generata email. 3. Accedere come docente o capo laboratorio. 4. Aprire il dettaglio della richiesta. 5. Usare l'azione Reinvia firma o equivalente. 6. Controllare il log email o la casella di test. 7. Aprire il centro notifiche interno e segnare una notifica come letta.		
Risultati attesi:	• l'email automatica viene inviata; • il reinvio manuale genera una nuova email valida; • le notifiche interne sono visibili; • una notifica marcata come letta non resta tra quelle non lette.		

Test Case: **Riferimento**:	TC-021 REQ-19.001-02	**Nome:**	Invio report mensile al tutore e approvazione
Descrizione:	Verificare generazione report mensile, invio email, caricamento report firmato e approvazione docente.		
Prerequisiti:	Scheduler/queue attivi. Allievo con tutore e dati mensili disponibili.		
Procedura:	1. Eseguire la generazione report: php artisan reports:generate-monthly 2. Controllare email/log del tutore. 3. Accedere come allievo. 4. Aprire /studente/report-mensili. 5. Scaricare il report generato. 6. Caricare un PDF firmato di test. 7. Accedere come docente. 8. Aprire /docente/report-mensili. 9. Aprire il dettaglio del report. 10. Approvare il report firmato.		
Risultati attesi:	• il report PDF viene generato; • l'email al tutore viene inviata; • l'allievo puo scaricare il report; • l'allievo puo caricare il PDF firmato; • il docente puo approvare il report; • lo stato passa attraverso generated/sent/signed/approved o stati equivalenti.		

Test Case: **Riferimento**:	TC-022 REQ-20.001-02	**Nome:**	Impostazioni regole
Descrizione:	Verificare configurazione delle 40 ore, regole ritardi e regole custom.		
Prerequisiti:	Utente admin autenticato.		
Procedura:	1. Accedere come admin. 2. Aprire /admin/configurazione. 3. Modificare il limite ore annue, ad esempio da 40 a 45. 4. Salvare. 5. Modificare o creare una regola ritardi custom. 6. Salvare. 7. Creare un'assenza o ritardo che ricade nella regola modificata. 8. Verificare dashboard e dettaglio richiesta.		
Risultati attesi:	• le impostazioni vengono salvate;		

	• valori fuori range o non validi vengono rifiutati; • le nuove regole influenzano i nuovi calcoli; • le modifiche vengono registrate nello storico.

Test Case: **Riferimento**:	TC-023 REQ-21.001-02	**Nome:**	Gestione documenti
Descrizione:	Verificare visualizzazione e download dei documenti caricati.		
Prerequisiti:	Esiste almeno un certificato medico o documento di congedo caricato.		
Procedura:	1. Accedere come allievo. 2. Aprire /studente/documenti. 3. Verificare la lista dei certificati/documenti disponibili. 4. Scaricare un documento. 5. Accedere come docente. 6. Aprire il dettaglio della richiesta collegata. 7. Visualizzare/scaricare lo stesso documento. 8. Tentare l'accesso al documento con un utente non autorizzato.		
Risultati attesi:	• i documenti caricati sono elencati; • il download funziona; • il documento corrisponde alla richiesta corretta; • utenti non autorizzati non possono accedere ai file.		

Test Case: **Riferimento**:	TC-024 REQ-22.001-03	**Nome:**	Esportazione dati
Descrizione:	Verificare esportazione elenco assenze, ritardi e report per classe.		
Prerequisiti:	Utente docente o admin autenticato. Esistono dati di assenze, ritardi e report.		
Procedura:	1. Accedere come docente. 2. Aprire la pagina studenti o profilo allievo. 3. Esportare i dati selezionando sezioni assenze e ritardi. 4. Aprire il file scaricato e verificare il contenuto. 5. Aprire /docente/report-mensili. 6. Filtrare per classe/mese. 7. Scaricare un report disponibile. 8. Accedere come admin. 9. Aprire export log errori o storico interazioni e scaricare il CSV.		

Risultati attesi:	• l'export contiene gli allievi/dati filtrati; • assenze e ritardi sono presenti con stato, date e ore/minuti; • i report per classe/mese sono scaricabili; • i CSV amministrativi rispettano i filtri applicati.

Test Case: **Riferimento**:	TC-025 REQ-23.001-05	**Nome:**	Gestione calendario scolastico e vacanze
Descrizione:	Verificare importazione calendario da PDF, estrazione automatica delle vacanze, salvataggio nel sistema e correzione manuale delle date importate.		
Prerequisiti:	Utente admin autenticato. File PDF calendario scolastico di test disponibile. Ambiente con parser PDF configurato.		
Procedura:	1. Accedere come admin@example.com. 2. Aprire /admin/configurazione. 3. Nella sezione vacanze, importare il PDF calendario scolastico. 4. Verificare che il sistema estragga automaticamente le date di vacanza e le mostri nell'elenco. 5. Controllare che le date importate siano salvate nel sistema con origine importata da PDF. 6. Modificare manualmente una data importata, cambiando ad esempio data o etichetta. 7. Salvare la modifica. 8. Eliminare una data errata o non desiderata. 9. Inserire manualmente una nuova data di vacanza. 10. Verificare che le nuove date siano considerate nei calcoli che escludono i giorni di vacanza, ad esempio nella scadenza certificati o nel preavviso congedi. 11. Aprire export log errori o storico interazioni e scaricare il CSV.		
Risultati attesi:	• il PDF viene accettato e processato senza errori; • le date di vacanza vengono estratte automaticamente; • le date estratte vengono salvate nel database; • le date importate possono essere corrette manualmente; • una data errata puo essere eliminata; • una nuova data puo essere inserita manualmente; • le vacanze salvate influenzano correttamente le regole applicative che escludono giorni non scolastici; • lo storico operazioni registra importazione, modifica e cancellazione.		

Test Case: **Riferimento**:	TC-026 REQ-24.001-05	**Nome:**	Notifiche interne applicative
Descrizione:	Verificare visualizzazione notifiche non lette, conteggio, collegamento rapido all'evento, aggiornamento stato letto e distribuzione corretta per ruolo.		
Prerequisiti:	Esiste almeno un evento notificabile, ad esempio nuova assenza da verificare o richiesta documentazione congedo. Utenti allievo, docente e admin disponibili.		
Procedura:	1. Generare un evento notificabile, ad esempio: - creare un'assenza come allievo per notificare il docente; - oppure richiedere documentazione su un congedo per notificare l'allievo. 2. Accedere con l'utente destinatario della notifica. 3. Aprire la dashboard o il menu notifiche. 4. Verificare la presenza della notifica tra quelle non lette. 5. Controllare il contatore notifiche non lette. 6. Aprire la notifica dal collegamento rapido. 7. Verificare il reindirizzamento alla schermata corretta collegata all'evento. 8. Tornare all'elenco notifiche e segnare la notifica come letta. 9. Se disponibile, usare anche l'azione per segnare tutte come lette. 10. Accedere con un ruolo non coinvolto nello stesso evento e verificare che non riceva quella notifica.		
Risultati attesi:	• la notifica compare per il ruolo corretto; • il contatore delle notifiche non lette aumenta correttamente; • il link rapido porta alla schermata giusta, ad esempio dettaglio assenza, sezione documenti o report; • dopo l'apertura o marcatura manuale la notifica risulta letta; • il contatore delle non lette diminuisce coerentemente; • i ruoli non coinvolti non ricevono notifiche non pertinenti; • il contenuto della notifica identifica chiaramente evento e azione richiesta.		

Test Case: **Riferimento**:	TC-027 REQ-25.001-05	**Nome:**	Archiviazione documentale per studente
Descrizione:	Verificare che i documenti dello studente siano salvati in modo strutturato, privato e recuperabile per categoria e autorizzazione.		
Prerequisiti:	Nota di tracciabilita: il requisito originale aveva ID duplicato REQ-24; viene trattato come REQ-25. Esiste almeno uno studente con documenti caricati: certificato medico, documentazione congedo, avatar profilo o report mensile firmato.		
Procedura:	1. Accedere come allievo e caricare almeno due documenti di categorie diverse, ad esempio: - certificato medico su un'assenza; - documentazione su un congedo; - report mensile firmato; - avatar profilo. 2. Verificare che ogni documento risulti collegato all'entita corretta. 3. Aprire /studente/documenti e controllare che i documenti siano elencati con categoria o contesto riconoscibile. 4. Scaricare un documento dall'interfaccia utente. 5. Accedere come docente o capo laboratorio, aprire la richiesta collegata e scaricare o visualizzare il documento autorizzato. 6. Tentare l'accesso allo stesso file con un utente non autorizzato. 7. Verificare, tramite controllo applicativo o evidenza tecnica, che il salvataggio avvenga in area privata/archivio dello studente e non in percorso pubblico generico. 8. Se presente una sostituzione documento, caricare una nuova versione e verificare il recupero del file aggiornato.		
Risultati attesi:	• Viene creata la cartella correttamente, • i file vengono divisi correttamente nelle apposite cartelle con gli appositi nomi		

5.2 Risultati test

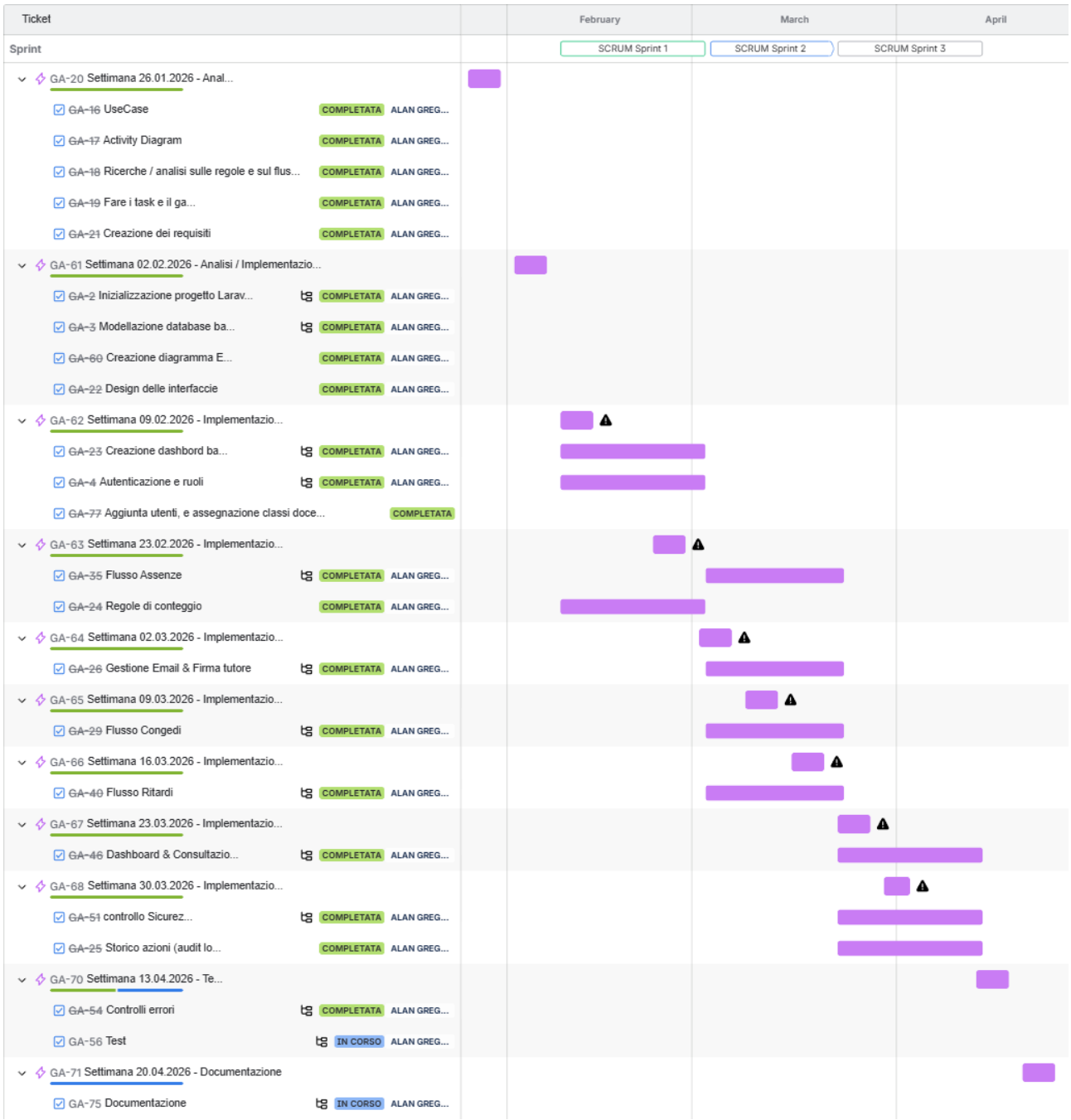
TC-001	22.04.2025	**Passato**
TC-002	22.04.2025	**Passato**
TC-003	22.04.2025	**Passato**
TC-004	22.04.2025	**Passato**
TC-005	22.04.2025	**Passato**
TC-006	22.04.2025	**Passato**
TC-007	22.04.2025	**Passato**
TC-008	22.04.2025	**Passato**
TC-009	22.04.2025	**Passato**
TC-010	22.04.2025	**Passato**
TC-011	22.04.2025	**Passato**
TC-012	22.04.2025	**Passato**
TC-013	22.04.2025	**Passato**
TC-014	22.04.2025	**Passato**
TC-015	22.04.2025	**Passato**
TC-016	22.04.2025	**Passato**
TC-017	22.04.2025	**Passato**
TC-018	22.04.2025	**Passato**
TC-019	22.04.2025	**Passato**
TC-020	22.04.2025	**Passato**
TC-021	22.04.2025	**Passato**
TC-022	22.04.2025	**Passato**
TC-023	22.04.2025	**Passato**
TC-024	22.04.2025	**Passato**
TC-025	22.04.2025	**Passato**
TC-026	22.04.2025	**Passato**
TC-027	22.04.2025	**Passato**
Passati:		**27**
Falliti:		**0**
Parzialmente Passato:		**0**

I risultati dei test case nel dettaglio si trovano in: **\3_Documentazione\TestCase.docx** o nel capitolo 10 Allegati

5.3 Mancanze/limitazioni conosciute

Non risultano elementi mancanti o parzialmente implementati rispetto ai requisiti previsti dal progetto, al di fuori dei test case. Tutte le funzionalità richieste sono state completate e integrate nella versione finale.

6 Consuntivo



Il diagramma di Gantt consuntivo mostra l'andamento reale del progetto, evidenziando le attività effettivamente svolte e il loro stato di avanzamento.
Rispetto al Gantt preventivo, alcune fasi di implementazione sono state prolungate e in parte sovrapposte. In particolare, attività come autenticazione e ruoli, dashboard, flusso assenze, gestione email e firma tutore, congedi e ritardi hanno richiesto più tempo del previsto. Anche le funzionalità di consultazione, storiche azioni e controlli di sicurezza sono state distribuite su più settimane.
Il consuntivo evidenzia quindi uno slittamento di alcune attività rispetto alla pianificazione iniziale, ma mostra anche che la maggior parte dei task principali è stata completata.

7 Conclusioni

La soluzione sviluppata ha un impatto concreto e rilevante nel contesto scolastico in cui verrà adottata, poiché sostituisce la gestione cartacea di assenze, congedi e ritardi con un sistema digitale centralizzato. Questo comporta una semplificazione delle procedure, una riduzione del rischio di errori o dimenticanze e un miglioramento nella tracciabilità delle informazioni.

L'applicativo aiuta anche i capi laboratorio e ai docenti di classe, facilitando il monitoraggio delle situazioni degli allievi e rendendo più immediata la consultazione dei dati. Anche per gli studenti il sistema rappresenta un vantaggio, in quanto aiuta a ricordare firme, giustificazioni e altre cose collegate alle assenze.

Il risultato ottenuto può quindi essere considerato significativo per la realtà scolastica di riferimento. Non si tratta di una soluzione con impatto generale o universale, ma di un applicativo pensato per rispondere in modo efficace a esigenze concrete e specifiche della scuola. Tuttavia, con opportuni adattamenti, l'approccio potrebbe essere esteso anche ad altri contesti scolastici con necessità simili.

Dal punto di vista qualitativo, l'aspetto grafico può ancora essere migliorato, ma durante lo sviluppo è stata data priorità alla correttezza funzionale e all'utilità pratica del sistema. Questa scelta ha permesso di ottenere un applicativo solido e utilizzabile, che risponde in modo efficace agli obiettivi principali del progetto.

7.1 Sviluppi futuri

Tra le principali estensioni future del prodotto vi è l'introduzione di un sistema di archiviazione annuale dei dati. Attualmente l'applicativo è pensato per essere utilizzato su un singolo anno scolastico; per un impiego continuativo negli anni successivi sarebbe utile implementare una funzione che permetta di chiudere l'anno corrente, archiviare i dati in modo consultabile e avviare un nuovo anno mantenendo disponibile lo storico degli studenti. Questa estensione renderebbe il sistema più completo e adatto a un utilizzo stabile nel tempo.

Un ulteriore miglioramento riguarda l'aspetto grafico e l'esperienza d'uso. L'interfaccia attuale risulta funzionale e utilizzabile, ma potrebbe essere resa più chiara, moderna e intuitiva. Un miglioramento grafico contribuirebbe a rendere l'applicativo più piacevole da usare e a facilitare ulteriormente l'interazione da parte di docenti, responsabili e studenti.

Un'altra possibile estensione riguarda la gestione di casi particolari legati alla figura del tutore. In particolare, potrebbe essere utile prevedere la possibilità di mantenere associato un tutore anche nel caso di uno studente maggiorenne, quando ciò risulti necessario per situazioni specifiche. Questa funzionalità permetterebbe di gestire in modo più flessibile casi reali che non rientrano nel flusso standard.

7.2 Considerazioni personali

Questo progetto mi ha permesso di sviluppare competenze sia tecniche sia organizzative. In particolare, ho imparato a interfacciarmi in modo più efficace con i committenti, comprendendo meglio le loro necessità e traducendole in requisiti concreti per l'applicativo. Ho inoltre migliorato la mia capacità di analizzare i flussi del sistema, ragionando con maggiore attenzione sulla struttura delle funzionalità e sul modo più corretto di implementarle.

Dal punto di vista tecnico, il progetto mi ha consentito di consolidare le mie conoscenze di Laravel, React e Tailwind CSS, applicandole in un contesto reale e orientato a bisogni concreti. Un altro aspetto importante che ho imparato riguarda la gestione delle priorità: durante lo sviluppo ho capito quanto sia fondamentale concentrarsi prima sui requisiti essenziali e sulla solidità funzionale del prodotto, rimandando eventuali miglioramenti secondari a una fase successiva.

8 Glossario

Termine	Descrizione
Backend	Parte dell'applicazione che gestisce la logica del sistema, l'accesso ai dati e le regole applicative**.**
Controller	Componente del backend che riceve una richiesta, esegue la logica necessaria e restituisce una risposta.
CSV	**Comma-Separated Values:** Formato di file testuale usato per importare o esportare dati tabellari.
Database	Sistema usato per memorizzare in modo strutturato i dati dell'applicazione.
Frontend	Parte visibile dell'applicazione con cui l'utente interagisce direttamente.
Hash	Valore generato a partire da un dato originale, usato per memorizzare informazioni sensibili in modo più sicuro.
Inertia.js	Libreria che collega Laravel e React permettendo di costruire un'applicazione moderna senza creare un'API separata.
Job	Attività eseguita in modo asincrono dal sistema, utile per operazioni come invii email o generazione report.
JSON	**JavaScript Object Notation:** Formato testuale strutturato usato per scambiare dati tra componenti software.
Laravel	Framework PHP usato per sviluppare il backend dell'applicazione.
Middleware	Componente che intercetta una richiesta prima che arrivi al controller, ad esempio per controllare autenticazione e permessi.
MySQL	Sistema di gestione di database relazionali usato nel progetto.
PDF	**Portable Document** Format: Formato di documento usato per report, certificati e file importati nel sistema.
PHP	**Hypertext Preprocessor:** Linguaggio di programmazione usato per sviluppare la parte server dell'applicazione.
Queue	Sistema che permette di gestire operazioni in coda, eseguite in modo differito o asincrono.
React	Libreria JavaScript usata per realizzare l'interfaccia utente del progetto.
Route	Collegamento tra un URL dell'applicazione e una specifica funzione del backend.
Scheduler	Sistema che esegue automaticamente comandi o operazioni pianificate a determinati orari.
Tailwind CSS	Framework CSS usato per costruire rapidamente l'aspetto grafico dell'interfaccia.
Token	Codice univoco e sicuro usato per identificare una richiesta o autorizzare un'azione, ad esempio la firma del tutore tramite link.

9 Bibliografia

9.1 Sitografia

- https://laravel.com/docs/12.x/documentation, Laravel 12.x Documentation, 28-01-2026.
- https://laravel.com/docs/12.x/routing, Routing - Laravel 12.x, 02-02-2026.
- https://laravel.com/docs/12.x/middleware, Middleware - Laravel 12.x, 03-02-2026.
- https://laravel.com/docs/12.x/controllers, Controllers - Laravel 12.x, 04-02-2026.
- https://laravel.com/docs/12.x/validation, Validation - Laravel 12.x, 06-02-2026.
- https://laravel.com/docs/12.x/authentication, Authentication - Laravel 12.x, 06-02-2026.
- https://laravel.com/docs/12.x/notifications, Notifications - Laravel 12.x, 03-02-2026.
- https://laravel.com/docs/12.x/mail, Mail - Laravel 12.x, 18-03-2026.
- https://laravel.com/docs/12.x/filesystem, Filesystem - Laravel 12.x, 17-03-2026.
- https://laravel.com/docs/12.x/queues, Queues - Laravel 12.x, 16-03-2026.
- https://laravel.com/docs/12.x/scheduling, Task Scheduling - Laravel 12.x, 20-03-2026.
- https://laravel.com/docs/12.x/structure, Directory Structure - Laravel 12.x, 28-01-2026.
- https://inertiajs.com/docs, Introduction - Inertia.js Documentation, 03-02-2026.
- https://react.dev/, React, 03-02-2026.
- https://react.dev/learn, Learn React, 24-04-2026.
- https://react.dev/reference/react, React Reference Overview, 04-02-2026.
- https://tailwindcss.com/, Tailwind CSS, 04-02-2026.
- https://tailwindcss.com/docs/installation/framework-guides, Get started with Tailwind CSS, 04-02-2026.
- https://vite.dev/guide/, Getting Started | Vite, 04-02-2026.
- https://www.php.net/manual/, PHP Manual, 10-02-2026.
- https://dev.mysql.com/doc/refman/8.0/en/index.html, MySQL 8.0 Reference Manual, 11-02-2026.
- https://pypdf.readthedocs.io/, Welcome to pypdf, 20-04-2026.
- https://docs.sheetjs.com/docs/, Overview | SheetJS Community Edition, 21-04-2026.
- https://www.figma.com/, Figma: The Collaborative Interface Design Tool, 28-01-2026.
- https://docs.docker.com/, Docker Docs, 22-04-2026.

10 Allegati

TestCase specifico:

TestCase.pdf

DiagrammaER:

DiagrammaER.png

Credenziali email:

account_gmail.txt

Qdc progetto:

QdC_Progetto_Assenze.pdf